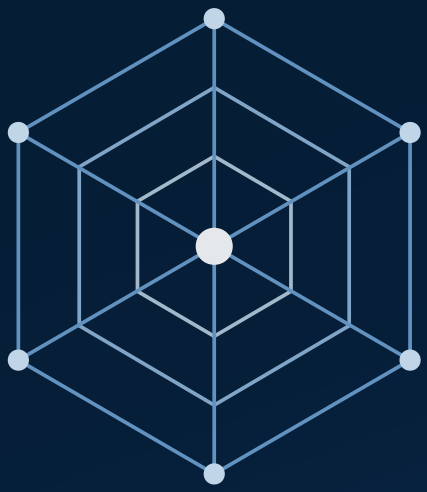
VOLUME 1 OF 6



THE COMPLETE SNOWFLAKE LEARNING BIBLE

Zero to Expert | Snowflake Core | SQL | Data Engineering | Architecture | Streaming | Governance | Cortex AI | RAG | Agents | MCP | Migration | Production Engineering

Foundations and Snowflake Core

Front matter, master manifest, What's New in Snowflake (to Oct 2026), Part I — Data & Cloud Foundations (Ch. 1–3) and Part II — Snowflake Core (Ch. 4–7).

Edition 2026.10 · Research cutoff: 10 October 2026 · Verified against docs.snowflake.com and modelcontextprotocol.io
An independent educational textbook. Not produced, endorsed or reviewed by Snowflake Inc. Snowflake and related names are trademarks of their owners.

Contents

How to Use This Book 7
About this textbook 7
Research cutoff and how currency is handled 7
Feature status labels 8
Conventions used in this book 8
Honesty about code testing 9
The running example: ByteMart 9
Setting up your lab environment 10
Learning roadmap 10
Master manifest 11
Coverage matrix (Volume 1) 13

What's New in Snowflake 15
How to read this section 15
Summary table 15
1. CoWork and CoCo: two product renames 16
2. Adaptive Compute and Adaptive Warehouses 17
3. Gen2 standard warehouses become the default 18
4. Snowflake Optima 18
5. Cortex AI Functions and the new privilege model 18
6. AI cost controls 19
7. Cortex Agents mature — and become the recommended entry point 19
8. The Snowflake-managed MCP server 20
9. Semantic views become the semantic layer 20
10. Snowpipe Streaming: high-performance architecture 20
11. Openflow, second generation 21
12. Dynamic tables keep expanding 21
13. Apache Iceberg and open table formats 21
14. Developer experience 21
15. Security and governance 22
16. Data types 22
17. Transactional workloads: hybrid tables and Snowflake Postgres 22
18. Announced, not generally available 22
Deprecated, renamed and legacy items to recognise 23
Further reading 23

PART I — FOUNDATIONS OF DATA AND CLOUD COMPUTING 25

Chapter 1: Introduction to Data Engineering 26
1.1 What data is and why organizations collect it 26
1.2 Structured, semi-structured and unstructured data 27
1.3 Operational versus analytical workloads: OLTP and OLAP 28
1.4 Databases, data warehouses, data lakes and lakehouses 30
1.5 Who does what: data engineering, analytics and data science 31
1.6 ETL and ELT 32
1.7 Batch and streaming processing 33

1.8 Data pipelines and layered architecture ... 35
1.9 Data quality, lineage, governance and observability ... 37
1.10 Guided lab: design ByteMart's first pipeline on paper ... 38
1.11 Production use case: from spreadsheets to a pipeline ... 38
1.12 Common mistakes ... 39
1.13 Troubleshooting exercise ... 39
1.14 Knowledge check ... 39
1.15 Hands-on assignment ... 40
1.16 Chapter summary ... 40
1.17 Key takeaways ... 40
1.18 Further reading ... 41
1.19 Solutions ... 41

Chapter 2: Cloud Computing Fundamentals ... 43
2.1 What cloud computing is ... 43
2.2 Service models: IaaS, PaaS and SaaS ... 44
2.3 Public, private and hybrid clouds ... 45
2.4 Compute, storage and networking ... 45
2.5 AWS, Azure and Google Cloud — the services that matter for Snowflake ... 46
2.6 Object storage and analytical file formats ... 47
2.7 Identity and access management ... 48
2.8 Regions, availability zones and private connectivity ... 49
2.9 Scalability, elasticity and availability ... 51
2.10 Cloud billing and consumption-based pricing ... 51
2.11 The shared-responsibility model ... 53
2.12 Guided lab: map cloud concepts to Snowflake ... 54
2.13 Production use case: choosing ByteMart's region ... 54
2.14 Common mistakes ... 55
2.15 Troubleshooting exercise ... 55
2.16 Knowledge check ... 55
2.17 Hands-on assignment ... 56
2.18 Chapter summary ... 56
2.19 Key takeaways ... 56
2.20 Further reading ... 56
2.21 Solutions ... 57

Chapter 3: SQL From Zero ... 59
3.0 How to practise this chapter ... 59
3.1 Tables, rows, columns, schemas and keys ... 60
3.2 SELECT and FROM ... 63
3.3 Filtering with WHERE ... 65
3.4 Sorting with ORDER BY ... 67
3.5 Aggregation, GROUP BY and HAVING ... 67
3.6 NULL: the value that is not a value ... 69
3.7 Joins ... 72
3.8 Subqueries and common table expressions (CTEs) ... 75
3.9 CASE expressions ... 77
3.10 Date and string functions ... 78
3.11 Window functions ... 80

3.12 Set operators ... 83
3.13 DDL: creating and changing structure ... 84
3.14 DML: INSERT, UPDATE, DELETE and MERGE ... 85
3.15 Transactions ... 87
3.16 How a query is evaluated: logical order ... 88
3.17 Intermediate patterns ... 89
3.18 Common mistakes ... 92
3.19 Troubleshooting exercise ... 92
3.20 Practice exercises ... 92
3.21 Chapter summary ... 93
3.22 Key takeaways ... 93
3.23 Further reading ... 93
3.24 Solutions ... 94

PART II — SNOWFLAKE CORE ... 101

Chapter 4: Introduction to Snowflake ... 102
4.1 What Snowflake is ... 102
4.2 A short history and evolution ... 103
4.3 Core use cases and workloads ... 104
4.4 How Snowflake runs on the clouds ... 105
4.5 Snowflake editions ... 106
4.6 Strengths, limitations and trade-offs ... 107
4.7 Guided lab: your first Snowflake session ... 108
4.8 Production use case: ByteMart's platform decision ... 110
4.9 Common mistakes ... 111
4.10 Troubleshooting exercise ... 111
4.11 Knowledge check ... 111
4.12 Hands-on assignment ... 112
4.13 Chapter summary ... 112
4.14 Key takeaways ... 112
4.15 Further reading ... 112
4.16 Solutions ... 113

Chapter 5: Snowflake Architecture in Depth ... 114
5.1 The big picture: three layers ... 114
5.2 The storage layer ... 116
5.3 Pruning: how Snowflake skips data ... 117
5.4 The compute layer: virtual warehouses ... 118
5.5 The cloud services layer ... 120
5.6 The query lifecycle ... 121
5.7 Scaling: up, out, and concurrency ... 122
5.8 The three kinds of caching ... 123
5.9 Fault tolerance and availability ... 125
5.10 Snowflake Optima: automatic optimisation ... 126
5.11 Reading the Query Profile ... 126
5.12 Guided labs ... 127
5.13 Production use case: separating ByteMart's workloads ... 128
5.14 Common mistakes and misconceptions ... 129

5.15 Troubleshooting exercise 129
5.16 Knowledge check 129
5.17 Hands-on assignment 130
5.18 Chapter summary 130
5.19 Key takeaways 130
5.20 Further reading 130
5.21 Solutions 131

Chapter 6: Snowflake Account and Object Fundamentals 133

6.1 The object hierarchy 133
6.2 Schema-level objects at a glance 135
6.3 Identifiers and naming conventions 136
6.4 Users, roles, privileges and ownership 137
6.5 Core object types in practice 138
6.6 How objects work together in a real application 142
6.7 Object dependencies 143
6.8 Object lifecycle and ownership 143
6.9 Guided lab: build ByteMart's object skeleton 144
6.10 Production use case: ByteMart's first platform layout 146
6.11 Common mistakes 146
6.12 Troubleshooting exercise 147
6.13 Knowledge check 147
6.14 Hands-on assignment 147
6.15 Chapter summary 147
6.16 Key takeaways 148
6.17 Further reading 148
6.18 Solutions 148

Chapter 7: Snowflake Data Types and Storage 150

7.1 Why types matter 150
7.2 Numeric types 151
7.3 String and binary types 152
7.4 BOOLEAN 152
7.5 Date and time types 152
7.6 Semi-structured types: VARIANT, OBJECT and ARRAY 154
7.7 Other types you will meet 155
7.8 Type conversion 156
7.9 NULL semantics in Snowflake 157
7.10 Constraints and enforcement 157
7.11 Storage implications 158
7.12 Table types 159
7.13 Guided lab: types in practice 162
7.14 Production use case: typing ByteMart's silver layer 162
7.15 Common mistakes 163
7.16 Troubleshooting exercise 164
7.17 Knowledge check 164
7.18 Hands-on assignment 164
7.19 Chapter summary 164
7.20 Key takeaways 165

7.21 Further reading 165
7.22 Solutions 165

How to Use This Book

About this textbook

The Complete Snowflake Learning Bible is a structured, end-to-end textbook for one kind of reader: a motivated person who has never used Snowflake, may know little SQL, has never built a data pipeline and has never worked with generative AI, retrieval-augmented generation (RAG), the Model Context Protocol (MCP) or AI agents — and who wants to reach the point where they can design, build, secure, operate and evolve real Snowflake platforms.

It is not an interview question bank, a certification cheat sheet or a quick tutorial. Every chapter teaches. Concepts are introduced in plain language first, then explained mechanically ("how does this actually work inside the platform?"), then implemented with real SQL, Python or configuration, then examined for cost, security, performance and failure modes. Each chapter ends with exercises whose solutions are worked out in full.

The book is published as **six volumes** plus a final compiled master edition. This is Volume 1.

Volume	Parts	Chapters	Theme
1 (this volume)	Front matter, What's New, I, II	1–7	Foundations and Snowflake Core
2	III, IV	8–16	SQL, programming, ingestion, streaming and ELT
3	V, VI	17–22	Performance, cost, operations, security, governance and AI guardrails
4	VII	23–31	AI fundamentals, Cortex AI, Search, Analyst, Agents, RAG, MCP and CoCo
5	VIII, IX, X	32–39	Streamlit, applications, migration, integration, enterprise architecture and recovery
6	XI, XII	40–44	Six guided projects, glossary, misconceptions, troubleshooting handbook and roadmap

Research cutoff and how currency is handled

Research cutoff: 10 October 2026. Version-sensitive statements in this book — product names, feature status, function names, SQL syntax, privileges, limits and endpoints — were checked against official sources on or shortly before that date, primarily:

- Snowflake documentation — https://docs.snowflake.com
- Snowflake release notes — https://docs.snowflake.com/en/release-notes/overview
- Model Context Protocol specification — https://modelcontextprotocol.io

Snowflake ships changes weekly. Anything marked *Preview* in this book may have changed status by the time you read it; anything marked *Generally Available* may have gained options. When a detail matters for production, check the linked documentation page — every version-sensitive chapter ends with a **Further reading** list of the exact pages used.

Where official documentation could not be verified (for example, a capability that has only been announced at a conference), the text says so explicitly and does not present it as working functionality.

Feature status labels

Every Snowflake capability whose availability is not universal carries a status label. The labels mean:

Label	Meaning	Safe for production?
GA — Generally Available	Released for all eligible accounts, covered by support and service terms. May still be edition- or region-restricted.	Yes
Public Preview	Available to eligible accounts for evaluation. Behaviour, syntax or pricing may change.	Evaluate only, unless you accept change risk
Private Preview	Available only to invited customers. Mentioned only when officially documented.	No — not generally obtainable
Announced	Publicly announced (for example at Snowflake Summit) but not documented as available.	No — roadmap only
Deprecated	Still works but scheduled for removal; a replacement exists.	Migrate away
Legacy syntax	Older syntax still supported alongside a newer form (for example `SNOWFLAKE.CORTEX.COMPLETE` beside `AI_COMPLETE`).	Works; prefer the new form for new code
Third-party	Not a Snowflake product (for example dbt Core, Apache Airflow, Matillion).	Depends on the vendor
Edition / Region	Available only in a given Snowflake edition (Standard, Enterprise, Business Critical, VPS) or region set.	Check your account

Conventions used in this book

Code blocks are written for a specific execution environment, which is always stated. The three you will see most are:

- **Snowsight worksheet or Workspace** — the web interface at `app.snowflake.com`, running SQL as your current role and warehouse.
- **Local Python** — Python 3.10+ on your own machine with a named package installed via `pip`.
- **Local SQL engine (DuckDB)** — used only in Chapter 3, so that a beginner can practise portable SQL before creating a Snowflake account.

Placeholders are written in angle brackets: `<database_name>`, `<warehouse_name>`. Replace the whole token including the brackets.

Callouts are colour-coded:

> **LEARNING OBJECTIVES**
> What you will be able to do at the end of a chapter.

CONCEPT
A definition or explanation worth remembering.

BEST PRACTICE
A pattern experienced engineers rely on.

COMMON MISTAKE
Something that regularly goes wrong, and how to avoid it.

SECURITY OR DATA-LOSS RISK
A mistake whose consequences are expensive or irreversible.

FEATURE STATUS
GA / Preview / edition / region information for a capability.

GUIDED LAB
Step-by-step hands-on work.

Honesty about code testing

The author has **not** executed the Snowflake SQL, Snowpark, Cortex, MCP or Streamlit examples in a live Snowflake account. Every Snowflake-specific statement was written against the current official syntax reference, and the reference page is cited. Treat each example as *documentation-verified but not run*: if a statement fails in your account, the most likely causes are (1) a missing privilege, (2) an edition or region restriction, or (3) a change to the product after the research cutoff — each chapter tells you how to diagnose these.

The portable SQL in Chapter 3 is different: those queries **were executed** on DuckDB (an in-process SQL engine whose syntax for these features matches Snowflake's), and the expected outputs printed in the chapter are the actual results. Where Snowflake behaves differently from DuckDB, the difference is noted.

The running example: ByteMart

Most examples in the book use one fictional company so that each chapter builds on the last. **ByteMart** is a mid-sized online retailer selling electronics and home goods in India, the UK and the US. It has:

- an **orders website and mobile app** backed by a PostgreSQL transactional database (customers, orders, order lines, payments);
- a **product catalogue** exported nightly as CSV from an ERP system;
- **clickstream events** (page views, add-to-cart, checkout) emitted as JSON to a message queue;
- **customer-support tickets and product manuals** stored as text and PDF;

- a growing analytics team that wants dashboards, a governed self-service layer, and — eventually — an AI assistant that can answer questions about sales and policy documents.

ByteMart begins Chapter 1 with spreadsheets and nightly database exports. By Chapter 40 it has a layered Snowflake platform with streaming ingestion, CDC, dynamic tables, governance policies, cost controls, a Cortex Search–based RAG assistant and an MCP-exposed Cortex Agent. You will build every step.

Setting up your lab environment

You need very little to begin.

1. **Chapters 1–3** require only a laptop. Install Python 3.10 or later and run `pip install duckdb` to practise SQL locally.
2. **From Chapter 4** you need a Snowflake account. A free trial account (sign up at https://signup.snowflake.com) is sufficient for nearly every lab. Choose **Enterprise** edition when offered — several features in later chapters (multi-cluster warehouses, masking policies, row access policies, extended Time Travel, materialized views, search optimization) require Enterprise Edition or higher.
3. Choose a **cloud region** close to you. If you plan to follow the Cortex AI chapters, prefer a region where Cortex AI functions are available (Chapter 24 explains regional availability and cross-region inference).
4. Never run labs as `ACCOUNTADMIN` once Chapter 6 has shown you how to create a dedicated lab role. Using the most powerful role for everyday work is the single most common bad habit in Snowflake.

Learning roadmap

The book follows seven levels. Each level assumes the previous ones. The diagram below shows the recommended order; arrows mean "should be studied before".

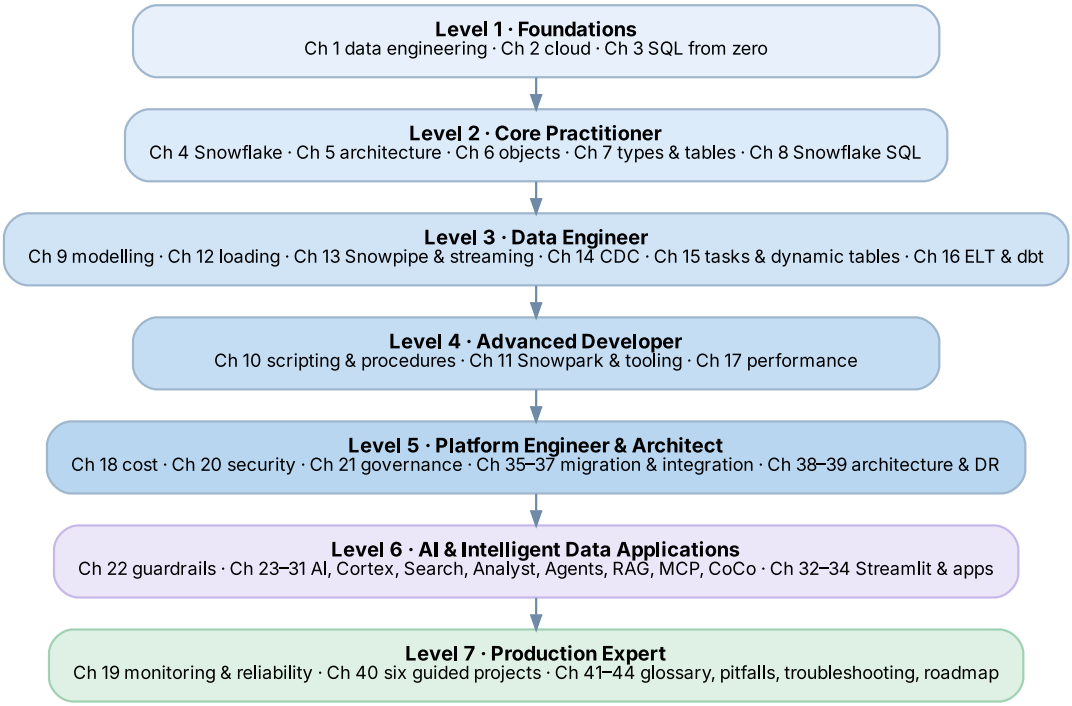


Figure 0.1 — Recommended learning order across the seven levels. Arrows indicate prerequisites.

Level	Name	You can…	Chapters
1	Foundations	explain data engineering and cloud concepts; write correct SQL	1–3
2	Core Practitioner	navigate Snowflake, create objects, choose types and table kinds, write Snowflake SQL	4–8
3	Data Engineer	build batch, continuous and CDC pipelines; model data; orchestrate transformations	9, 12–16
4	Advanced Developer	write procedures, UDFs and Snowpark code; diagnose performance	10, 11, 17
5	Platform Engineer & Architect	design security, governance, cost control, migration and disaster recovery	18, 20, 21, 35–39
6	AI & Intelligent Data Applications	build Cortex AI, Search, Analyst, Agents, RAG, MCP and Streamlit solutions with guardrails	22–34
7	Production Expert	operate, monitor, troubleshoot and evolve the whole platform	19, 40–44

Master manifest

This manifest tracks every planned chapter against its volume and status. It is updated in each volume. **Written** means the chapter text, examples, exercises and solutions are complete. **Doc-verified** means version-sensitive details were checked against the official documentation at the research cutoff. **Rendered** means the chapter has been rendered to PDF and visually inspected.

#	Chapter	Vol	Written	Doc-verified	Rendered
—	How to Use This Book; What's New in Snowflake	1	Yes	Yes	Yes
1	Introduction to Data Engineering	1	Yes	n/a (stable concepts)	Yes
2	Cloud Computing Fundamentals	1	Yes	n/a (stable concepts)	Yes
3	SQL From Zero	1	Yes	Executed on DuckDB	Yes
4	Introduction to Snowflake	1	Yes	Yes	Yes
5	Snowflake Architecture in Depth	1	Yes	Yes	Yes
6	Snowflake Account and Object Fundamentals	1	Yes	Yes	Yes
7	Snowflake Data Types and Storage	1	Yes	Yes	Yes
8	Snowflake SQL — Complete Reference	2	Planned	—	—
9	Advanced SQL and Data Modeling	2	Planned	—	—
10	Snowflake Scripting, Procedures, and Functions	2	Planned	—	—
11	Snowpark and Developer Tooling	2	Planned	—	—
12	File Formats, Stages, and Bulk Loading	2	Planned	—	—
13	Snowpipe and Continuous Ingestion	2	Planned	—	—
14	Change Data Capture and Streams	2	Planned	—	—
15	Tasks, Dynamic Tables, and Orchestration	2	Planned	—	—
16	Modern Data Engineering and ELT	2	Planned	—	—
17	Performance Optimization	3	Planned	—	—
18	Cost Optimization and FinOps	3	Planned	—	—
19	Monitoring, Reliability, and Troubleshooting	3	Planned	—	—
20	Snowflake Security Architecture	3	Planned	—	—
21	Data Governance and Protection	3	Planned	—	—
22	AI Governance, Guardrails, and Responsible AI	3	Planned	—	—
23	AI Fundamentals for Snowflake Engineers	4	Planned	—	—
24	Snowflake Cortex AI Ecosystem	4	Planned	—	—
25	Cortex AI Functions — Complete Practical Guide	4	Planned	—	—
26	Cortex Search and Enterprise Search	4	Planned	—	—
27	Cortex Analyst and Natural-Language Analytics	4	Planned	—	—
28	Cortex Agents and Agentic Workflows	4	Planned	—	—
29	RAG From Zero to Production	4	Planned	—	—
30	MCP — Model Context Protocol and Snowflake	4	Planned	—	—
31	CoCo (formerly Cortex Code) and AI-Assisted Data Engineering	4	Planned	—	—

#	Chapter	Vol	Written	Doc-verified	Rendered
32	Streamlit From Zero	5	Planned	—	—
33	Streamlit in Snowflake	5	Planned	—	—
34	Full-Stack Snowflake Applications	5	Planned	—	—
35	Snowflake Migration From Zero to Expert	5	Planned	—	—
36	Advanced Migration Engineering	5	Planned	—	—
37	Integrations and the Modern Data Stack	5	Planned	—	—
38	Enterprise Architecture and Workload Design	5	Planned	—	—
39	Advanced Reliability and Recovery	5	Planned	—	—
40	Guided Projects From Beginner to Expert	6	Planned	—	—
41	Snowflake Glossary	6	Planned	—	—
42	Common Mistakes and Misconceptions	6	Planned	—	—
43	Practical Troubleshooting Handbook	6	Planned	—	—
44	Complete Learning Roadmap	6	Planned	—	—

Final compilation status: Volume 1 complete. Master PDF will be compiled after Volume 6.

Coverage matrix (Volume 1)

The full coverage matrix grows with each volume. For the subjects covered in this volume:

Subject	Chapters	Concepts covered	Practical implementation	Exercises	Current docs verified	Remaining limitations
Data engineering foundations	1	Data types of data, OLTP/OLAP, warehouse/lake/lakehouse, ETL/ELT, batch/streaming, layers, quality, lineage, governance, observability	ByteMart pipeline design walkthrough	Yes, with solutions	n/a	Conceptual chapter; tools introduced later
Cloud fundamentals	2	Service models, compute/storage/network, AWS/Azure/GCP, object storage, file formats, regions, IAM, private connectivity, elasticity, billing, shared responsibility	Mapping exercise from cloud concepts to Snowflake objects; credit maths	Yes, with solutions	Snowflake-specific parts verified	No cloud-console labs (vendor UIs change frequently)
SQL foundations	3	DDL, DML, SELECT, filtering, NULL, joins, CTEs, CASE, functions, windows, set ops, transactions, constraints, execution order, patterns	60+ executed queries on ByteMart sample data	Yes, 20 with solutions	Executed on DuckDB; Snowflake differences noted	Snowflake-only SQL deferred to Ch 8

Subject	Chapters	Concepts covered	Practical implementation	Exercises	Current docs verified	Remaining limitations
Snowflake overview	4	Definition, history, clouds, editions, workloads, strengths, limitations	First login, first warehouse, first query	Yes, with solutions	Yes (editions, Gen2, Adaptive, Optima)	Pricing figures intentionally omitted (region and contract specific)
Architecture	5	Three layers, micro-partitions, columnar storage, metadata, compilation, pruning, three caches, concurrency, multi-cluster, Gen2, Adaptive, Optima, query lifecycle	Cache and pruning experiments on sample data	Yes, with solutions	Yes	Internal implementation details are summarised only as far as Snowflake documents them
Objects	6	Organization → account → database → schema → objects; all major object types; naming; dependencies; ownership; lifecycle	Full ByteMart object skeleton with least-privilege lab role	Yes, with solutions	Yes	Each object gets its deep dive in later chapters
Types and tables	7	All type families incl. DECFLOAT, VECTOR, UUID, FILE, structured types; timestamps; conversion; NULL; collation; constraints; table types incl. hybrid, Iceberg, external, dynamic	Type and table-type labs	Yes, with solutions	Yes	Iceberg and hybrid table operations deepened in Vol 5

What's New in Snowflake

Status of the Snowflake platform at the research cutoff, 10 October 2026, and what each change means for the way you learn and build.

How to read this section

Snowflake releases server updates roughly weekly, and feature announcements appear several times a week. This section does not list every release note. It selects the changes that alter *how a learner or engineer should work* — renamed products, new defaults, new privileges, new architectures — and for each one answers the same eight questions: what changed, what existed before, why it matters, who should care, how it works, its current status, whether it replaces or coexists with something, and what to do differently.

If you are a complete beginner, skim this section now and return to it after Chapter 7 — many terms (warehouse, Cortex, semantic view, stream) are introduced properly in the chapters. Every item is cross-referenced to the chapter that teaches it.

STATUS VOCABULARY

GA = Generally Available. *Public Preview* and *Private Preview* are as defined in "How to Use This Book". *Announced* means stated publicly but not documented as available. Dates are the release-note dates published by Snowflake.

Summary table

#	Change	Status at cutoff	Teaching chapter
1	Snowflake Intelligence renamed **Snowflake CoWork**; Cortex Code renamed **Snowflake CoCo**	Renamed 2 Jun 2026	24, 28, 31
2	**Adaptive Compute** (Adaptive Warehouses)	GA (Enterprise+, select regions)	5, 17, 18
3	**Gen2** standard warehouses becoming the default generation	Gen2 GA; default change in the 2026_03 behaviour-change bundle	5, 17
4	**Snowflake Optima** automatic optimisation	Included in all editions; Gen2 only	5, 17
5	Cortex AI Functions family complete; **new privilege model** (`USE AI FUNCTIONS`, `AI_FUNCTIONS_USER`, per-function privileges)	GA	24, 25
6	AI cost controls: budgets, per-user quotas, response caching	GA (various dates)	18, 25
7	**Cortex Agents** maturity; Snowflake recommends Agents over direct Cortex Analyst	Agents GA since Nov 2025; many 2026 additions	27, 28
8	**Snowflake-managed MCP server**	Documented; supports MCP revision 2025-11-25	30

#	Change	Status at cutoff	Teaching chapter
9	**Semantic views** become the semantic layer; Semantic Studio	GA (Semantic Studio 30 Sep 2026)	27
10	**Snowpipe Streaming high-performance architecture** with the PIPE object	GA; elastic channels GA 15 Sep 2026	13
11	**Openflow** (Apache NiFi–based integration) second generation	GA; gen-1 → gen-2 migration GA 1 Oct 2026	13, 37
12	**Dynamic tables** incremental-refresh expansion	Mostly GA; adaptive refresh mode in Public Preview	15
13	**Apache Iceberg v3**, Snowflake-managed Iceberg storage, external-engine writes	GA (various dates)	7, 37
14	Developer experience: Workspaces, Notebooks in Workspaces, dbt Projects on Snowflake, DCM Projects, CoCo surfaces	Mostly GA	11, 16, 31
15	Security and governance: Cortex AI Guardrails, multi-party approval, data movement policies, inherited grants, service-agent users	GA (various)	20, 21, 22
16	New and updated data types: DECFLOAT, UUID, PERIOD, structured types, user-defined types	Documented; PERIOD GA Sep 2026	7
17	Hybrid tables and Snowflake Postgres for transactional workloads	GA	7
18	Announced or private-preview items: Datastream, Cortex Sense, Cortex Training, Snowflake Decision	**Not generally available**	Roadmap notes only

1. CoWork and CoCo: two product renames

What changed. On 2 June 2026, at Snowflake Summit 2026, Snowflake renamed two AI products. *Snowflake Intelligence* — the conversational agent experience for business users — became **Snowflake CoWork**. *Cortex Code* — the AI coding agent for engineers — became **Snowflake CoCo**.

What existed before. Snowflake Intelligence (announced at BUILD 2024) let business users ask questions of governed data through agents built on Cortex Analyst and Cortex Search. Cortex Code was the agentic assistant for SQL, Python, pipelines and administration, available as a CLI (GA 2 Feb 2026) and inside Snowsight (GA 9 Mar 2026).

Why it matters. Older blogs, courses and internal wikis use the old names; you need to recognise them. The rename also signals broader scope: CoWork adds automations, deep research, artifacts and connectors to external tools; CoCo is available in Snowsight, as a CLI, as a desktop application (CoCo Desktop) and inside the Snowflake Extension for Visual Studio Code (GA 27 Aug 2026).

How it works. Both products are agents that run inside Snowflake's security perimeter and act with the privileges of the signed-in user's role. CoCo plans multi-step tasks, proposes changes as a diff for you to review, and can run SQL on your behalf. CoWork answers business questions using Cortex Agents configured with semantic views and search services.

Status. Renamed products; existing customers were migrated automatically. Some documentation URLs and system objects still use the old names — for example, the documentation path `/user-guide/cortex-code/` and the Account Usage view `SNOWFLAKE_INTELLIGENCE_USAGE_HISTORY` (GA 25 Feb 2026). A newer `SNOWFLAKE_COCO_USAGE_HISTORY` view went GA on 17 Aug 2026.

What to do differently. Search documentation using both names. When you read "Snowflake Intelligence" in material dated before June 2026, read "CoWork". This book uses the new names and notes the old ones on first use.

2. Adaptive Compute and Adaptive Warehouses

What changed. Adaptive Compute is now **Generally Available** for Enterprise Edition (or higher) accounts in select AWS, Azure and Google Cloud regions. You use it through an *Adaptive Warehouse*: `CREATE ADAPTIVE WAREHOUSE my_wh;` or convert an existing warehouse with `ALTER WAREHOUSE my_wh SET WAREHOUSE_TYPE = 'ADAPTIVE';`.

What existed before. You chose a fixed T-shirt size (X-Small to 6X-Large), optionally configured multi-cluster scaling, the Query Acceleration Service, and auto-suspend/resume. Sizing was a recurring operational task — and a frequent source of waste.

Why it matters. With an Adaptive Warehouse you no longer set size, multi-cluster settings, Query Acceleration Service settings or suspend/resume policies. All Adaptive Warehouses in an account draw from a shared, account-dedicated compute pool; Snowflake decides per query how much compute to apply. Billing becomes **query-based**: each query's cost depends on the compute and software resources it used, and per-query credits appear in `ACCOUNT_USAGE.QUERY_METERING_HISTORY`.

How you control it. Two properties replace size and cluster counts:

- `MAX_QUERY_PERFORMANCE_LEVEL` (XSMALL … X4LARGE, default XLARGE) — the upper bound of performance Snowflake may apply to a single query when it is confident that more compute helps;
- `QUERY_THROUGHPUT_MULTIPLIER` (0 = unlimited, or an integer ≥ 2; default 2) — how much total concurrent work the warehouse admits before queuing.

Budgets and resource monitors still govern total spend; a resource monitor can *disable* an Adaptive Warehouse, and `SHOW WAREHOUSES` gains `STATE` and `DISABLED_REASONS` columns.

Coexistence. Adaptive coexists with standard (Gen1/Gen2), Snowpark-optimized and interactive warehouses. Snowflake's own guidance is to prefer standard Gen2 when you need direct control over size and scaling, interactive warehouses for very low-latency user-facing queries, and Snowpark-optimized for high-memory single-node work. Conversions to or from 5X-Large/6X-Large, Snowpark-optimized and interactive warehouses are not supported.

What to do differently. Learn classic warehouse sizing anyway (Chapter 5) — it is how most existing accounts run and it remains the mental model behind `MAX_QUERY_PERFORMANCE_LEVEL`. For new Enterprise-edition workloads in supported regions, evaluate Adaptive first. Use `SYSTEM$BULK_UPDATE_WH(..., 'DRY_RUN')` before converting many warehouses.

3. Gen2 standard warehouses become the default

What changed. Generation 2 standard warehouses (GA since June 2025) use newer hardware and software optimisations for analytics and data-engineering workloads. Behaviour change **2250** in the **2026_03 bundle** makes Gen2 the default generation for new standard warehouses in regions where Gen2 is available, when `GENERATION` is not specified. Existing Gen1 warehouses remain Gen1 unless you change them.

Watch for. Gen2 is not available for X5LARGE and X6LARGE sizes; those default to Gen1. A `CREATE OR ALTER WAREHOUSE` without a `GENERATION` clause can convert a Gen1 warehouse to Gen2 when it performs an ALTER — include `GENERATION = '1'` if you must keep Gen1. Credit consumption for Gen2 differs from Gen1; consult the Service Consumption Table before converting large fleets.

Why it matters. Gen2 is required for Snowflake Optima (item 4). Chapter 17 shows how to benchmark Gen1 against Gen2 before switching.

4. Snowflake Optima

What changed. Snowflake Optima is an automatic workload-optimisation layer, included in all editions at no additional cost but **available only on Gen2 standard warehouses**. It observes recurring queries and applies optimisations without configuration. Documented components include *Optima Indexing* (hidden indexes for repetitive point lookups, built on the Search Optimization Service), *Optima Metadata* (custom pruning metadata for filters on expressions such as `UPPER()` or `LIKE`) and *Optima Planning* (learned execution plans). Snowflake's August 2026 engineering blog also describes *Optima Clustering*.

Coexistence. Optima does not replace the Search Optimization Service, clustering keys or materialized views. Snowflake explicitly recommends applying search optimization directly when you need *guaranteed* index freshness and consistent performance. Optima is best-effort.

What to do differently. Keep following good design (Chapter 17). When reading a Query Profile, look for indications that a learned plan or Optima-generated metadata was applied, so you do not attribute an improvement to your own change by mistake.

5. Cortex AI Functions and the new privilege model

What changed. The Cortex AI Functions family is complete and largely GA. The current documented task functions are `AI_COMPLETE`, `AI_CLASSIFY`, `AI_FILTER`, `AI_AGG`, `AI_EMBED`, `AI_MULTI_EMBED`, `AI_EXTRACT`, `AI_SENTIMENT`, `AI_SUMMARIZE`, `AI_SUMMARIZE_AGG`, `AI_SIMILARITY`, `AI_TRANSCRIBE`, `AI_PARSE_DOCUMENT`, `AI_REDACT` and `AI_TRANSLATE`, with helper functions `TO_FILE`, `AI_COUNT_TOKENS` (GA 27 Jan 2026) and `PROMPT`. Individual functions or features within them can still be Preview — for example, multimodal theme summarisation in `AI_SUMMARIZE` (Public Preview, 14 Sep 2026) and document classification in `AI_CLASSIFY` (Public Preview, 13 May 2026).

Privileges. To call these functions, a role now needs the account-level **`USE AI FUNCTIONS`** privilege (introduced 20 Feb 2026) *and* one of the database roles **`SNOWFLAKE.CORTEX_USER`** or **`SNOWFLAKE.AI_FUNCTIONS_USER`** (GA 2 Apr 2026). Per-function privileges (GA 13 May 2026) let you grant, for example, only `AI_COMPLETE`.

What existed before. The `SNOWFLAKE.CORTEX.*` functions (`COMPLETE`, `SUMMARIZE`, `SENTIMENT`, `TRANSLATE`, `EXTRACT_ANSWER`, `CLASSIFY_TEXT`, `EMBED_TEXT_768/1024`). These are now **legacy syntax**: `AI_COMPLETE` is the updated version of `COMPLETE`; `AI_EXTRACT` is the updated version of `EXTRACT_ANSWER`.

Why it matters. Older tutorials grant only `CORTEX_USER` and call `SNOWFLAKE.CORTEX.COMPLETE`. In a current account, missing `USE AI FUNCTIONS` is the most likely reason a "correct" tutorial fails with an authorisation error.

What to do differently. Use the `AI_*` functions for new code (Chapter 25 includes a migration table). Grant AI privileges to dedicated roles, never to `PUBLIC`.

6. AI cost controls

Several 2026 releases give administrators guardrails on AI spending:

- **Monitor and control Cortex AI Functions spending** (GA 2 Mar 2026) and **Budgets for AI features** (GA 10 Apr 2026);
- **Resource budgets** for Cortex Agents and CoWork (11 Mar 2026) and for Cortex Search (3 Jul 2026);
- **Per-user quotas** (introduced as Preview in July 2026) with **weekly per-user quota limits** and include/exclude-by-name (both GA 16 Sep 2026) — monthly, weekly or daily credit limits per user across AI functions, Cortex Agents, CoWork and CoCo, with automatic blocking;
- **Response caching for Cortex AI Functions** (29 Sep 2026);
- **Cortex AI Function Studio** (Public Preview, 20 May 2026) and **Cortex AI Function Evaluation** and **Optimization** (Public Preview, 21 Sep 2026) for measuring and tuning quality and cost.

What to do differently. Treat AI cost as a first-class FinOps dimension from the first prototype. Chapter 18 builds a per-user AI spend dashboard.

7. Cortex Agents mature — and become the recommended entry point

What changed. Cortex Agents became GA on 4 Nov 2025, with a schema-level `AGENT` object (`CREATE AGENT … FROM SPECIFICATION`). 2026 added agent evaluations (GA 13 Mar), the asynchronous API (GA 30 Aug), a Coding Agent (GA 26 Aug), object enhancements (GA 16 Sep), a code execution tool (GA 5 Oct), and a Compact API (Preview, 21 Sep). Data lineage now shows the tables, semantic views and search services an agent can reach.

Recommendation change. On 28 Aug 2026 Snowflake published a recommendation to **transition from calling Cortex Analyst directly to using Cortex Agents**, which orchestrate Cortex Analyst, Cortex Search and custom tools behind one governed interface.

What to do differently. Learn Cortex Analyst and semantic views (Chapter 27) because agents depend on them, but design new applications around an agent (Chapter 28).

8. The Snowflake-managed MCP server

What changed. Snowflake now hosts MCP servers as schema-level objects: `CREATE MCP SERVER <name> FROM SPECIFICATION $$ tools: … $$`. External MCP clients — Claude, ChatGPT, Cursor, VS Code, agent frameworks — connect over HTTPS at `https://<account_url>/api/v2/databases/<db>/schemas/<schema>/mcp-servers/<name>` and discover tools with `tools/list` and invoke them with `tools/call`.

Supported tool types. `CORTEX_AGENT_RUN`, `CORTEX_SEARCH_SERVICE_QUERY`, `CORTEX_ANALYST_MESSAGE`, `SYSTEM_EXECUTE_SQL` (with a `read_only` option) and `GENERIC` (UDFs and stored procedures). Up to 50 tools per server; generic-tool and SQL responses are truncated at 250 KB.

Security model. Snowflake OAuth by default, optional External OAuth (Okta, Entra ID), no dynamic client registration. Access to the server does **not** grant access to its tools; each underlying object needs its own grant. Snowflake recommends `OAUTH_USE_SECONDARY_ROLES = NONE`, an `ALLOWED_ROLES_LIST` restricted to a dedicated MCP role, and exposing a Cortex Agent rather than raw SQL for business-data use.

Protocol versions. Snowflake's documentation states support for MCP revision **2025-11-25** and that it supports only *tool* capabilities (no resources, prompts, sampling or roots). Since 20 Aug 2026, `tools/call` responses are streamed as Server-Sent Events, so clients must send `Accept: application/json, text/event-stream`. Separately, the MCP project published a new, stateless specification revision dated **2026-07-28**; at the research cutoff, Snowflake's MCP page did not state support for it. Chapter 30 teaches both revisions and explains what the stateless change means.

9. Semantic views become the semantic layer

Semantic views — schema-level objects that define business entities, dimensions, metrics and relationships — are now central to Cortex Analyst, Cortex Agents, CoWork and BI interoperability. Milestones: querying semantic views with standard SQL clauses (GA 2 Mar 2026), verified queries stored in semantic views (5 Apr 2026), entity-level filters (GA 5 May 2026), Power BI ingestion for Semantic View Autopilot (GA 18 Aug 2026), `DEFINE SEMANTIC VIEW` in DCM Projects (GA 30 Sep 2026), and **Semantic Studio** (GA 30 Sep 2026). YAML semantic model files on stages are the older approach; prefer semantic views for new work (Chapter 27).

10. Snowpipe Streaming: high-performance architecture

The high-performance architecture introduces a **PIPE object** as the entry point for streaming rows: client applications open *channels* against a pipe rather than directly against a table. It uses a new `snowpipe-streaming` SDK with a Rust-based core (Java and Python) and a REST API, supports up to 10 GB/s uncompressed aggregate throughput per table, uses throughput-based pricing, and can cluster data at ingest time. The pipe's `ON_ERROR` option supports only `CONTINUE`. 2026 additions include error logging (GA 8 Apr), elastic channels (GA 15 Sep), event-table monitoring and partitioned Iceberg targets (24 Sep). The **classic** architecture still exists; new projects should start with high-performance (Chapter 13). The Snowflake Connector for Kafka 4.0 went GA on 20 Apr 2026.

11. Openflow, second generation

Snowflake Openflow is a managed data-integration service built on Apache NiFi for batch and streaming connectors (databases with CDC, SaaS applications, files, Kafka). Second-generation deployments and runtimes arrived in Preview on 31 Aug 2026, reached Google Cloud on 11 Sep, and **migration from gen 1 to gen 2 became GA on 1 Oct 2026**. Notable connectors include Oracle (GA 27 Feb 2026, multi-PDB replication 25 May), SQL Server CDC (GA 5 Aug 2026), Google BigQuery (Preview) and MongoDB CDC. Openflow is a Snowflake product; the connectors' source systems are third-party (Chapter 37).

12. Dynamic tables keep expanding

Dynamic tables are declarative pipelines: you write a `SELECT`, set a `TARGET_LAG`, and Snowflake keeps the result fresh, incrementally where possible. 2026 broadened what can refresh incrementally — primary keys (GA Apr), `MIN_BY`/`MAX_BY`, expanded outer joins, `VOLATILE` scalar UDFs (GA Jul), non-deterministic aggregates and `VALUES` clauses (GA Sep) — and added a `SCHEDULER` attribute (GA Mar), custom incremental dynamic tables (GA Jul), `EXPLAIN CHANGES` to predict refresh behaviour (GA 15 Sep), storage lifecycle policies (GA May), refreshing several dynamic tables in one `ALTER` (Jul), and continued incremental refresh after failover (GA 30 Sep). **Adaptive refresh mode** is in Public Preview (26 May). "Immutability constraints" were renamed **frozen regions** (28 May). Chapter 15 teaches dynamic tables with these capabilities in mind.

13. Apache Iceberg and open table formats

Snowflake's Iceberg support moved quickly in 2026: **Iceberg v3 support** (GA 7 May), **Snowflake storage for Iceberg tables** — Snowflake manages the storage location (GA 1 Jun), **write support from external query engines** (GA 26 May), partition evolution (GA 18 Sep), Amazon S3 Tables and Google BigLake Metastore catalog integrations (GA), and access to Snowflake-managed Iceberg tables from external engines through **Horizon Catalog** (GA Feb). Iceberg v3 introduced an `UNKNOWN` data type supported only in Iceberg tables. Chapter 7 introduces Iceberg tables; Volume 5 covers lakehouse architecture.

14. Developer experience

- **Workspaces** are the file-based editor in Snowsight (shared Workspaces GA Jan 2026). **Notebooks in Workspaces** became GA on 5 Feb 2026, and the older notebooks were renamed **Legacy Snowflake Notebooks** on 16 Mar 2026. Notebook project objects are now called **Code Bundles** (Aug 2026; Code Bundles Preview 24 Sep).
- **dbt Projects on Snowflake** run dbt natively; Slim CI with deferral to production went GA on 10 Sep 2026.
- **DCM Projects** (declarative change management for Snowflake objects) went GA on 7 Aug 2026.
- **CoCo** is available in Snowsight, the CLI, CoCo Desktop (GA July 2026) and VS Code.
- The **Snowflake CLI** (`snow`) is at 3.29 and manages Snowpark, Streamlit, Native Apps, notebooks, Git repositories, dbt and DCM projects.

15. Security and governance

- **Cortex AI Guardrails** (GA 20 Apr 2026), extended to CoWork and Cortex Agents (GA 14 May) — input and output safety filtering. Chapter 22 explains what guardrails do *not* protect against.
- **Multi-party approval** (GA 4 Aug 2026) and **data movement policies** (GA 19 Aug 2026).
- **Inherited grants and container-level `MANAGE GRANTS`** (GA 10 Sep 2026); **`WITH GRANT OPTION` for role grants** (5 Oct 2026).
- **`SERVICE_AGENT` user type** (GA 23 Jul 2026) for AI agents acting as principals; organization user types (GA 26 Aug).
- **Trust Center** expanded with detections, data-security and AI-security scanners.
- **Multi-value tags** (GA 25 Aug), **Snowflake-provided tags** (Public Preview 31 Aug), Snowsight interface for tags and data protection policies (GA 9 Oct).
- **OIDC federated authentication** (Public Preview, Jul 2026); session policies gained `SESSION_MAX_LIFESPAN_MINS` (Apr 2026).

16. Data types

The data-type reference now documents **DECFLOAT** (exact decimal floating point, up to 38 significant digits, exponent −16383 to 16384), **UUID** (128-bit, with `UUID_STRING()` and `TO_UUID`), **PERIOD** (GA 2026 release 10.32, September), **structured** `ARRAY`, `OBJECT` and `MAP`, **FILE** for unstructured data, **VECTOR** for embeddings, **user-defined types** (`CREATE TYPE age AS NUMBER(3,0)`), and the Iceberg-only **UNKNOWN** type. Several of these are not supported in non-SQL UDFs, Snowpark or hybrid tables. Chapter 7 covers each, including where the documentation does not state a release status.

17. Transactional workloads: hybrid tables and Snowflake Postgres

Hybrid tables (row-oriented storage with enforced primary keys, for low-latency point operations) received CHECK constraints (GA 24 Sep 2026), inline stored procedures (GA 28 Sep), online add/drop of UNIQUE and FOREIGN KEY constraints, simplified pricing and the removal of separate request metering (March 2026). **Snowflake Postgres** — a managed PostgreSQL service inside Snowflake — became GA on 24 Feb 2026. Chapter 7 explains when to use each.

18. Announced, not generally available

The following were announced at Summit 2026 or listed only as Private Preview. **Do not plan production work on them until their documentation shows availability.**

Item	What was announced	Status at cutoff
Snowflake Datastream	Kafka-compatible managed streaming service native to Snowflake	Private Preview (per Summit coverage)
Cortex Sense	Runtime context layer assembling business definitions for agents	Private Preview (per Summit coverage)
Cortex Training	Custom training of foundation models on managed GPUs	Announced
Snowflake Decision with AI_COMPLETE	—	Private Preview (7 Oct 2026 release note)
Agentic Marketplace Discovery	Agent-oriented discovery in Snowflake Marketplace	Public Preview (9 Oct 2026)

Models available through Cortex also change frequently — recent release notes add or preview models from Anthropic, OpenAI, Google, Meta, Mistral, DeepSeek, xAI and Moonshot (Kimi). Always check the model availability table for your region before choosing a model name in code.

Deprecated, renamed and legacy items to recognise

Older name or approach	Current equivalent	Notes
Snowflake Intelligence	Snowflake CoWork	Renamed 2 Jun 2026
Cortex Code	Snowflake CoCo	Renamed 2 Jun 2026
`SNOWFLAKE.CORTEX.COMPLETE` and sibling functions	`AI_COMPLETE` and the `AI_*` family	Legacy syntax, still documented
Granting only `SNOWFLAKE.CORTEX_USER`	Add `USE AI FUNCTIONS` on the account	Required since Feb 2026
Calling Cortex Analyst directly from applications	Cortex Agents with an Analyst tool	Snowflake recommendation, 28 Aug 2026
Semantic model YAML files on a stage	Semantic views	YAML still works; views are preferred
Snowflake Notebooks	Notebooks in Workspaces	Old notebooks are "Legacy Snowflake Notebooks"
Snowpipe Streaming classic SDK	High-performance architecture (`snowpipe-streaming` SDK, PIPE object)	Classic still documented
Openflow gen 1 deployments	Openflow gen 2	Migration GA 1 Oct 2026
Dynamic-table "immutability constraints"	"Frozen regions"	Renamed 28 May 2026
Gen1 as default standard warehouse	Gen2 default	Bundle 2026_03

Further reading

- Snowflake server release notes and feature updates — https://docs.snowflake.com/en/release-notes/new-features

- Feature updates earlier in 2026 — https://docs.snowflake.com/en/release-notes/feature-releases-2026
- Adaptive Compute — https://docs.snowflake.com/en/user-guide/warehouses-adaptive
- Gen2 default (behaviour change 2250) — https://docs.snowflake.com/release-notes/bcr-bundles/2026_03/bcr-2250
- Snowflake Optima — https://docs.snowflake.com/en/user-guide/snowflake-optima
- Cortex AI Functions — https://docs.snowflake.com/en/user-guide/snowflake-cortex/aisql
- Snowflake-managed MCP server — https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-agents-mcp
- Snowpipe Streaming high-performance architecture — https://docs.snowflake.com/en/user-guide/snowpipe-streaming-high-performance-overview
- Overview of Snowflake CoCo — https://docs.snowflake.com/en/user-guide/cortex-code/cortex-code
- Snowflake editions — https://docs.snowflake.com/en/user-guide/intro-editions
- Summary of data types — https://docs.snowflake.com/en/sql-reference/intro-summary-data-types
- MCP specification — https://modelcontextprotocol.io/specification

PART I

Foundations of Data and Cloud Computing

Levels 1 of 7. Everything a complete beginner needs before touching Snowflake: what data engineering is, how the cloud works, and SQL from the very first SELECT.

CHAPTER 1

Introduction to Data Engineering

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- explain what data is and how an organization turns raw records into decisions;
- classify data as structured, semi-structured or unstructured and give examples of each;
- distinguish OLTP from OLAP systems and explain why companies need both;
- compare databases, data warehouses, data lakes and lakehouses;
- describe the roles of data engineers, analysts and data scientists;
- explain ETL and ELT, batch and streaming, and when each is appropriate;
- draw a layered data pipeline from source systems to dashboards;
- define data quality, lineage, governance and observability in practical terms.

Prerequisites: none. This chapter assumes no technical background.

Where this leads: Chapter 2 explains the cloud infrastructure these ideas run on; Chapter 3 teaches the SQL language used to query data; Chapter 4 introduces Snowflake as one platform that implements the ideas in this chapter.

1.1 What data is and why organizations collect it

Concept

Data is a recorded fact. "Customer 4211 bought two USB-C chargers for ₹1,798 at 14:03 on 2 March 2026" is data. So is the temperature reading from a warehouse freezer, the text of a customer complaint, and a photograph of a damaged parcel.

A single fact is rarely useful. Value appears when many facts are **collected, organized, combined and summarized** so that a person — or a program — can make a better decision. That transformation from raw records to decisions is the subject of this book.

A useful mental model is the ladder from data to action:

Rung	Example at ByteMart	Question it answers
Data	2.4 million order rows from last year	What happened? (individual facts)
Information	Revenue per month per country	What happened, summarized?
Insight	UK revenue fell 18 % in Q3 while India grew 31 %	Why does it matter?
Decision	Shift Q4 marketing budget toward India	What should we do?

Rung	Example at ByteMart	Question it answers
Action & feedback	Campaign launched; measure results next month	Did it work?

Every rung depends on the one below. If the order data is incomplete, duplicated or late, every dashboard, forecast and AI answer built on it inherits the error. This is why organizations invest in **data engineering**: the discipline of reliably moving, shaping and protecting data so that the higher rungs can be trusted.

Why it matters

Consider what ByteMart's leadership wants to know in a typical week:

- Which products are selling, in which country, at what margin?
- Is the new checkout page causing more abandoned carts?
- Which customers are likely to stop buying (churn)?
- Are we complying with data-protection laws when we store customer addresses?
- Can a support agent ask a question in plain English ("what is our return policy for opened electronics in the UK?") and get a correct, sourced answer?

None of these can be answered from the orders database alone. They require combining data from several systems, keeping history, cleaning inconsistencies, and controlling who sees what.

1.2 Structured, semi-structured and unstructured data

Data arrives in three broad shapes. Snowflake handles all three, but differently, so learning to recognise them now will save time later.

Structured data

Structured data fits neatly into rows and columns with a fixed, known **schema** (the list of columns and the type of each). Every row has the same columns.

```
order_id | customer_id | order_date | country | amount_inr
---------+-------------+------------+---------+-----------
  10001  |    4211     | 2026-03-02 |   IN    |  1798.00
  10002  |    5530     | 2026-03-02 |   UK    |  4999.00
```

Sources: transactional databases, spreadsheets, ERP and CRM exports. Structured data is the easiest to query with SQL.

Semi-structured data

Semi-structured data has internal structure — names and values, nesting, lists — but the structure can vary from record to record and is described *inside* the data rather than by a separate schema. JSON is the most common format; XML, Avro, Parquet and ORC are others.

```
{
  "event": "add_to_cart",
  "ts": "2026-03-02T14:01:55Z",
```

```
  "user": {"id": 4211, "logged_in": true},
  "product": {"sku": "CHG-USBC-65W", "qty": 2},
  "device": {"os": "Android", "app_version": "5.12.0"},
  "experiment": ["new_checkout_v2"]
}
```

Notice the nesting (`user.id`), the list (`experiment`), and that a different event — say `page_view` — would carry different fields. Clickstream events, API responses, IoT telemetry and application logs are typically semi-structured. Snowflake stores such data in a type called `VARIANT` (Chapter 7) and lets you query nested fields directly with SQL.

Unstructured data

Unstructured data has no record-level structure that a database can interpret directly: free text, PDFs, images, audio, video. ByteMart's support tickets, product manuals and parcel photos are unstructured.

Until recently, unstructured data was mostly stored and ignored by analytical systems. Modern AI changes that: large language models can summarise tickets, extract fields from invoices, classify images and answer questions over manuals. Snowflake's Cortex AI functions (Chapters 24–25) and Cortex Search (Chapter 26) operate on exactly this data.

Shape	Schema	Typical formats	ByteMart example	Snowflake starting point
Structured	Fixed, external	Tables, CSV	Orders, customers	Standard tables (Ch. 6–7)
Semi-structured	Flexible, embedded	JSON, Avro, Parquet, XML	Clickstream events	`VARIANT` columns (Ch. 7–8)
Unstructured	None at record level	Text, PDF, images, audio	Tickets, manuals	Stages, `FILE` type, Cortex AI (Ch. 12, 25)

1.3 Operational versus analytical workloads: OLTP and OLAP

Concept

The single most important architectural idea in data engineering is that **running the business** and **analysing the business** are different workloads that need different systems.

- **OLTP** — *Online Transaction Processing* — systems run the business. When a shopper clicks "Buy", ByteMart's PostgreSQL database must insert an order, decrement stock and record a payment, in milliseconds, for thousands of shoppers at once, without ever losing or double-counting an order.
- **OLAP** — *Online Analytical Processing* — systems analyse the business. "Total revenue by country by month for the last three years" must read millions of rows and aggregate them.

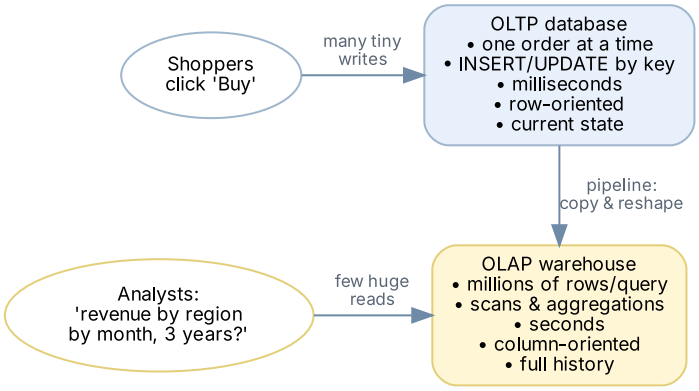


Figure 1.1 — OLTP systems serve many small writes; OLAP systems serve few, very large reads. A pipeline copies and reshapes data from one to the other.

Why one system is not enough

You might ask: why not just run the revenue query on the orders database? Three reasons:

1. **Interference.** A query that scans three years of orders consumes CPU, memory and disk bandwidth that checkout needs *now*. Analysts would slow down sales.
2. **Storage layout.** OLTP databases usually store data **row by row** — ideal for fetching or updating one whole order. Analytical queries typically need *a few columns from many rows* (just `order_date`, `country`, `amount`). A **columnar** store keeps each column together, so it reads only the columns needed and compresses them well. Chapter 5 shows how Snowflake does this with micro-partitions.
3. **History and integration.** The OLTP database holds *current state*: if a customer moves from Pune to Bengaluru, their row is overwritten. Analysis often needs *history* ("where did customers live when they ordered?") and data from other systems (marketing, ERP, clickstream). An analytical platform keeps history and integrates sources.

Characteristic	OLTP	OLAP
Purpose	Run the business	Analyse the business
Typical operation	Insert/update one record by key	Scan and aggregate millions of rows
Latency target	Milliseconds	Seconds to minutes
Concurrency	Thousands of tiny transactions	Tens to hundreds of large queries
Storage layout	Row-oriented	Column-oriented
Data scope	Current state, one application	History, many sources
Schema design	Normalised (many small tables)	Often denormalised (facts and dimensions)
ByteMart example	PostgreSQL orders database	Snowflake

WHERE SNOWFLAKE FITS

Snowflake is primarily an **OLAP** platform. It also offers **hybrid tables** (row-oriented storage with enforced keys for fast single-row operations) and **Snowflake Postgres** (a managed PostgreSQL service) for transactional needs, covered in Chapter 7. Even so, the core lesson stands: separate the operational and analytical workloads, even if they live on one vendor's platform.

1.4 Databases, data warehouses, data lakes and lakehouses

These four terms describe where data is kept and how it is organized. They evolved in response to one another, so they are easiest to understand in historical order.

Database

A **database** is an organized collection of data managed by a **database management system (DBMS)**, which provides storage, a query language (usually SQL), concurrency control and recovery. Every system in this list is technically a database; in everyday speech "database" usually means an *operational* (OLTP) database such as PostgreSQL, MySQL, Oracle or SQL Server.

Data warehouse

A **data warehouse** is a database designed for analysis. It collects data from many operational systems, cleans and integrates it, keeps history, and organizes it for querying — typically into **fact tables** (events you measure, like order lines) and **dimension tables** (the context, like customer, product and date). Chapter 9 teaches this modelling style in depth.

Traditional warehouses (Teradata, Oracle Exadata, Netezza, on-premises SQL Server) were powerful but expensive, ran on fixed hardware, and struggled with semi-structured data. Scaling meant buying a bigger appliance months in advance.

Data lake

A **data lake** is a large, cheap store of files — typically cloud **object storage** such as Amazon S3, Azure Blob Storage or Google Cloud Storage (Chapter 2) — holding data in its raw form: CSV, JSON, Parquet, images, logs. Lakes became popular around 2010–2015 because storage was cheap, any format could be kept, and engines such as Hadoop and Spark could process the files.

The weakness of the lake was *management*. Without transactions, schemas, access controls and quality checks, many lakes became "data swamps": nobody knew which file was current or trustworthy.

Lakehouse

A **lakehouse** combines the cheap, open file storage of a lake with the reliability features of a warehouse. The key enabler is an **open table format** — Apache Iceberg, Delta Lake or Apache Hudi — that adds a metadata layer on top of Parquet files so that engines can perform transactions, schema evolution and time travel on them. Multiple engines can read and write the same tables.

Snowflake participates in this world through **Iceberg tables** (Chapter 7): you can keep data in Iceberg format, in your own storage or in Snowflake-managed storage, and still query it with Snowflake's engine while other engines (Spark, Trino, others) read it too.

	Database (OLTP)	Data warehouse	Data lake	Lakehouse
Main purpose	Run applications	Governed analytics	Cheap storage of everything	Governed analytics on open files

	Database (OLTP)	Data warehouse	Data lake	Lakehouse
Data form	Tables	Tables	Files, any format	Open-format tables on files
Schema	On write	On write	On read	On write, evolvable
Transactions	Yes	Yes	No (files)	Yes (via table format)
Strength	Fast small operations	Performance, SQL, governance	Flexibility, cost	Openness + governance
Weakness	Poor at large scans	Historically costly, closed	Quality and governance	Operational complexity varies

DO NOT TREAT THESE AS RIVAL CAMPS

Modern platforms blur the lines. Snowflake is a cloud data warehouse that also stores semi-structured and unstructured data, reads and writes Iceberg tables, and offers transactional hybrid tables. What matters is choosing the right storage and table type for each workload, which later chapters teach.

1.5 Who does what: data engineering, analytics and data science

Three overlapping roles turn data into value. In small companies one person may do all three; in large ones they are separate teams.

Role	Core question	Typical outputs	Main tools
Data engineer	Is the right data available, correct, timely, secure and affordable?	Pipelines, models, data quality checks, platform configuration	SQL, Python, orchestration (tasks, dbt, Airflow), cloud services
Data / analytics engineer	Is the data modelled so the business can use it? (a role between engineering and analysis)	Tested transformation models, metric definitions, semantic layers	SQL, dbt, semantic views
Data analyst	What happened and why?	Dashboards, reports, ad-hoc analyses	SQL, BI tools (Power BI, Tableau, Snowsight)
Data scientist / ML engineer	What will happen, and what should we do?	Forecasts, classifiers, recommendation models, experiments	Python, notebooks, ML libraries, Snowpark ML
AI engineer (newer)	How do we build reliable applications on LLMs and our data?	RAG assistants, agents, evaluations, guardrails	Cortex AI, Cortex Search, Agents, MCP

This book trains the **data engineer** first and foremost, then extends into analytics engineering, platform architecture and AI engineering — because on a platform like Snowflake those responsibilities increasingly sit with the same person.

1.6 ETL and ELT

Concept

Moving data from sources to an analytical platform involves three steps:

- **Extract** — read data from the source system (a database query, an API call, a file export, a change feed).
- **Transform** — clean, convert, combine and reshape it (fix types, remove duplicates, join to reference data, compute metrics).
- **Load** — write it into the target platform.

The order of the last two steps defines two approaches.

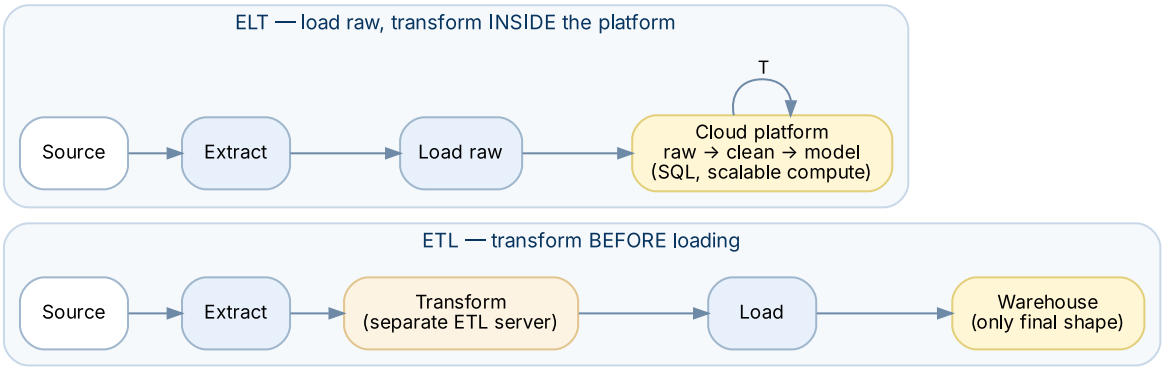


Figure 1.2 — ETL transforms data on a separate server before loading only the final shape; ELT loads raw data first and transforms it inside the analytical platform.

ETL — Extract, Transform, Load

In **ETL**, data is transformed *before* it reaches the warehouse, on a separate integration server (Informatica PowerCenter, SSIS, Talend, DataStage). This made sense when warehouse compute was scarce and expensive: you did not want to waste it on cleaning.

Disadvantages: raw data is not kept in the warehouse, so if a transformation was wrong, you must re-extract from the source (which may no longer have the old data). The integration server becomes a bottleneck and a separate system to scale and secure.

ELT — Extract, Load, Transform

In **ELT**, raw data is loaded first, then transformed *inside* the analytical platform using its own scalable compute — usually with SQL. Cloud platforms like Snowflake made this the dominant pattern because compute can be scaled up for a transformation and turned off afterwards (Chapter 2 explains this elasticity).

Advantages: raw data is preserved, so transformations can be fixed and re-run; transformations are written in SQL that analysts can read; one platform handles security and auditing. Tools such as **dbt** (Chapter 16) organise ELT transformations as version-controlled, tested SQL models.

Watch out for: loading everything raw can mean loading sensitive data you should not keep (for example, full card numbers). Some light transformation — filtering, masking, hashing — may still belong *before* load. Chapter 21 covers this.

	ETL	ELT
Where transformation runs	Separate integration engine	Inside the warehouse/lakehouse
Raw data kept in target?	Usually not	Yes
Re-processing after a bug	Re-extract from source	Re-run SQL on raw data
Typical era	On-premises warehouses	Cloud data platforms
Typical tools	Informatica, SSIS, DataStage	Snowflake SQL, dbt, Snowflake tasks, dynamic tables
Still appropriate when	Data must be masked/filtered before leaving source; target is not a scalable engine	Most cloud analytics

In practice most modern stacks are **ELT with a little ETL**: light cleansing or protection on the way in, heavy modelling inside Snowflake.

1.7 Batch and streaming processing

Concept

The second big design choice is *how often* data moves.

- **Batch processing** collects data over a period and processes it all at once — every night, every hour, every 15 minutes.
- **Streaming processing** handles each event (or a small micro-batch) within seconds of it happening.

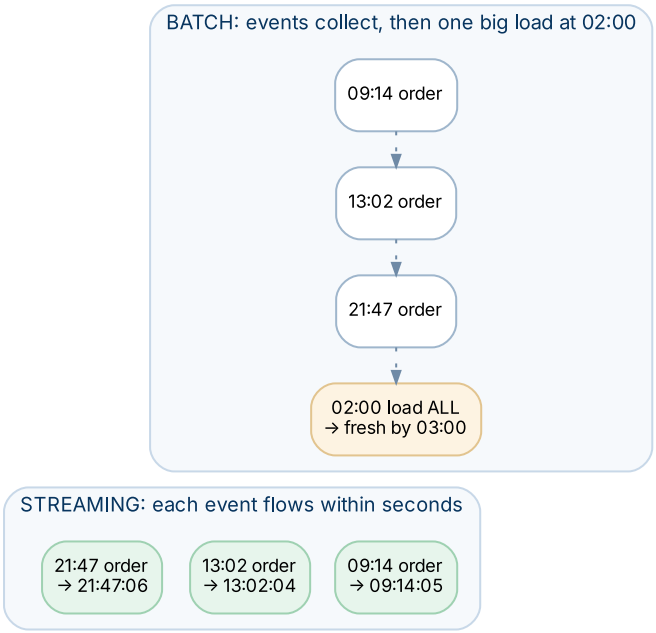


Figure 1.3 — In batch processing, data waits for a scheduled load; in streaming, each event becomes available within seconds.

Choosing between them

The key term is **latency** — the delay between something happening and it being available for analysis — and its business counterpart, **freshness**.

Need	Acceptable latency	Approach
Monthly finance reporting	Hours to a day	Daily batch
Daily sales dashboard	1 hour	Hourly batch or micro-batch
Stock levels shown on website	Minutes	Frequent micro-batch or streaming
Fraud check before payment completes	Sub-second	Streaming into an operational system (often not the warehouse)
"Trending now" products on homepage	Seconds to a minute	Streaming

> **COMMON MISTAKE: STREAMING EVERYTHING**
>
> Streaming systems are more complex to build, monitor and pay for than batch systems. Ask "what decision changes if this data is 5 seconds old instead of 1 hour old?" If the answer is "none", use batch. Snowflake makes it easy to move from batch to continuous ingestion later (Chapter 13), so start simple.

Snowflake supports a spectrum: `COPY INTO` for bulk batch loads (Chapter 12), **Snowpipe** for continuous file-based micro-batches, **Snowpipe Streaming** for row-level streaming with seconds of latency (Chapter 13), and **dynamic tables** that keep transformations fresh to a target lag (Chapter 15).

1.8 Data pipelines and layered architecture

Concept

A **data pipeline** is an automated sequence of steps that moves data from sources to consumers, transforming it along the way. Pipelines run repeatedly — on a schedule, when files arrive, or continuously — and must handle failures gracefully.

Well-designed pipelines are organised in **layers**, each with a clear purpose. The names vary between companies — *raw/staging/marts*, *landing/conformed/presentation*, or the **medallion** names *bronze/silver/gold* used later in this book — but the idea is the same.

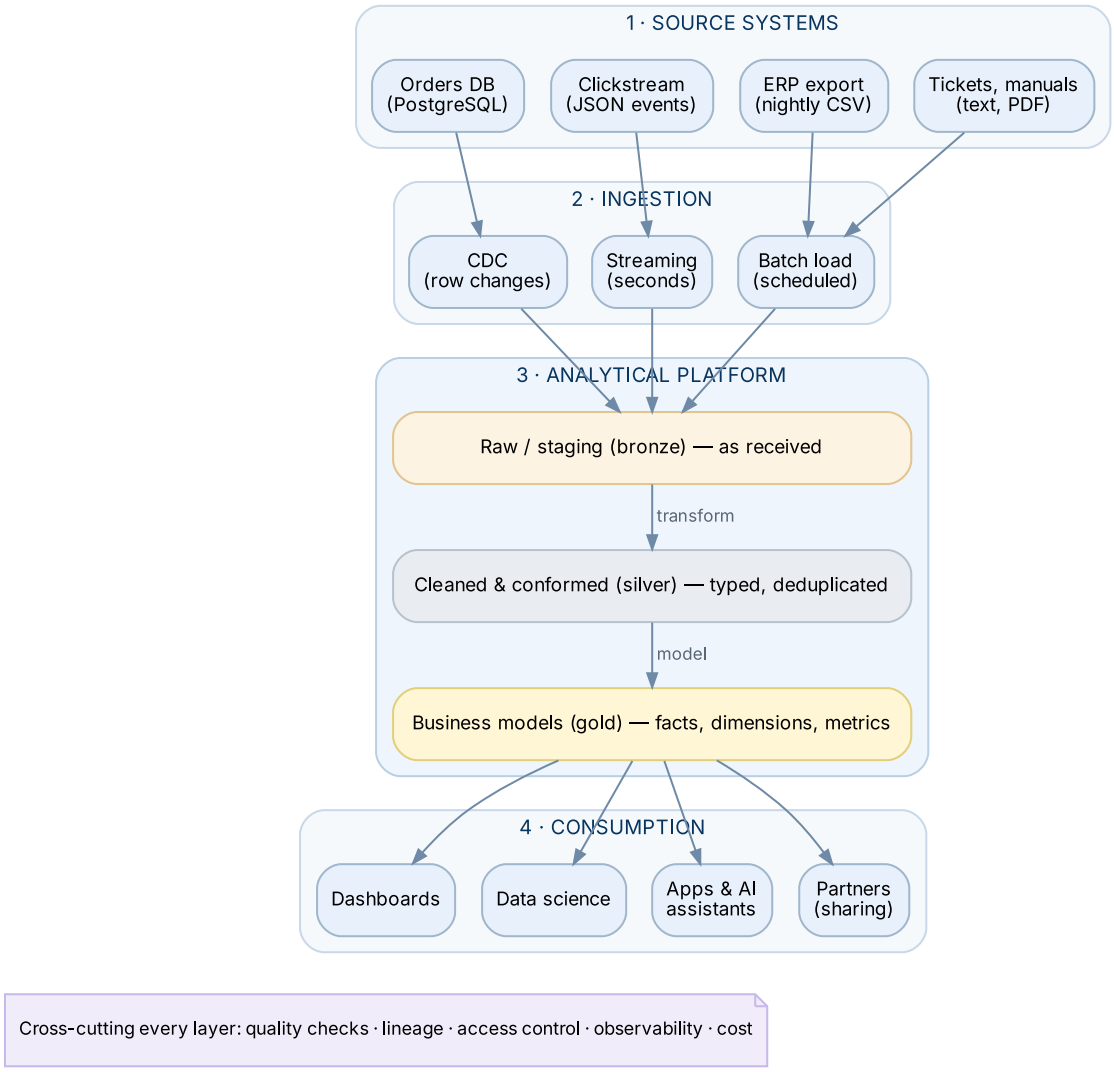


Figure 1.4 — ByteMart's layered pipeline: sources are ingested into a raw layer, cleaned and conformed, then modelled for consumption. Quality, lineage, access control and observability cut across every layer.

The layers in detail

1. Source systems. Where data is created. They are owned by other teams and were not designed for analytics. ByteMart's sources are its PostgreSQL orders database, nightly ERP CSV exports, JSON clickstream events and documents.

2. Ingestion. Getting data out of sources and into the platform. Three common patterns:

- *Full or incremental batch extracts* — "all orders changed since yesterday".
- *Change Data Capture (CDC)* — reading the database's change log to capture every insert, update and delete as it happens (Chapter 14).
- *Event streaming* — applications publish events to a message system such as Apache Kafka (Chapter 13).

3a. Raw / staging layer (bronze). Data stored *as received*, with minimal changes, plus metadata such as load time and source file name. Rule: never edit raw data — it is your evidence and your recovery point.

3b. Cleaned / conformed layer (silver). Data converted to correct types, deduplicated, validated, and *conformed* — the same customer identified the same way across systems, currencies standardised, time zones normalised.

3c. Business / presentation layer (gold). Data modelled for consumption: fact and dimension tables, aggregated metrics, feature tables for ML, curated document collections for AI search.

4. Consumption. Dashboards, notebooks, applications, AI assistants, partners receiving shared data.

Properties every pipeline needs

Property	Meaning	Example failure if missing
Correctness	Output reflects source truth	Revenue double-counted after a retry
Idempotency	Running the same step twice gives the same result as once	A re-run after failure inserts duplicate orders
Restartability	Can resume after failure without manual surgery	A failed night requires an engineer to delete partial data by hand
Freshness	Data is as recent as the business needs	Dashboard silently shows yesterday's data
Observability	You can see what ran, when, how long, with what result	Nobody notices the pipeline stopped for three days
Security	Only authorised people see sensitive data	Analysts see full customer phone numbers
Cost awareness	Compute and storage spending is understood and controlled	A misconfigured job runs every minute all month

Idempotency deserves special attention because it is the property beginners most often miss. Suppose a job appends yesterday's orders to a table and fails half-way. If you simply re-run it, the first half is inserted twice. An idempotent design instead *replaces* yesterday's slice, or *merges* by key, so that re-running is always safe. Chapters 9 and 14 implement idempotent loads in Snowflake.

1.9 Data quality, lineage, governance and observability

These four disciplines run across every layer. Beginners often treat them as "extras"; experienced engineers treat them as the difference between a platform people trust and one they work around.

Data quality

Data quality is the degree to which data is fit for its purpose. It is measured along several **dimensions**:

Dimension	Question	ByteMart check
Completeness	Is anything missing?	Every order has a `customer_id`
Validity	Does it obey the rules?	`amount` ≥ 0; `country` in the allowed list
Uniqueness	Are there duplicates?	`order_id` unique
Consistency	Do related values agree?	Order total = sum of its lines
Timeliness	Is it fresh enough?	Latest order in warehouse < 1 hour old
Accuracy	Does it match reality?	Warehouse revenue reconciles with finance ledger within 0.1 %

Quality checks should run *automatically* in the pipeline, and failures should stop or quarantine bad data rather than silently publishing it. Snowflake provides **data metric functions** for this (Chapter 19); dbt provides **tests** (Chapter 16).

Lineage

Data lineage records where data came from and how it was transformed: "the `gold.monthly_revenue` table is computed from `silver.orders` and `silver.fx_rates`, which come from `raw.orders_cdc` and the ERP export". Lineage lets you answer two critical questions:

- *Impact analysis:* "If I change this column, what breaks downstream?"
- *Root-cause analysis:* "This dashboard number is wrong — where did the error enter?"

Snowflake captures lineage automatically for many operations and shows it in Snowsight; its `ACCESS_HISTORY` view records which objects each query read and wrote (Chapters 19 and 21).

Governance

Data governance is the set of policies, roles and controls that ensure data is used correctly, securely and legally. In practice it includes:

- **Ownership** — every dataset has an accountable owner.
- **Classification** — knowing which columns contain personal or sensitive data.
- **Access control** — who may see what, at the level of databases, tables, rows and columns.
- **Retention** — how long data is kept and when it must be deleted.
- **Compliance** — meeting laws such as India's Digital Personal Data Protection Act, the EU/UK GDPR, or HIPAA for health data.

Snowflake implements governance with roles and privileges (Chapter 20), tags, masking policies and row access policies (Chapter 21).

Observability

Data observability means being able to see the health of your data and pipelines: did each job run, how long did it take, how many rows moved, did volumes or distributions change unexpectedly, how much did it cost? Without observability, failures are discovered by angry dashboard users. Chapter 19 builds monitoring and alerting on Snowflake's history views.

1.10 Guided lab: design ByteMart's first pipeline on paper

LAB 1.1 — PIPELINE DESIGN WORKSHEET

Goal: produce a one-page design for a daily sales dashboard. No software needed.

Scenario. ByteMart's head of sales wants a dashboard by 08:00 every morning showing yesterday's revenue, order count and average order value by country and product category, plus a 13-month trend.

Step 1 — Identify sources. List every system that holds a piece of the needed data. (Orders and order lines: PostgreSQL. Product category: ERP CSV export. Exchange rates for converting INR/GBP/USD into one currency: a finance API.)

Step 2 — Choose latency. The dashboard is needed at 08:00; data up to midnight suffices. → Daily batch is enough.

Step 3 — Choose ingestion per source. Orders: incremental extract of rows changed since the last run (or CDC). Products: full nightly CSV (small). FX rates: daily API call.

Step 4 — Define layers. Raw: `raw.orders`, `raw.order_lines`, `raw.products_csv`, `raw.fx_rates_json`. Silver: `silver.orders` (typed, deduplicated, one row per order), `silver.products`, `silver.fx_rates`. Gold: `gold.fct_daily_sales_by_country_category`.

Step 5 — Define quality checks. Unique `order_id`; no negative amounts; every order line maps to a known product; yesterday's order count within ±40 % of the 28-day average (alert, do not fail).

Step 6 — Define schedule and dependencies. 01:00 extract → 01:30 load raw → 02:00 transform silver → 02:30 build gold → 02:45 quality checks → 03:00 publish/refresh dashboard. Total slack before 08:00: 5 hours.

Step 7 — Define failure behaviour. If quality checks fail, the gold table is not published and an alert goes to the on-call engineer; the dashboard shows the last good data with a "stale" banner.

Expected result: a diagram resembling Figure 1.4 with your table names, plus a short table of checks and a timeline. Keep it — you will implement this exact pipeline in Snowflake in Project 2 (Chapter 40).

1.11 Production use case: from spreadsheets to a pipeline

ByteMart's analytics began like many companies': each Monday an analyst exported orders from the production database into Excel, pasted in a product list from the ERP, and built pivot tables. Problems accumulated:

- the export query slowed down checkout on Monday mornings (OLTP interference);

- product categories were pasted from an out-of-date file, so new products appeared as "Unknown";
- two analysts produced different revenue numbers because one included cancelled orders;
- nobody could answer "what did this report say last month?" because files were overwritten;
- the spreadsheet with customer phone numbers was emailed to an external agency.

The fixes map one-to-one onto this chapter's concepts: a separate analytical platform (OLAP) fed by incremental extracts (ingestion), a raw layer preserving history, conformed product data (silver), a single governed revenue definition (gold, semantic layer), and access controls with masking for phone numbers (governance). The rest of this book implements each fix in Snowflake.

1.12 Common mistakes

MISTAKES BEGINNERS MAKE IN PIPELINE DESIGN

1. **Querying production OLTP databases for analytics.** Use replicas, CDC or extracts.
2. **Transforming raw data in place.** Always keep an untouched raw copy.
3. **Non-idempotent loads.** Design every step so it can be safely re-run.
4. **No definition of "done".** Without quality checks, a pipeline that "succeeded" may have published garbage.
5. **Streaming by default.** Start with batch unless the business needs lower latency.
6. **Ignoring time zones.** Store timestamps with explicit zones or in UTC; ByteMart operates in IST, GMT/BST and US zones.
7. **One giant job.** Split work into layers and steps that can be monitored and retried independently.
8. **Governance later.** Classifying sensitive data after it has spread to twenty tables is far harder than at ingestion.

1.13 Troubleshooting exercise

EXERCISE — THE REVENUE NUMBERS DON'T MATCH

Finance reports March revenue of ₹4.82 crore. The new dashboard shows ₹5.06 crore. List at least five plausible causes and, for each, the evidence you would look for. (Solution at the end of the chapter.)

1.14 Knowledge check

1. Classify each as structured, semi-structured or unstructured: (a) a Parquet file of orders, (b) an MP3 of a support call, (c) a CSV of products, (d) a JSON webhook from a payment provider, (e) a scanned invoice PDF.
2. Give two reasons why running analytics directly on an OLTP database is a bad idea.
3. What problem did lakehouse table formats such as Apache Iceberg solve for data lakes?
4. Explain the difference between ETL and ELT in one sentence each, and give one situation where some transformation should still happen before load.

5. A job inserts yesterday's orders into a table. It failed after inserting half the rows and was re-run. What property was missing, and how would you redesign the job?
6. Name the six data-quality dimensions in Section 1.9 and give one ByteMart check for each.
7. What is the difference between lineage and observability?
8. A stakeholder asks for "real-time" stock levels in the warehouse. What question should you ask before agreeing?

1.15 Hands-on assignment

Choose a business you know well (a hospital, a bank, a food-delivery app, your own employer). Produce a two-page design document containing:

1. A table of at least four source systems with data shape (structured/semi/unstructured), owner and change frequency.
2. A layered pipeline diagram in the style of Figure 1.4.
3. A latency decision for each consumer, with justification.
4. Six data-quality checks covering all six dimensions.
5. A list of sensitive data elements and how you would protect them.
6. A list of three metrics you would monitor to know the pipeline is healthy.

1.16 Chapter summary

Data becomes valuable when it is collected, cleaned, combined and summarized reliably enough to support decisions. Operational (OLTP) and analytical (OLAP) workloads have opposite requirements, so organizations copy data from operational systems into analytical platforms through pipelines. Those platforms evolved from on-premises warehouses through data lakes to lakehouses built on open table formats; Snowflake sits at the convergence, offering warehouse capabilities over structured, semi-structured and unstructured data and participating in open formats through Iceberg. Modern pipelines generally follow ELT: load raw data, then transform inside the platform. Latency requirements determine whether batch or streaming is appropriate. Layered architectures — raw, cleaned, business — make pipelines understandable, recoverable and testable. Quality, lineage, governance and observability are not extras; they make the platform trustworthy.

1.17 Key takeaways

- Separate *running* the business (OLTP) from *analysing* it (OLAP).
- Keep raw data untouched; transform in layers.
- Design every step to be idempotent and restartable.
- Choose latency by business need, not fashion.
- Build quality checks, lineage, access control and monitoring from day one.

1.18 Further reading

- Snowflake — Introduction to unstructured data: https://docs.snowflake.com/en/user-guide/unstructured-intro
- Snowflake — Semi-structured data overview: https://docs.snowflake.com/en/user-guide/semistructured-intro
- Snowflake — Introduction to data quality and data metric functions: https://docs.snowflake.com/en/user-guide/data-quality-intro
- Apache Iceberg project: https://iceberg.apache.org

1.19 Solutions

Troubleshooting exercise — revenue mismatch. Plausible causes and evidence:

1. *Cancelled or refunded orders included.* Compare totals with and without `status IN ('CANCELLED','REFUNDED')`; ask finance for their definition.
2. *Duplicate orders from a re-run, non-idempotent load.* Count `order_id` values that appear more than once; check load history for re-runs.
3. *Different time-zone boundaries.* Finance may close March in IST while the dashboard uses UTC; orders between 00:00 and 05:30 IST on 1 April fall into different months. Recompute with consistent zone conversion.
4. *Taxes or shipping included in one figure but not the other.* Compare line-item amounts with order totals; check whether GST/VAT is included.
5. *Currency conversion differences.* Dashboard may convert GBP/USD at daily rates while finance uses month-end rates. Recompute with finance's rates.
6. *Late-arriving data.* Finance's ledger may include adjustments posted in April for March orders. Compare by posting date versus order date.
7. *Test or internal orders.* Check for staff or test customer IDs.

The general method — *agree the definition, then reconcile in steps, isolating one cause at a time* — is formalised as data reconciliation in Chapter 9.

Knowledge check answers.

1. (a) Semi-structured is the common classification for Parquet because it carries its own embedded schema and supports nesting — although a flat Parquet file of orders is effectively structured; either answer with the reasoning is acceptable. (b) Unstructured. (c) Structured. (d) Semi-structured. (e) Unstructured (until information is extracted from it).
2. Large analytical scans compete for resources with live transactions and can slow the business; row-oriented storage is inefficient for scanning a few columns of many rows; OLTP systems hold only current state, not history; data from other systems cannot be integrated there.
3. They added transactional metadata on top of files, enabling atomic commits, schema evolution, time travel and consistent reads by multiple engines — turning a pile of files into reliable tables.

4. ETL transforms data on a separate engine before loading only the final shape; ELT loads raw data first and transforms it with the target platform's compute. Transform before load when data must be masked, filtered or minimised before it leaves the source — for example, dropping full card numbers or hashing national IDs.
5. Idempotency. Redesign so the job deletes and replaces yesterday's slice in a single transaction, or uses a MERGE keyed on `order_id`, so a re-run produces the same final state.
6. Completeness (every order has a customer), validity (non-negative amount), uniqueness (unique `order_id`), consistency (order total equals sum of lines), timeliness (latest order less than an hour old), accuracy (reconciles with finance ledger).
7. Lineage describes *how data is derived* — which sources and transformations produced it. Observability describes *the operational health* of data and pipelines — whether jobs ran, volumes, freshness, errors and cost.
8. "What decision will you make differently with data that is seconds old rather than, say, 15 minutes old?" — and "where will the decision be made?" If stock levels drive the website, the source system or an operational store may be a better home than the warehouse.

CHAPTER 2

Cloud Computing Fundamentals

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- define cloud computing and the IaaS, PaaS and SaaS service models;
- distinguish public, private and hybrid clouds;
- explain compute, storage and networking, and how they are billed;
- recognise the equivalent core services on AWS, Microsoft Azure and Google Cloud;
- explain object storage and the common analytical file formats (CSV, JSON, Parquet, Avro, ORC);
- explain regions, availability zones and why location matters for latency, cost and law;
- describe identity and access management and private network connectivity;
- distinguish scalability, elasticity and availability;
- calculate simple consumption-based costs, including Snowflake credits;
- explain the shared-responsibility model and apply it to Snowflake.

Prerequisites: Chapter 1.

Where this leads: Chapter 4 explains how Snowflake is delivered on top of the three clouds; Chapter 12 uses object storage and storage integrations to load data; Chapter 18 builds on the billing concepts here; Chapter 20 deepens identity and network security.

2.1 What cloud computing is

Concept

Cloud computing is renting computing resources — processors, memory, disks, networks, and higher-level services built on them — over the internet, on demand, paying for what you use, instead of buying and operating your own hardware.

The U.S. National Institute of Standards and Technology (NIST) definition, still the standard reference, lists five essential characteristics:

Characteristic	Plain meaning	Example
On-demand self-service	You provision resources yourself, in minutes, without a human approving a purchase order	Create a server from a web console
Broad network access	Resources are reached over standard networks from any device	SQL clients, browsers, APIs

Characteristic	Plain meaning	Example
Resource pooling	The provider serves many customers from shared pools of hardware	Your virtual machine shares a physical host with others, isolated
Rapid elasticity	Capacity can grow and shrink quickly, often automatically	Add servers during a sale, remove them after
Measured service	Usage is metered and billed	Per-second compute, per-GB-month storage

Why it matters

Before cloud, a company that wanted a data warehouse estimated its peak load for the next three to five years, bought hardware for that peak, waited months for delivery, and paid for idle capacity the rest of the time. If it guessed too low, queries slowed; too high, money was wasted.

With cloud, ByteMart can run a large compute cluster for twenty minutes during the nightly transformation and pay only for those twenty minutes. This change in economics is the foundation of Snowflake's design.

2.2 Service models: IaaS, PaaS and SaaS

The service model describes *how much* of the technology stack the provider manages for you.

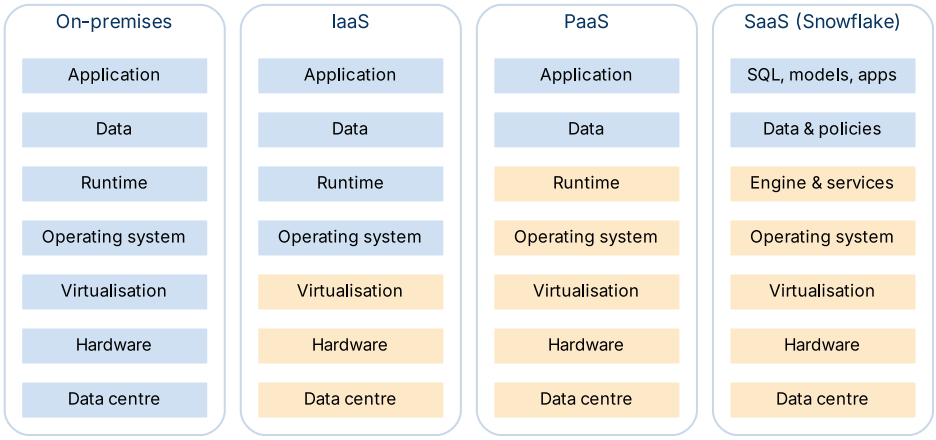


Figure 2.1 — Who manages what under each service model. Snowflake is consumed as SaaS: you manage your data, SQL, access policies and applications; Snowflake manages everything beneath.

- **IaaS — Infrastructure as a Service.** You rent virtual machines, disks and networks. You install and patch the operating system, the database software and everything above it. Examples: Amazon EC2, Azure Virtual Machines, Google Compute Engine.
- **PaaS — Platform as a Service.** The provider runs the operating system and middleware; you deploy code or data. Examples: Amazon RDS for PostgreSQL, Azure SQL Database, Google Cloud SQL.
- **SaaS — Software as a Service.** You use a finished application through a browser or API. The provider runs everything. Examples: Salesforce, Microsoft 365 — and **Snowflake**.

Snowflake is often described as a fully managed service: there is no software to install, no hardware or operating system to configure, no indexes to rebuild, no upgrades to schedule. You create objects with SQL, load data, run queries, and Snowflake handles the rest. Understanding this explains many Snowflake behaviours later in the book — for example, why you never choose disk types or patch versions, and why some low-level tuning knobs that exist in other databases simply do not exist.

2.3 Public, private and hybrid clouds

Model	Definition	Typical reason
Public cloud	Infrastructure owned by a provider and shared (securely isolated) among many customers	Scale, speed, no capital expenditure
Private cloud	Cloud-style self-service on infrastructure dedicated to one organization, on-premises or hosted	Regulatory or legacy constraints
Hybrid cloud	A combination: some workloads on-premises or private, some in public cloud, connected	Gradual migration; data that must stay on-premises
Multi-cloud	Using more than one public cloud provider	Avoid lock-in, regional coverage, acquisitions

Snowflake runs only on **public cloud** infrastructure — AWS, Microsoft Azure and Google Cloud — but it is commonly part of hybrid architectures, ingesting data from on-premises systems. Snowflake also supports multi-cloud designs: an organization can have accounts on different clouds and replicate or share data between them (Chapters 21 and 38).

For customers with the strictest isolation requirements, Snowflake offers **Virtual Private Snowflake (VPS)**, an edition in which the account runs in a completely separate Snowflake environment that shares no hardware with accounts outside it (Chapter 4).

2.4 Compute, storage and networking

Every cloud service is built from three resource types.

Compute

Compute is processing power: CPU cores and memory that execute instructions. It is delivered as virtual machines, containers or serverless functions, billed by time (per second or per hour) and size.

Compute is the expensive, *perishable* resource: an idle server still costs money. The most important cost habit in the cloud — and in Snowflake — is **turning compute off when it is not needed**.

In Snowflake, compute is provided by **virtual warehouses** (and, for some features, by **serverless** compute that Snowflake sizes automatically). A warehouse can be started, resized and suspended in seconds and is billed only while running (Section 2.9).

Storage

Storage keeps data persistently. Three kinds matter:

Kind	How you use it	Analogy	Example
Block storage	A virtual disk attached to one server	A hard drive	Amazon EBS, Azure Managed Disks
File storage	A shared folder hierarchy over a network	A network drive	Amazon EFS, Azure Files
Object storage	Objects (files) in buckets, accessed via HTTP API	An infinitely large, durable warehouse of labelled boxes	Amazon S3, Azure Blob Storage, Google Cloud Storage

Storage is cheap and *durable* compared with compute. A key Snowflake design principle is to keep data in cheap, durable cloud storage and attach compute only when needed — the **separation of storage and compute** (Chapter 5).

Networking

Networking connects resources to each other and to users. Concepts you will meet:

- **Virtual network** — a private, isolated network inside a cloud account (AWS *VPC*, Azure *VNet*, Google *VPC*).
- **Subnet** — a range of addresses within a virtual network.
- **Public endpoint** — a service reachable over the internet (encrypted with TLS).
- **Private endpoint / private link** — a service reachable only through the cloud provider's private network, never traversing the public internet (Section 2.8).
- **Ingress / egress** — traffic coming into / leaving a network or region. Cloud providers usually charge for **egress** — data leaving a region or the cloud — which matters when you move data between regions or out to users.

2.5 AWS, Azure and Google Cloud — the services that matter for Snowflake

You do not need to be an expert in any cloud to use Snowflake, but you will interact with the cloud where *your data originates* — especially object storage and identity. This table maps the equivalent services.

Purpose	AWS	Microsoft Azure	Google Cloud
Virtual machines	EC2	Virtual Machines	Compute Engine
Object storage	S3 (buckets)	Blob Storage / ADLS Gen2 (storage accounts, containers)	Cloud Storage (buckets)
Identity & access	IAM (users, roles, policies)	Microsoft Entra ID + Azure RBAC	Cloud IAM (principals, roles)
Event notifications on new files	S3 Event Notifications → SQS / SNS	Event Grid	Pub/Sub notifications
Message streaming	Amazon MSK (Kafka), Kinesis	Event Hubs	Pub/Sub, Managed Kafka
Key management	AWS KMS	Azure Key Vault	Cloud KMS
Private connectivity	AWS PrivateLink	Azure Private Link	Private Service Connect

Purpose	AWS	Microsoft Azure	Google Cloud
Managed relational DB	RDS, Aurora	Azure SQL Database, Azure Database for PostgreSQL	Cloud SQL, AlloyDB

HOW THIS APPEARS IN SNOWFLAKE

When you create a Snowflake account you choose a **cloud** (AWS, Azure or Google Cloud) and a **region** on that cloud. Snowflake then provisions the account on that provider. Your data, tables and compute live there. You can still load data from *any* cloud's object storage — cross-cloud loading works but incurs the source provider's egress charges and higher latency, so it is usual to choose the same cloud and region as your main data sources.

2.6 Object storage and analytical file formats

Object storage in depth

Object storage is the landing zone for most data entering Snowflake, so it repays careful study.

- An **object** is a file plus metadata, identified by a **key** — a name that looks like a path: `orders/2026/03/02/orders_0001.csv.gz`. The "folders" are only prefixes in the key; object storage has no real directories.
- Objects live in a **bucket** (S3, GCS) or **container** inside a **storage account** (Azure).
- Objects are **immutable**: to change one, you write a new version.
- Durability is extremely high (providers design for eleven nines on standard tiers), but availability and access speed vary by **storage class** (hot, cool, archive).
- Access is controlled by the cloud's IAM policies — Section 2.7.

In Snowflake, a **stage** is a named pointer to a location in object storage (an *external stage*) or to Snowflake-managed storage (an *internal stage*). You load from and unload to stages (Chapter 12).

File formats for analytics

Format	Type	Structure	Compression	Strengths	Weaknesses
CSV / TSV	Text, row	Flat; schema external	External (gzip, etc.)	Universal, human-readable	No types; quoting and delimiter problems; large
JSON / NDJSON	Text, row	Nested; self-describing	External	Flexible, ubiquitous in APIs	Verbose; repeated field names; slow to parse
XML	Text, row	Nested; self-describing	External	Legacy integrations	Very verbose
Avro	Binary, row	Schema stored in file	Built-in	Schema evolution; streaming (Kafka)	Not columnar
Parquet	Binary, **columnar**	Schema stored; nesting supported	Built-in, per column	Fast analytical reads; small; types preserved	Not human-readable; inefficient for tiny files

Format	Type	Structure	Compression	Strengths	Weaknesses
ORC	Binary, columnar	Schema stored	Built-in	Similar to Parquet; Hive ecosystem	Less common outside Hadoop

Row versus columnar files. A row format stores each record's fields together; a columnar format stores each column's values together. For analytics that read a few columns of many rows, columnar formats read far less data and compress better (similar values sit next to each other). This is the same principle Snowflake uses internally in micro-partitions (Chapter 5).

> **FORMAT GUIDANCE FOR SNOWFLAKE LOADING**
>
> Prefer compressed files, and aim for files of roughly 100–250 MB compressed for bulk loading — Snowflake parallelises across files, so many tiny files add overhead and one giant file limits parallelism. Chapter 12 explains the reasoning and the current documented recommendations.

2.7 Identity and access management

Concept

Identity and Access Management (IAM) answers two questions for every request:

1. **Authentication** — *Who are you?* (password, multi-factor code, certificate, key pair, token from an identity provider)
2. **Authorization** — *What are you allowed to do?* (read this bucket, run this query)

The core building blocks are similar on every cloud:

- **Principals** — users, groups, and *non-human identities* such as service accounts or roles used by applications.
- **Permissions** — allowed actions on resources (`s3:GetObject` on a bucket).
- **Policies / role assignments** — documents binding principals to permissions.
- **Roles** — collections of permissions that principals can assume temporarily.

Least privilege

The guiding rule is **least privilege**: give each identity only the permissions it needs, for as long as it needs them. An application that only reads files should not be able to delete them.

How Snowflake connects to your cloud identity

When Snowflake reads files from your S3 bucket, it must authenticate to AWS. The insecure way is to paste an access key and secret into SQL. The secure way is a **storage integration** — a Snowflake object that represents a trust relationship: Snowflake is given an identity (on AWS, it assumes an IAM role you create with a trust policy naming Snowflake's principal). No secret ever appears in SQL, and you can revoke access from the cloud side at any time.

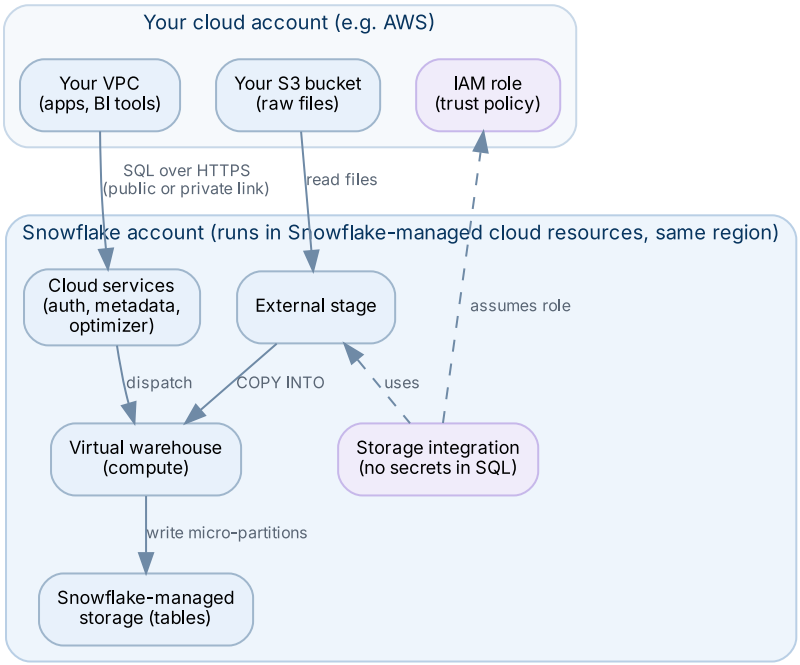


Figure 2.2 — Snowflake reading from your cloud storage through a storage integration, and serving queries from your network. Dashed lines indicate trust relationships rather than data flow.

Snowflake has its own, separate access-control system — users, **roles** and **privileges** on Snowflake objects (Chapters 6 and 20). Keep the two layers distinct in your mind: *cloud IAM* controls what Snowflake can do in your cloud account; *Snowflake RBAC* controls what users can do inside Snowflake.

2.8 Regions, availability zones and private connectivity

Regions and availability zones

A **region** is a geographic area containing a cluster of data centres — for example, AWS `ap-south-1` (Mumbai), Azure Central India (Pune), Google Cloud `asia-south1` (Mumbai). Each region contains several **availability zones (AZs)**: physically separate data-centre groups with independent power and networking, connected by low-latency links.

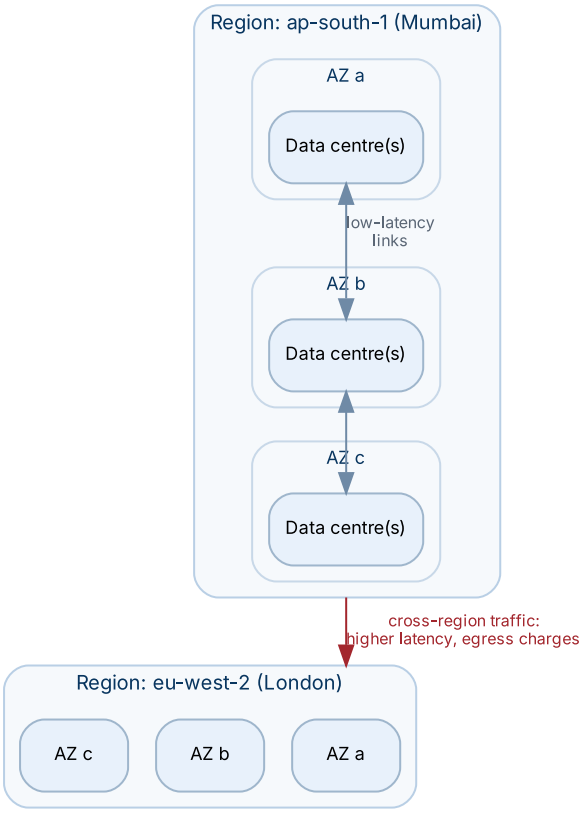


Figure 2.3 — A region contains several availability zones. Traffic between regions is slower and usually incurs egress charges.

Why region choice matters:

1. **Latency.** Users and data sources far from the region see slower responses.
2. **Cost.** Prices differ by region; moving data between regions costs egress.
3. **Law and residency.** Regulations may require data about a country's residents to stay in that country or bloc.
4. **Feature availability.** New features — including specific AI models in Snowflake Cortex — often launch in some regions first.
5. **Resilience.** A whole region can fail; disaster-recovery plans place a copy elsewhere.

Availability zones and Snowflake. Snowflake's service is designed to tolerate the loss of an availability zone within a region; you do not configure this. Protection against the loss of an entire region requires *replication* of databases and accounts to another region — a customer-configured capability covered in Chapter 39.

Private connectivity

By default, clients connect to Snowflake over the public internet using TLS encryption. Many enterprises require that traffic never leave the cloud provider's private backbone. **Private connectivity** services — AWS PrivateLink, Azure Private Link, Google Cloud Private Service Connect — expose a service as a private endpoint inside the customer's virtual network.

EDITION REQUIREMENT

Private connectivity to the Snowflake service and to internal stages requires **Business Critical Edition** or higher, according to Snowflake's edition matrix at the research cutoff. Network policies (allow-lists of IP addresses or private endpoints) are available in all editions.

2.9 Scalability, elasticity and availability

These three terms are often confused.

Term	Definition	Snowflake example
Scalability	The *ability* to handle more load by adding resources	Snowflake can process a 10× larger table by using a larger warehouse
Scale up (vertical)	Use a bigger machine	Resize a warehouse from MEDIUM to LARGE
Scale out (horizontal)	Use more machines in parallel	A multi-cluster warehouse adds clusters when many users query at once
Elasticity	Scaling *automatically and quickly* in both directions as demand changes, so you pay only for what is used	Auto-resume when a query arrives; auto-suspend after 60 s idle; clusters added and removed automatically
Availability	The proportion of time a service is usable	Snowflake's service level agreement; tolerance of AZ failure
Durability	The likelihood that stored data is not lost	Cloud object storage beneath Snowflake tables

A useful mental distinction: **scaling up** makes *individual* large queries faster; **scaling out** lets *more* queries run at the same time without queuing. Chapter 17 turns this into a decision procedure.

2.10 Cloud billing and consumption-based pricing

General cloud pricing

Cloud providers charge for:

- **Compute** — per second or per hour, by instance size;
- **Storage** — per GB-month, by storage class;
- **Requests** — per thousand API calls (for object storage);
- **Data transfer** — usually free inbound, charged for egress across regions or to the internet.

How Snowflake charges

Snowflake uses its own consumption units. At a high level, an account is billed for:

Component	Unit	How it accrues
Virtual warehouse compute	Credits	While a warehouse is running, by size; billed per second with a 60-second minimum each time it starts or resumes

Component	Unit	How it accrues
Serverless features	Credits	Compute Snowflake manages for you — e.g. Snowpipe, automatic clustering, serverless tasks, search optimization, many AI features
Cloud services	Credits	Metadata, authentication, query compilation; billed only for the portion that exceeds 10 % of the day's warehouse compute
Storage	Per terabyte per month	Average compressed bytes stored, including Time Travel and Fail-safe history and internal stages
Data transfer	Per byte	Egress to another region or cloud (loading *into* Snowflake is generally not charged by Snowflake)
AI features	Credits or tokens	Cortex functions and services, priced per model and feature

The **price of a credit** depends on your edition, cloud region and contract (on-demand versus capacity commitment), so this book never quotes a currency figure as fact. When an example needs money, it uses an explicitly labelled *illustrative* price.

Warehouse size and credits

For a classic (Gen1) standard virtual warehouse, credit consumption per hour doubles with each size step:

Size	Credits per hour (Gen1 standard)
X-Small	1
Small	2
Medium	4
Large	8
X-Large	16
2X-Large	32
3X-Large	64
4X-Large	128
5X-Large	256
6X-Large	512

Gen2 standard warehouses, Snowpark-optimized warehouses and Adaptive Warehouses consume credits at different rates or on a per-query basis; always check the current **Snowflake Service Consumption Table** for exact figures.

Worked example: the economics of elasticity

ByteMart's nightly transformation takes 40 minutes on a Medium warehouse (4 credits/hour).

- Credits used: 4 × 40/60 = **2.67 credits**.
- If the same job takes 20 minutes on a Large (8 credits/hour) because it parallelises well: 8 × 20/60 = **2.67 credits** — *the same cost, finished twice as fast.*

- If the warehouse is mistakenly left running all day at Medium: 4 × 24 = **96 credits** — 36 times more, for no extra work.

Two lessons follow, and they recur throughout the book:

1. **For work that parallelises, a bigger warehouse can cost the same and finish sooner** — but only up to the point where the work stops parallelising (Chapter 17 shows how to measure that).
2. **Idle compute is the most common source of waste.** Configure auto-suspend (Chapter 6) and monitor spending (Chapter 18).

2.11 The shared-responsibility model

Concept

In the cloud, security is a **shared responsibility**. The provider secures what it operates; the customer secures what it configures and uses. Misunderstanding the boundary is a leading cause of data breaches — not because the provider failed, but because a customer assumed the provider was handling something it was not.

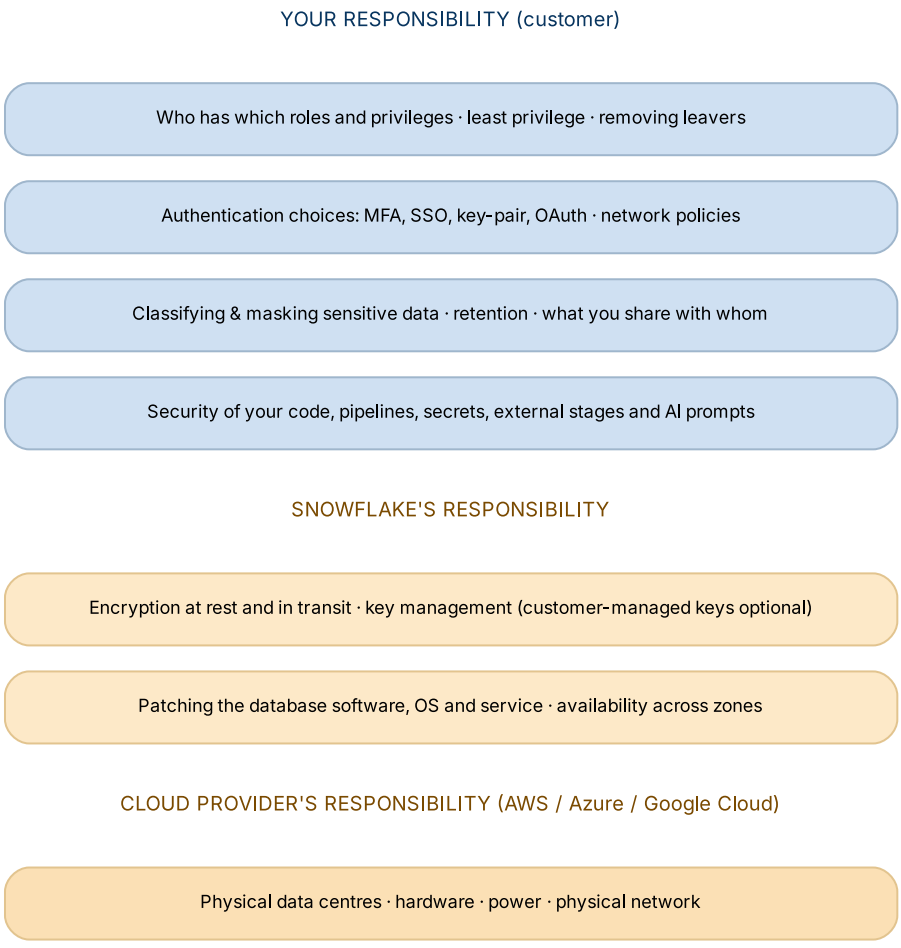


Figure 2.4 — The shared-responsibility model as it applies to Snowflake. The higher the layer, the more of it is yours.

Applying it to Snowflake

Snowflake encrypts all data at rest and in transit, manages keys, patches the software, and runs the service across availability zones. **You** decide:

- who gets which roles, and whether anyone uses `ACCOUNTADMIN` for daily work;
- whether multi-factor authentication, single sign-on or key-pair authentication is enforced;
- which IP addresses or private endpoints may connect;
- which columns are masked and which rows each role can see;
- what data you share with partners;
- what your applications, pipelines and AI agents are allowed to do with the privileges you give them.

SECURITY RISK: ASSUMING 'MANAGED' MEANS 'SECURE BY DEFAULT'

A fully managed service removes infrastructure work, not accountability. A Snowflake account where every analyst has a powerful role, passwords are not protected by MFA, and sensitive columns are unmasked is insecure no matter how well Snowflake runs the platform. Chapters 20–22 cover the customer side in depth.

2.12 Guided lab: map cloud concepts to Snowflake

LAB 2.1 — CLOUD-TO-SNOWFLAKE MAPPING (PAPER EXERCISE)

Goal: connect each cloud concept to the Snowflake feature you will use later.

Copy and complete this table. Answers are in Section 2.19.

Cloud concept	Snowflake feature that uses or replaces it	Chapter
Object storage bucket	?	12
IAM role with trust policy	?	12, 20
Auto-scaling compute cluster	?	5, 17
Per-second compute billing	?	18
Region selection	?	4
Private endpoint	?	20
Event notification on file arrival	?	13
Key management service	?	20

2.13 Production use case: choosing ByteMart's region

ByteMart's customers and orders database are in India; the UK business is growing; the US business is small. The data team must choose where to create the Snowflake account.

Considerations: the orders database runs on AWS in Mumbai (`ap-south-1`); Indian data-protection obligations favour keeping Indian customer data in India; most analysts are in Bengaluru; the UK team wants low-latency dashboards; Cortex AI features are needed and must be available in the chosen region or reachable through cross-region inference.

Decision: create the primary account on **AWS Asia Pacific (Mumbai)**, co-located with the main source, minimising egress and latency. UK dashboards query the same account (latency of a few hundred milliseconds is acceptable for dashboards). If UK regulators later require UK data residency, a second account in London can hold UK data, with only aggregated, non-personal data shared back. Disaster recovery: replicate critical databases to a second Indian region if available on the chosen cloud, or another approved location (Chapter 39). The AI-availability check is repeated whenever a new model is adopted.

2.14 Common mistakes

CLOUD MISTAKES THAT FOLLOW TEAMS INTO SNOWFLAKE

1. **Leaving compute running.** Always set auto-suspend.
2. **Embedding access keys in code or SQL.** Use storage integrations, OAuth, key-pair authentication or secrets objects.
3. **Choosing a region far from the data.** Pay egress and latency forever.
4. **Ignoring egress.** Unloading terabytes to another region or cloud is not free.
5. **Millions of tiny files.** Per-file overhead dominates; batch files to sensible sizes.
6. **Confusing cloud IAM and Snowflake roles.** Both must be configured correctly.
7. **Over-trusting "managed".** Configuration is still your responsibility.

2.15 Troubleshooting exercise

EXERCISE — THE MONTHLY BILL DOUBLED

ByteMart's Snowflake compute spend doubled in one month, although data volume grew only 5 %. List the cloud-billing causes you would investigate, in order, and what you would check for each. (Solution at the end of the chapter.)

2.16 Knowledge check

1. Which service model is Snowflake, and name three things you therefore never manage?
2. What is the difference between scaling up and scaling out, and which problem does each solve?
3. Why is egress relevant when choosing a Snowflake region?
4. Why is Parquet usually faster than CSV for analytical queries?
5. What is a storage integration and why is it preferred over access keys?
6. A warehouse of size Large runs for 90 seconds, suspends, then resumes for 30 seconds. Using Gen1 standard rates and per-second billing with a 60-second minimum per resume, how many credits are consumed?

7. Under the shared-responsibility model, who is responsible for (a) encrypting data at rest, (b) enforcing MFA for users, (c) masking customer phone numbers, (d) patching the query engine?
8. Name two reasons a feature might be available in one Snowflake region but not another.

2.17 Hands-on assignment

1. Create a free account on any public cloud you choose, create an object-storage bucket, upload a small CSV file, and write down (a) the region, (b) the object key, (c) the storage class, (d) how access is controlled. Delete the bucket afterwards.
2. Write a one-page region-selection memo for a fictional company of your choice, covering latency, cost, law, feature availability and disaster recovery.
3. Using the credit table, estimate monthly credits for a workload that runs a Small warehouse for 3 hours on weekdays and an X-Large for 25 minutes nightly. State your assumptions.

2.18 Chapter summary

Cloud computing rents compute, storage and networking on demand and bills by consumption. Snowflake is delivered as SaaS on AWS, Azure and Google Cloud: you choose a cloud and region, and Snowflake manages everything beneath your data, SQL and policies. Object storage is the main landing zone for files, and columnar formats such as Parquet are efficient for analytics. Cloud IAM controls what Snowflake can do in your cloud account; Snowflake's own role-based access control governs what users do inside Snowflake. Regions affect latency, cost, law and feature availability. Elasticity — starting, resizing and suspending compute quickly — is the economic heart of Snowflake, and idle compute is the most common waste. Security is shared: Snowflake secures the platform; you secure your configuration and usage.

2.19 Key takeaways

- Snowflake is SaaS on public cloud; choose the cloud and region deliberately.
- Separate storage (cheap, durable) from compute (expensive, perishable).
- Use trust-based integrations, never embedded keys.
- Bigger compute can cost the same and finish sooner for parallel work; idle compute is pure waste.
- "Managed" does not mean "configured securely for you".

2.20 Further reading

- Snowflake — Supported cloud platforms: https://docs.snowflake.com/en/user-guide/intro-cloud-platforms
- Snowflake — Supported cloud regions: https://docs.snowflake.com/en/user-guide/intro-regions
- Snowflake — Understanding compute cost: https://docs.snowflake.com/en/user-guide/cost-understanding-compute
- Snowflake — Snowflake editions: https://docs.snowflake.com/en/user-guide/intro-editions

- Snowflake — Configuring a storage integration for Amazon S3: https://docs.snowflake.com/en/user-guide/data-load-s3-config-storage-integration
- NIST SP 800-145, The NIST Definition of Cloud Computing: https://csrc.nist.gov/pubs/sp/800/145/final

2.21 Solutions

Lab 2.1 mapping.

Cloud concept	Snowflake feature	Chapter
Object storage bucket	External stage (internal stage for Snowflake-managed storage)	12
IAM role with trust policy	Storage integration (and other integration objects)	12, 20
Auto-scaling compute cluster	Virtual warehouse with auto-resume/suspend; multi-cluster warehouse; Adaptive Warehouse	5, 17
Per-second compute billing	Credits per second with 60-second minimum per resume	18
Region selection	Account region chosen at account creation	4
Private endpoint	AWS PrivateLink / Azure Private Link / Private Service Connect (Business Critical+)	20
Event notification on file arrival	Snowpipe auto-ingest	13
Key management service	Snowflake-managed keys; Tri-Secret Secure for customer-managed keys (Business Critical+)	20

Troubleshooting — bill doubled. Investigate in this order: (1) **warehouses not suspending** — auto-suspend disabled or set very high, or a BI tool sending keep-alive queries; check warehouse metering per hour; (2) **a new or changed schedule** — a task now running every minute instead of hourly; check task history; (3) **warehouse resized** and never resized back; (4) **multi-cluster settings** — maximum clusters raised, so concurrency spikes add clusters; (5) **serverless features** newly enabled — automatic clustering on a large, frequently changing table, search optimization, or AI functions applied to millions of rows; check metering by service type; (6) **inefficient new queries** — a dashboard with a missing filter scanning full history; check query history by credits; (7) **cloud services** exceeding the 10 % allowance because of very many tiny queries or metadata operations. Chapter 18 implements these checks as SQL against `ACCOUNT_USAGE`.

Knowledge check answers.

1. SaaS. You never manage hardware, the operating system, database software installation and patching, storage layout or index maintenance.
2. Scaling up (bigger warehouse) speeds up individual large or complex queries; scaling out (more clusters) handles more concurrent queries without queuing.
3. Data moving out of a region or cloud is charged as egress, so a Snowflake region far from your sources or consumers causes continuing transfer charges as well as latency.
4. Parquet is columnar and compressed: a query reads only the needed columns and fewer bytes, and types are stored so parsing is cheaper.

5. A Snowflake object representing a trust relationship with your cloud's IAM (for example, an assumable IAM role). No long-lived secret is stored in SQL or code, and access can be revoked or scoped from the cloud side.
6. Large = 8 credits/hour. First run: 90 s billed = 8 × 90/3600 = 0.2 credits. Second run: 30 s, but the 60-second minimum applies → 8 × 60/3600 ≈ 0.133 credits. Total ≈ **0.333 credits**.
7. (a) Snowflake; (b) the customer (configuring and enforcing MFA); (c) the customer; (d) Snowflake.
8. The underlying cloud provider may not offer required hardware (for example GPUs for a model) in that region; Snowflake releases features region by region; regulatory or certification constraints (for example government regions).

CHAPTER 3

SQL From Zero

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- explain tables, rows, columns, schemas, primary keys and foreign keys;
- write SELECT queries with filtering, sorting, grouping and aggregation;
- reason correctly about NULL and three-valued logic;
- combine tables with inner, outer, cross, self and anti joins — and avoid row multiplication;
- use subqueries, common table expressions (CTEs) and CASE expressions;
- manipulate dates and strings;
- use window functions for ranking, running totals, comparisons with previous rows and moving averages;
- combine results with set operators;
- create and change tables and data with DDL and DML, including MERGE;
- use transactions and constraints;
- explain the logical order in which SQL clauses are evaluated;
- apply common intermediate patterns: top-N per group, deduplication, percent of total, conditional aggregation.

Prerequisites: Chapter 1 (concepts of tables and OLTP/OLAP). No programming experience is needed.

Where this leads: Chapter 8 extends this chapter with Snowflake-specific SQL (semi-structured data, `FLATTEN`, `PIVOT`, Time Travel, recursive CTEs, metadata views). Chapter 9 applies SQL to data modelling problems.

3.0 How to practise this chapter

You do not need a Snowflake account yet. Every query in this chapter was **executed on DuckDB**, a free SQL engine that runs inside Python, and the result tables shown beneath each query are the actual output. DuckDB's syntax for everything in this chapter matches Snowflake's closely; where Snowflake behaves differently, a note says so.

To follow along on your own machine:

```
pip install duckdb
python -c "import duckdb; con = duckdb.connect('bytemart.duckdb'); con.sql('SELECT 42 AS
answer').show()"
```

Or use DuckDB's command-line shell, or the DuckDB UI. Once you have a Snowflake account (Chapter 4), every query here also runs in a Snowsight worksheet after you create the tables.

> **HOW TO LEARN SQL EFFICIENTLY**
> Type every query yourself rather than copying it. Before running it, predict the result. When your prediction is wrong, you have found exactly the thing you needed to learn.

3.1 Tables, rows, columns, schemas and keys

Concept

A relational database stores data in **tables**. A table has:

- **columns** (also called fields or attributes), each with a **name** and a **data type** — for example `order_id` of type `INTEGER`;
- **rows** (also called records or tuples), each holding one value per column.

A **schema** has two meanings in SQL, and you need both:

1. *Structure:* the set of column names and types of a table ("the orders table's schema").
2. *Namespace:* a named container for tables and other objects inside a database ("the `sales` schema"). In Snowflake you write `database.schema.table`, for example `bytemart.sales.orders` (Chapter 6).

Keys identify rows and connect tables:

- A **primary key (PK)** is a column (or set of columns) whose value uniquely identifies each row: `orders.order_id`.
- A **composite key** uses several columns together: an order line is identified by `(order_id, line_no)`.
- A **foreign key (FK)** is a column whose values refer to a primary key in another (or the same) table: `orders.customer_id` refers to `customers.customer_id`.
- A **natural key** comes from the business (an SKU code, an email); a **surrogate key** is a meaningless identifier created by the system (an auto-incrementing number). Chapter 9 discusses when to use which.

The ByteMart sample schema

This chapter uses four small tables. They are deliberately tiny so that you can check every result by eye.

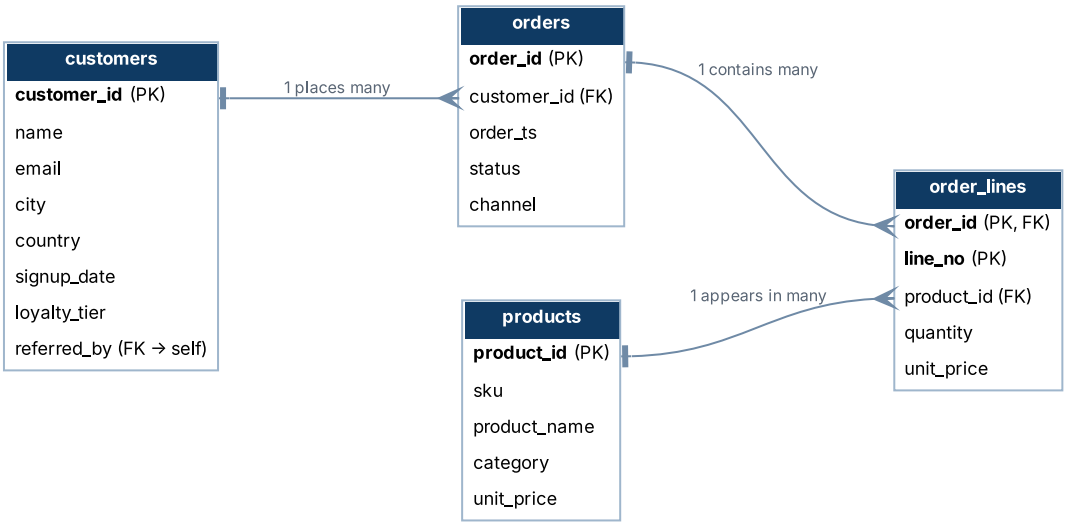


Figure 3.1 — The ByteMart sample schema. Crow's feet mark the "many" side of each relationship. A customer can be referred by another customer (a self-reference).

Creating the tables

The statements below are **DDL** (Data Definition Language): they define structure. Run them once.

```
CREATE TABLE customers (
    customer_id   INTEGER      PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(200) NOT NULL,
    city          VARCHAR(60),
    country       CHAR(2)      NOT NULL,
    signup_date   DATE         NOT NULL,
    loyalty_tier  VARCHAR(10),
    referred_by   INTEGER      REFERENCES customers (customer_id)
);

CREATE TABLE products (
    product_id    INTEGER       PRIMARY KEY,
    sku           VARCHAR(30)   NOT NULL UNIQUE,
    product_name  VARCHAR(100)  NOT NULL,
    category      VARCHAR(40)   NOT NULL,
    unit_price    DECIMAL(10,2) NOT NULL
);

CREATE TABLE orders (
    order_id      INTEGER     PRIMARY KEY,
    customer_id   INTEGER     NOT NULL REFERENCES customers (customer_id),
    order_ts      TIMESTAMP   NOT NULL,
    status        VARCHAR(12) NOT NULL,
    channel       VARCHAR(6)  NOT NULL
);

CREATE TABLE order_lines (
    order_id      INTEGER       NOT NULL REFERENCES orders (order_id),
    line_no       INTEGER       NOT NULL,
    product_id    INTEGER       NOT NULL REFERENCES products (product_id),
    quantity      INTEGER       NOT NULL CHECK (quantity > 0),
    unit_price    DECIMAL(10,2) NOT NULL,
```

```
    PRIMARY KEY (order_id, line_no)
);
```

Line by line:

- `CREATE TABLE customers ( … );` creates a table with the listed columns.
- `INTEGER`, `VARCHAR(100)`, `CHAR(2)`, `DATE`, `TIMESTAMP`, `DECIMAL(10,2)` are data types. `DECIMAL(10,2)` stores exact numbers with up to 10 digits, 2 after the decimal point — the right choice for money. Chapter 7 covers every Snowflake type.
- `PRIMARY KEY` declares the unique identifier. `NOT NULL` forbids missing values. `UNIQUE` forbids duplicates. `CHECK (quantity > 0)` forbids invalid values.
- `REFERENCES customers (customer_id)` declares a foreign key. `referred_by` references the *same* table — a self-reference.
- `PRIMARY KEY (order_id, line_no)` declares a composite key at table level.

SNOWFLAKE DIFFERENCE: WHICH CONSTRAINTS ARE ENFORCED

DuckDB (like PostgreSQL) **enforces** primary, unique and foreign keys: inserting a duplicate `order_id` fails. On Snowflake **standard tables**, `PRIMARY KEY`, `UNIQUE` and `FOREIGN KEY` are recorded as metadata but **not enforced**; `NOT NULL` and `CHECK` **are** enforced. On Snowflake **hybrid tables**, primary, unique and foreign keys are enforced. Chapter 7 explains why and what this means for pipeline design: in Snowflake, *your pipeline* must guarantee uniqueness.

Loading the sample data

DML (Data Manipulation Language) changes data. `INSERT INTO … VALUES` adds rows. Customers are inserted in three statements because DuckDB enforces the self-referencing foreign key: a customer's referrer must already exist when the row is inserted. (On a Snowflake standard table, which does not enforce foreign keys, one statement would succeed — and would equally succeed if the referrer did not exist at all.)

```
INSERT INTO customers VALUES
 (1, 'Aarav Sharma', 'aarav@example.com',    'Bengaluru',  'IN', DATE '2024-01-15', 'GOLD',
 NULL),
 (3, 'Oliver Smith', 'oliver@example.co.uk', 'London',     'UK', DATE '2024-05-20', NULL,
 NULL),
 (5, 'Ethan Brown',  'ethan@example.com',    'Austin',     'US', DATE '2025-02-14',
'BRONZE', NULL),
 (7, 'Noah Wilson',  'noah@example.com',     'Seattle',    'US', DATE '2025-09-05', NULL,
 NULL);

INSERT INTO customers VALUES
 (2, 'Priya Nair',   'priya@example.com',    'Kochi',      'IN', DATE '2024-03-02',
'SILVER', 1),
 (4, 'Amelia Jones', 'amelia@example.co.uk', 'Manchester', 'UK', DATE '2025-01-11', 'GOLD',
 3),
 (6, 'Diya Patel',   'diya@example.com',     NULL,         'IN', DATE '2025-06-30',
'BRONZE', 1);

INSERT INTO customers VALUES
 (8, 'Kavya Reddy',  'kavya@example.com',    'Hyderabad',  'IN', DATE '2026-01-20',
'SILVER', 2);

INSERT INTO products VALUES
 (101, 'CHG-USBC-65W', 'USB-C Charger 65W',           'Electronics',   899.00),
```

```
 (102, 'HP-NC-700',    'Noise-Cancelling Headphones', 'Electronics', 12999.00),
 (103, 'KB-MECH-87',   'Mechanical Keyboard',         'Electronics',  4499.00),
 (104, 'MUG-STEEL',    'Insulated Steel Mug',         'Home',          749.00),
 (105, 'LAMP-LED',     'LED Desk Lamp',               'Home',         1899.00),
 (106, 'CHAIR-ERG',    'Ergonomic Chair',             'Furniture',   15999.00),
 (107, 'CABLE-HDMI',   'HDMI Cable 2m',               'Electronics',   399.00);

INSERT INTO orders VALUES
 (1001, 1, TIMESTAMP '2026-01-05 10:15:00', 'DELIVERED', 'web'),
 (1002, 2, TIMESTAMP '2026-01-07 19:42:00', 'DELIVERED', 'app'),
 (1003, 1, TIMESTAMP '2026-01-20 08:05:00', 'DELIVERED', 'app'),
 (1004, 3, TIMESTAMP '2026-01-28 21:10:00', 'CANCELLED', 'web'),
 (1005, 4, TIMESTAMP '2026-02-03 12:00:00', 'DELIVERED', 'web'),
 (1006, 5, TIMESTAMP '2026-02-11 16:30:00', 'SHIPPED',   'web'),
 (1007, 2, TIMESTAMP '2026-02-14 09:20:00', 'DELIVERED', 'app'),
 (1008, 6, TIMESTAMP '2026-02-20 23:55:00', 'DELIVERED', 'app'),
 (1009, 1, TIMESTAMP '2026-03-01 13:45:00', 'SHIPPED',   'web'),
 (1010, 7, TIMESTAMP '2026-03-03 07:10:00', 'PENDING',   'app'),
 (1011, 4, TIMESTAMP '2026-03-08 18:25:00', 'SHIPPED',   'app'),
 (1012, 3, TIMESTAMP '2026-03-09 11:00:00', 'DELIVERED', 'web');

INSERT INTO order_lines VALUES
 (1001, 1, 101, 2,   899.00), (1001, 2, 104, 1,   749.00),
 (1002, 1, 102, 1, 12999.00),
 (1003, 1, 103, 1,  4499.00), (1003, 2, 101, 1,   899.00),
 (1004, 1, 106, 1, 15999.00),
 (1005, 1, 105, 2,  1899.00), (1005, 2, 104, 2,   749.00),
 (1006, 1, 102, 1, 12999.00), (1006, 2, 101, 1,   899.00),
 (1007, 1, 104, 4,   749.00),
 (1008, 1, 106, 1, 15999.00),
 (1009, 1, 105, 1,  1899.00),
 (1010, 1, 103, 1,  4499.00),
 (1011, 1, 102, 1, 12499.00),
 (1012, 1, 101, 3,   899.00), (1012, 2, 105, 1,  1899.00);
```

Notice that `order_lines.unit_price` repeats the product price. That is deliberate and realistic: the line records the price *actually charged at the time of sale*. Order 1011 was sold at a discount (12,499 instead of 12,999), and if the catalogue price changes next month, historical orders must not change. Storing a value at the time of the event is a key analytics design idea we return to in Chapter 9.

3.2 SELECT and FROM

Concept

`SELECT` is how you read data. The two essential clauses are `SELECT` (which columns or expressions to return) and `FROM` (which table to read). The result of every query is itself a table — a *result set* — which is why queries can be nested and combined.

Selecting everything

```
SELECT * FROM products;
```

Result (executed on DuckDB):

product_id	sku	product_name	category	unit_price
101	CHG-USBC-65W	USB-C Charger 65W	Electronics	899.00
102	HP-NC-700	Noise-Cancelling Headphones	Electronics	12999.00
103	KB-MECH-87	Mechanical Keyboard	Electronics	4499.00
104	MUG-STEEL	Insulated Steel Mug	Home	749.00
105	LAMP-LED	LED Desk Lamp	Home	1899.00
106	CHAIR-ERG	Ergonomic Chair	Furniture	15999.00
107	CABLE-HDMI	HDMI Cable 2m	Electronics	399.00

7 rows.

`*` means "all columns". It is convenient for exploration but should be avoided in production code: it returns columns you do not need (wasting work — especially in a columnar engine like Snowflake, where reading fewer columns means scanning fewer bytes), and it silently changes when someone adds a column.

Selecting columns, expressions and aliases

```
SELECT product_name,
       category,
       unit_price,
       unit_price * 1.18          AS price_incl_gst,
       ROUND(unit_price / 83.0, 2) AS approx_price_usd
FROM products;
```

Result (executed on DuckDB):

product_name	category	unit_price	price_incl_gst	approx_price_usd
USB-C Charger 65W	Electronics	899.00	1060.8200	10.83
Noise-Cancelling Headphones	Electronics	12999.00	15338.8200	156.61
Mechanical Keyboard	Electronics	4499.00	5308.8200	54.2
Insulated Steel Mug	Home	749.00	883.8200	9.02
LED Desk Lamp	Home	1899.00	2240.8200	22.88
Ergonomic Chair	Furniture	15999.00	18878.8200	192.76
HDMI Cable 2m	Electronics	399.00	470.8200	4.81

7 rows.

Line by line: `unit_price * 1.18` is an *expression* computed for each row; `AS price_incl_gst` gives the computed column a name (an **alias**). `ROUND(x, 2)` rounds to two decimal places. The conversion rate 83 is illustrative only.

DISTINCT and LIMIT

```
SELECT DISTINCT category FROM products ORDER BY category;
```

Result (executed on DuckDB):

category
Electronics
Furniture
Home

3 rows.

`DISTINCT` removes duplicate rows from the result. `ORDER BY` sorts (Section 3.4).

```
SELECT order_id, order_ts FROM orders ORDER BY order_ts DESC LIMIT 3;
```

Result (executed on DuckDB):

order_id	order_ts
1012	2026-03-09 11:00:00
1011	2026-03-08 18:25:00
1010	2026-03-03 07:10:00

3 rows.

`LIMIT 3` returns at most three rows. Without `ORDER BY`, *which* three is undefined — never rely on the order of rows unless you sort explicitly. (Snowflake also supports the standard `FETCH FIRST 3 ROWS ONLY` form.)

3.3 Filtering with WHERE

Concept

`WHERE` keeps only the rows for which a **predicate** — a true/false condition — evaluates to TRUE.

```
SELECT order_id, customer_id, status
FROM orders
WHERE status = 'DELIVERED' AND channel = 'app';
```

Result (executed on DuckDB):

order_id	customer_id	status
1002	2	DELIVERED
1003	1	DELIVERED
1007	2	DELIVERED
1008	6	DELIVERED

4 rows.

Comparison and logical operators

Operator	Meaning	Example
`=`, `<>` (or `!=`)	equal, not equal	`status <> 'CANCELLED'`
`<`, `<=`, `>`, `>=`	ordering	`unit_price >= 1000`
`AND`, `OR`, `NOT`	combine conditions	`a AND (b OR c)`
`IN (…)`	matches any value in a list	`country IN ('IN','UK')`
`BETWEEN x AND y`	inclusive range	`unit_price BETWEEN 500 AND 2000`
`LIKE`	pattern match, case-sensitive; `%` any characters, `_` one character	`sku LIKE 'CHG-%'`
`ILIKE`	case-insensitive `LIKE` (Snowflake and DuckDB)	`email ILIKE '%.CO.UK'`

Operator	Meaning	Example
`IS NULL`, `IS NOT NULL`	test for missing values	`city IS NULL`

```sql
SELECT product_name, unit_price
FROM products
WHERE category IN ('Home', 'Furniture')
   OR (category = 'Electronics' AND unit_price < 1000);
```

Result (executed on DuckDB):

product_name	unit_price
USB-C Charger 65W	899.00
Insulated Steel Mug	749.00
LED Desk Lamp	1899.00
Ergonomic Chair	15999.00
HDMI Cable 2m	399.00

5 rows.

OPERATOR PRECEDENCE

`AND` binds more tightly than `OR`. `WHERE a OR b AND c` means `a OR (b AND c)`. Always use parentheses when mixing them — the query above would return different rows without them.

```sql
SELECT name, email FROM customers WHERE email ILIKE '%.co.uk';
```

Result (executed on DuckDB):

name	email
Oliver Smith	oliver@example.co.uk
Amelia Jones	amelia@example.co.uk

2 rows.

Filtering dates and timestamps

```sql
SELECT order_id, order_ts
FROM orders
WHERE order_ts >= TIMESTAMP '2026-02-01'
  AND order_ts <  TIMESTAMP '2026-03-01';
```

Result (executed on DuckDB):

order_id	order_ts
1005	2026-02-03 12:00:00
1006	2026-02-11 16:30:00
1007	2026-02-14 09:20:00
1008	2026-02-20 23:55:00

4 rows.

The **half-open interval** pattern — `>=` start and `<` next start — is the safest way to filter a period. `BETWEEN '2026-02-01' AND '2026-02-28'` would miss orders placed during 28 February after midnight, because the timestamp `2026-02-28 14:00` is greater than `2026-02-28 00:00`.

3.4 Sorting with ORDER BY

```
SELECT name, country, loyalty_tier
FROM customers
ORDER BY country ASC, loyalty_tier DESC NULLS LAST, name;
```

Result (executed on DuckDB):

name	country	loyalty_tier
Kavya Reddy	IN	SILVER
Priya Nair	IN	SILVER
Aarav Sharma	IN	GOLD
Diya Patel	IN	BRONZE
Amelia Jones	UK	GOLD
Oliver Smith	UK	NULL
Ethan Brown	US	BRONZE
Noah Wilson	US	NULL

8 rows.

- Sort keys are applied left to right: first by country, then within a country by tier, then by name.
- `ASC` (ascending) is the default; `DESC` is descending.
- `NULLS LAST` / `NULLS FIRST` controls where missing values go. Defaults differ between databases; in Snowflake, NULLs sort as the *largest* values by default (last in ascending order, first in descending), controlled by the `DEFAULT_NULL_ORDERING` parameter. Be explicit when it matters.

3.5 Aggregation, GROUP BY and HAVING

Concept

Aggregate functions collapse many rows into one value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. With `GROUP BY`, they produce one value *per group*.

```
SELECT COUNT(*)                 AS order_count,
       COUNT(DISTINCT customer_id) AS customers_who_ordered,
       MIN(order_ts)            AS first_order,
       MAX(order_ts)            AS last_order
FROM orders;
```

Result (executed on DuckDB):

order_count	customers_who_ordered	first_order	last_order
12	7	2026-01-05 10:15:00	2026-03-09 11:00:00

1 row.

Revenue per order line

Revenue is not stored; it is `quantity × unit_price`. Computing it at query time keeps one source of truth.

```
SELECT order_id,
       SUM(quantity * unit_price) AS order_revenue,
       SUM(quantity)              AS items
FROM order_lines
GROUP BY order_id
ORDER BY order_id;
```

Result (executed on DuckDB):

order_id	order_revenue	items
1001	2547.00	3
1002	12999.00	1
1003	5398.00	2
1004	15999.00	1
1005	5296.00	4
1006	13898.00	2
1007	2996.00	4
1008	15999.00	1
1009	1899.00	1
1010	4499.00	1
1011	12499.00	1
1012	4596.00	4

12 rows.

The GROUP BY rule: every column in `SELECT` must either be in `GROUP BY` or be inside an aggregate function. Selecting `product_id` here would be an error, because each order has several products — which one should the single result row show?

```
SELECT order_id, product_id, SUM(quantity * unit_price) AS order_revenue
FROM order_lines
GROUP BY order_id;
```

Result (executed on DuckDB): error — `Binder Error: column "product_id" must appear in the GROUP BY clause or must be part of an aggregate function.`

Snowflake raises the same class of error ("... is not a valid group by expression"). Snowflake also supports `GROUP BY ALL`, which groups by every non-aggregated column in the `SELECT` list — handy, but be sure you understand what it groups by.

Filtering groups with HAVING

`WHERE` filters rows *before* grouping; `HAVING` filters groups *after* aggregation.

```
SELECT o.customer_id,
       COUNT(*)  AS orders_placed
FROM orders o
WHERE o.status <> 'CANCELLED'
GROUP BY o.customer_id
```

```
HAVING COUNT(*) >= 2
ORDER BY orders_placed DESC, o.customer_id;
```

Result (executed on DuckDB):

customer_id	orders_placed
1	3
2	2
4	2

3 rows.

Here `o` is a **table alias**, a short name for `orders` used to qualify columns (`o.customer_id`). Aliases become essential once queries involve several tables.

3.6 NULL: the value that is not a value

Concept

`NULL` means *unknown or missing*. It is not zero, not an empty string, and not equal to anything — **not even to another NULL**. SQL therefore uses **three-valued logic**: a predicate can be TRUE, FALSE or UNKNOWN. `WHERE` keeps only rows where the predicate is TRUE.

```
SELECT NULL = NULL          AS null_equals_null,
       NULL <> 1            AS null_not_equal_one,
       NULL IS NULL         AS null_is_null,
       1 + NULL             AS arithmetic_with_null,
       'a' || NULL          AS concat_with_null;
```

Result (executed on DuckDB):

null_equals_null	null_not_equal_one	null_is_null	arithmetic_with_null	concat_with_null
NULL	NULL	TRUE	NULL	NULL

1 row.

Every comparison with NULL yields NULL (UNKNOWN), and arithmetic or concatenation with NULL yields NULL.

> **SNOWFLAKE DIFFERENCE: STRING CONCATENATION**
> The `||` operator returns NULL if either side is NULL, in Snowflake and DuckDB alike. In Snowflake, the functions `CONCAT` and `CONCAT_WS` also return NULL when any argument is NULL — whereas DuckDB and PostgreSQL skip NULL arguments in those functions. To treat NULL as an empty string portably, wrap arguments in `COALESCE(x, '')`.

How NULL affects filters

```
SELECT name, loyalty_tier FROM customers WHERE loyalty_tier <> 'GOLD';
```

Result (executed on DuckDB):

name	loyalty_tier
Ethan Brown	BRONZE
Priya Nair	SILVER
Diya Patel	BRONZE
Kavya Reddy	SILVER

4 rows.

Oliver and Noah have no tier (NULL). They are *not* returned, because `NULL <> 'GOLD'` is UNKNOWN. If you meant "everyone who is not known to be gold", write:

```
SELECT name, loyalty_tier
FROM customers
WHERE loyalty_tier <> 'GOLD' OR loyalty_tier IS NULL;
```

Result (executed on DuckDB):

name	loyalty_tier
Oliver Smith	NULL
Ethan Brown	BRONZE
Noah Wilson	NULL
Priya Nair	SILVER
Diya Patel	BRONZE
Kavya Reddy	SILVER

6 rows.

How NULL affects aggregates

Aggregate functions **ignore** NULLs, except `COUNT(*)`, which counts rows.

```
SELECT COUNT(*)             AS all_customers,
       COUNT(city)          AS customers_with_city,
       COUNT(loyalty_tier)  AS customers_with_tier
FROM customers;
```

Result (executed on DuckDB):

all_customers	customers_with_city	customers_with_tier
8	7	6

1 row.

COALESCE and NULLIF

- `COALESCE(a, b, c)` returns the first non-NULL argument — used to supply defaults.
- `NULLIF(a, b)` returns NULL if `a = b`, otherwise `a` — commonly used to avoid division by zero.

```
SELECT name,
       COALESCE(city, 'Unknown')         AS city,
       COALESCE(loyalty_tier, 'NONE')    AS tier,
       100 / NULLIF(customer_id - 1, 0)  AS demo_safe_division
FROM customers
```

```
ORDER BY customer_id
LIMIT 3;
```

Result (executed on DuckDB):

name	city	tier	demo_safe_division
Aarav Sharma	Bengaluru	GOLD	NULL
Priya Nair	Kochi	SILVER	100
Oliver Smith	London	NONE	50

3 rows.

For Aarav (`customer_id = 1`), `customer_id - 1` is 0; `NULLIF` turns it into NULL, and dividing by NULL gives NULL instead of a division-by-zero error. (Snowflake also offers `DIV0` and `DIV0NULL`, which return 0 when the divisor is 0 or NULL respectively.)

The NOT IN trap

This is one of the most damaging NULL mistakes in production SQL. Suppose a list of "products to exclude" accidentally contains a NULL:

```
CREATE TABLE excluded_products (product_id INTEGER);
INSERT INTO excluded_products VALUES (106), (NULL);
```

```
SELECT product_id, product_name
FROM products
WHERE product_id NOT IN (SELECT product_id FROM excluded_products);
```

Result (executed on DuckDB):

product_id	product_name

0 rows.

No rows at all. `x NOT IN (106, NULL)` means `x <> 106 AND x <> NULL`; the second part is UNKNOWN for every row, so the whole predicate is never TRUE. Use `NOT EXISTS` (Section 3.8), which handles NULLs as you intend:

```
SELECT p.product_id, p.product_name
FROM products p
WHERE NOT EXISTS (SELECT 1 FROM excluded_products e WHERE e.product_id = p.product_id)
ORDER BY p.product_id;
```

Result (executed on DuckDB):

product_id	product_name
101	USB-C Charger 65W
102	Noise-Cancelling Headphones
103	Mechanical Keyboard
104	Insulated Steel Mug
105	LED Desk Lamp
107	HDMI Cable 2m

6 rows.

3.7 Joins

Concept

A **join** combines rows from two tables based on a condition, usually matching a foreign key to a primary key. The *type* of join determines what happens to rows that have no match.

Join	Keeps	Unmatched rows from the left	Unmatched rows from the right
`INNER JOIN`	matching pairs only	dropped	dropped
`LEFT [OUTER] JOIN`	all left rows	kept, right columns NULL	dropped
`RIGHT [OUTER] JOIN`	all right rows	dropped	kept, left columns NULL
`FULL [OUTER] JOIN`	all rows from both	kept	kept
`CROSS JOIN`	every combination	—	—

INNER JOIN

```
SELECT o.order_id, c.name, c.country, o.status
FROM orders o
INNER JOIN customers c
        ON c.customer_id = o.customer_id
WHERE o.order_ts < TIMESTAMP '2026-02-01'
ORDER BY o.order_id;
```

Result (executed on DuckDB):

order_id	name	country	status
1001	Aarav Sharma	IN	DELIVERED
1002	Priya Nair	IN	DELIVERED
1003	Aarav Sharma	IN	DELIVERED
1004	Oliver Smith	UK	CANCELLED

4 rows.

LEFT JOIN: keep everything on the left

Which customers have *never* ordered? Kavya (customer 8) has no orders, so an inner join would hide her. A left join keeps her with NULLs:

```
SELECT c.customer_id, c.name, COUNT(o.order_id) AS orders_placed
FROM customers c
LEFT JOIN orders o
       ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY c.customer_id;
```

Result (executed on DuckDB):

customer_id	name	orders_placed
1	Aarav Sharma	3

customer_id	name	orders_placed
2	Priya Nair	2
3	Oliver Smith	2
4	Amelia Jones	2
5	Ethan Brown	1
6	Diya Patel	1
7	Noah Wilson	1
8	Kavya Reddy	0

8 rows.

Note `COUNT(o.order_id)`, not `COUNT(*)`: for Kavya the joined row exists but `o.order_id` is NULL, so `COUNT(o.order_id)` correctly gives 0 while `COUNT(*)` would give 1.

Anti-join: rows with no match

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_lines l ON l.product_id = p.product_id
WHERE l.product_id IS NULL;
```

Result (executed on DuckDB):

product_id	product_name
107	HDMI Cable 2m

1 row.

This "LEFT JOIN … WHERE right side IS NULL" pattern is an **anti-join**. `NOT EXISTS` expresses the same thing and is often clearer.

A filter in ON versus WHERE

With outer joins, *where* you put a condition changes the result.

```
SELECT c.name, o.order_id, o.status
FROM customers c
LEFT JOIN orders o
       ON o.customer_id = c.customer_id
      AND o.status = 'SHIPPED'
WHERE c.country = 'UK'
ORDER BY c.name;
```

Result (executed on DuckDB):

name	order_id	status
Amelia Jones	1011	SHIPPED
Oliver Smith	NULL	NULL

2 rows.

The status condition is in `ON`, so UK customers *without* shipped orders are still listed (with NULLs). Moving `o.status = 'SHIPPED'` to `WHERE` would remove Oliver's row entirely, silently turning the left join into an inner join — a very common bug.

Joining three or more tables

```
SELECT o.order_id,
       c.name                       AS customer,
       p.product_name,
       l.quantity,
       l.quantity * l.unit_price    AS line_revenue
FROM orders o
JOIN customers   c ON c.customer_id = o.customer_id
JOIN order_lines l ON l.order_id    = o.order_id
JOIN products    p ON p.product_id  = l.product_id
WHERE o.order_id IN (1001, 1005)
ORDER BY o.order_id, l.line_no;
```

Result (executed on DuckDB):

order_id	customer	product_name	quantity	line_revenue
1001	Aarav Sharma	USB-C Charger 65W	2	1798.00
1001	Aarav Sharma	Insulated Steel Mug	1	749.00
1005	Amelia Jones	LED Desk Lamp	2	3798.00
1005	Amelia Jones	Insulated Steel Mug	2	1498.00

4 rows.

`JOIN` on its own means `INNER JOIN`.

Self-join

A table joined to itself, using two aliases. Who referred whom?

```
SELECT c.name  AS customer,
       r.name  AS referred_by
FROM customers c
JOIN customers r ON r.customer_id = c.referred_by
ORDER BY c.customer_id;
```

Result (executed on DuckDB):

customer	referred_by
Priya Nair	Aarav Sharma
Amelia Jones	Oliver Smith
Diya Patel	Aarav Sharma
Kavya Reddy	Priya Nair

4 rows.

FULL OUTER JOIN and CROSS JOIN

```
SELECT ch.channel, st.status
FROM (SELECT DISTINCT channel FROM orders) ch
CROSS JOIN (SELECT DISTINCT status FROM orders) st
ORDER BY ch.channel, st.status;
```

Result (executed on DuckDB):

channel	status
app	CANCELLED
app	DELIVERED
app	PENDING
app	SHIPPED
web	CANCELLED
web	DELIVERED
web	PENDING
web	SHIPPED

8 rows.

A **cross join** pairs every row with every row (2 channels × 4 statuses = 8 rows). It is useful for building complete grids — for example, every product × every day, so that days with zero sales still appear — but an accidental cross join on large tables can produce billions of rows.

The fan-out trap

When you join a "one" table to a "many" table, rows on the "one" side are **repeated**. Aggregating a "one"-side value after such a join double-counts it. Suppose orders carried a delivery fee of 50 per order:

```
SELECT COUNT(*)      AS joined_rows,
       SUM(50)       AS wrong_total_delivery_fees,
       COUNT(DISTINCT o.order_id) * 50 AS correct_total_delivery_fees
FROM orders o
JOIN order_lines l ON l.order_id = o.order_id;
```

Result (executed on DuckDB):

joined_rows	wrong_total_delivery_fees	correct_total_delivery_fees
17	850	600

1 row.

Seventeen order lines, twelve orders: summing a per-order amount after joining to lines overstates fees by 250. The fix is to aggregate each table to the same **grain** (level of detail) *before* joining, a principle formalised in Chapter 9.

3.8 Subqueries and common table expressions (CTEs)

Subqueries

A **subquery** is a query inside another query.

Scalar subquery — returns one value:

```
SELECT product_name, unit_price
FROM products
WHERE unit_price > (SELECT AVG(unit_price) FROM products)
ORDER BY unit_price DESC;
```

Result (executed on DuckDB):

product_name	unit_price
Ergonomic Chair	15999.00
Noise-Cancelling Headphones	12999.00

2 rows.

IN subquery — returns a list:

```
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE channel = 'web')
ORDER BY name;
```

Result (executed on DuckDB):

name
Aarav Sharma
Amelia Jones
Ethan Brown
Oliver Smith

4 rows.

EXISTS / correlated subquery — the inner query refers to the outer row (`c.customer_id`), and is logically evaluated for each outer row:

```
SELECT c.name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    JOIN order_lines l ON l.order_id = o.order_id
    JOIN products p    ON p.product_id = l.product_id
    WHERE o.customer_id = c.customer_id
      AND p.category = 'Furniture'
)
ORDER BY c.name;
```

Result (executed on DuckDB):

name
Diya Patel
Oliver Smith

2 rows.

Common table expressions

A **CTE** names a subquery with `WITH name AS ( … )` so that the main query can use it like a table. CTEs make complex queries readable as a sequence of steps.

```
WITH order_totals AS (
    SELECT order_id, SUM(quantity * unit_price) AS order_revenue
    FROM order_lines
    GROUP BY order_id
),
valid_orders AS (
    SELECT o.order_id, o.customer_id, t.order_revenue
```

```
    FROM orders o
    JOIN order_totals t ON t.order_id = o.order_id
    WHERE o.status <> 'CANCELLED'
)
SELECT c.country,
       COUNT(*)                    AS orders,
       SUM(v.order_revenue)        AS revenue,
       ROUND(AVG(v.order_revenue), 2) AS avg_order_value
FROM valid_orders v
JOIN customers c ON c.customer_id = v.customer_id
GROUP BY c.country
ORDER BY revenue DESC;
```

Result (executed on DuckDB):

country	orders	revenue	avg_order_value
IN	6	41838.00	6973
UK	3	22391.00	7463.67
US	2	18397.00	9198.5

3 rows.

Reading it as steps: (1) total each order; (2) keep non-cancelled orders with their totals; (3) aggregate by country. This structure — *aggregate to the right grain, filter, then join and summarise* — is the backbone of analytical SQL. Recursive CTEs, which reference themselves to walk hierarchies, are covered in Chapter 8.

3.9 CASE expressions

`CASE` returns different values depending on conditions — SQL's if/else.

```
SELECT product_name,
       unit_price,
       CASE
           WHEN unit_price >= 10000 THEN 'Premium'
           WHEN unit_price >=  1500 THEN 'Mid-range'
           ELSE 'Budget'
       END AS price_band
FROM products
ORDER BY unit_price DESC;
```

Result (executed on DuckDB):

product_name	unit_price	price_band
Ergonomic Chair	15999.00	Premium
Noise-Cancelling Headphones	12999.00	Premium
Mechanical Keyboard	4499.00	Mid-range
LED Desk Lamp	1899.00	Mid-range
USB-C Charger 65W	899.00	Budget
Insulated Steel Mug	749.00	Budget
HDMI Cable 2m	399.00	Budget

7 rows.

Conditions are tested top to bottom; the first TRUE wins. Without `ELSE`, unmatched rows get NULL.

Conditional aggregation

`CASE` inside an aggregate turns rows into columns — a manual pivot that works in every SQL dialect.

```
SELECT DATE_TRUNC('month', order_ts)::DATE                    AS month,
       COUNT(*)                                               AS all_orders,
       SUM(CASE WHEN channel = 'app' THEN 1 ELSE 0 END)       AS app_orders,
       SUM(CASE WHEN channel = 'web' THEN 1 ELSE 0 END)       AS web_orders,
       SUM(CASE WHEN status = 'CANCELLED' THEN 1 ELSE 0 END)  AS cancelled
FROM orders
GROUP BY 1
ORDER BY 1;
```

Result (executed on DuckDB):

month	all_orders	app_orders	web_orders	cancelled
2026-01-01	4	2	2	1
2026-02-01	4	2	2	0
2026-03-01	4	2	2	0

3 rows.

`GROUP BY 1` and `ORDER BY 1` refer to the first column of the SELECT list by position — convenient in exploration, but naming the expression is clearer in production code. Snowflake also provides `IFF(condition, a, b)` as a compact two-branch CASE, and `COUNT_IF(condition)`.

3.10 Date and string functions

Dates and timestamps

```
SELECT order_id,
       order_ts,
       order_ts::DATE                                AS order_date,
       EXTRACT(YEAR  FROM order_ts)                  AS yr,
       EXTRACT(MONTH FROM order_ts)                  AS mon,
       EXTRACT(HOUR  FROM order_ts)                  AS hr,
       DATE_TRUNC('month', order_ts)::DATE           AS month_start,
       LAST_DAY(order_ts::DATE)                      AS month_end,
       order_ts::DATE + INTERVAL '30 days'           AS return_window_ends
FROM orders
WHERE order_id IN (1001, 1008);
```

Result (executed on DuckDB):

order_id	order_ts	order_date	yr	mon	hr	month_start	month_end	return_window_ends
1001	2026-01-05 10:15:00	2026-01-05	2026	1	10	2026-01-01	2026-01-31	2026-02-04 00:00:00
1008	2026-02-20 23:55:00	2026-02-20	2026	2	23	2026-02-01	2026-02-28	2026-03-22 00:00:00

2 rows.

Function	Purpose	Snowflake notes
`x::DATE`, `CAST(x AS DATE)`	Convert type	Snowflake also has `TO_DATE`, `TRY_TO_DATE` (returns NULL instead of error)
`EXTRACT(part FROM x)`	Get year, month, day, hour...	Also `YEAR(x)`, `MONTH(x)`, `DAYOFWEEK(x)`...
`DATE_TRUNC('month', x)`	Round down to start of period	Returns the input's type
`LAST_DAY(d)`	Last day of month	Optional second argument for other periods
`d + INTERVAL '30 days'`	Date arithmetic	Snowflake also has `DATEADD(day, 30, d)`
`DATEDIFF(part, a, b)`	Difference in units	Snowflake: `DATEDIFF(day, a, b)` counts boundaries crossed

Days between dates

Subtracting one `DATE` from another returns the number of days in both engines:

```sql
SELECT c.name,
       c.signup_date,
       MIN(o.order_ts)::DATE                  AS first_order_date,
       MIN(o.order_ts)::DATE - c.signup_date  AS days_to_first_order
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.name, c.signup_date
ORDER BY days_to_first_order;
```

Result (executed on DuckDB):

name	signup_date	first_order_date	days_to_first_order
Noah Wilson	2025-09-05	2026-03-03	179
Diya Patel	2025-06-30	2026-02-20	235
Ethan Brown	2025-02-14	2026-02-11	362
Amelia Jones	2025-01-11	2026-02-03	388
Oliver Smith	2024-05-20	2026-01-28	618
Priya Nair	2024-03-02	2026-01-07	676
Aarav Sharma	2024-01-15	2026-01-05	721

7 rows.

TIME ZONES

`TIMESTAMP` in this chapter has no time zone. ByteMart operates in IST, UK time and several US zones, so "orders on 1 March" depends on whose midnight you use. Snowflake has three timestamp types — `TIMESTAMP_NTZ`, `TIMESTAMP_LTZ` and `TIMESTAMP_TZ` — and Chapter 7 explains how to choose between them. The rule of thumb: store in UTC or with an explicit offset; convert to local time only for presentation.

Strings

```sql
SELECT email,
       UPPER(name)                            AS name_upper,
       LENGTH(name)                           AS name_len,
       SPLIT_PART(email, '@', 1)              AS mailbox,
```

```
       SPLIT_PART(email, '@', 2)                AS domain,
       SUBSTR(sku, 1, 3)                        AS sku_prefix,
       REPLACE(sku, '-', '_')                   AS sku_underscored,
       LPAD(CAST(product_id AS VARCHAR), 6, '0') AS padded_id,
       country || '-' || CAST(customer_id AS VARCHAR) AS customer_code
FROM customers
JOIN products ON products.product_id = customers.customer_id + 100
WHERE customer_id <= 3;
```

Result (executed on DuckDB):

email	name_upper	name_len	mailbox	domain	sku_prefix	sku_underscored	padded_id	customer_code
aarav@example.com	AARAV SHARMA	12	aarav	example.com	CHG	CHG_USBC_65W	000101	IN-1
priya@example.com	PRIYA NAIR	10	priya	example.com	HP-	HP_NC_700	000102	IN-2
oliver@example.co.uk	OLIVER SMITH	12	oliver	example.co.uk	KB-	KB_MECH_87	000103	UK-3

3 rows.

That join condition is purely a trick to place three customers next to three products for demonstration; it has no business meaning. Other everyday string functions: `LOWER`, `TRIM`/`LTRIM`/`RTRIM`, `POSITION(sub IN str)`, `LEFT`, `RIGHT`, `REGEXP_SUBSTR`, `REGEXP_REPLACE`. Snowflake string comparisons are **case-sensitive** by default unless a collation is specified (Chapter 7).

3.11 Window functions

Concept

An aggregate with `GROUP BY` collapses rows. A **window function** computes a value *across a set of related rows* but **keeps every row**. The set of rows is the **window**, defined with `OVER (…)`:

- `PARTITION BY` — split rows into groups (like GROUP BY, but without collapsing);
- `ORDER BY` — order rows within each partition;
- a **frame** such as `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` — limit the window to nearby rows.

Ranking

```
CREATE VIEW order_revenue AS
SELECT o.order_id, o.customer_id, o.order_ts, o.status, o.channel,
       SUM(l.quantity * l.unit_price) AS revenue
FROM orders o
JOIN order_lines l ON l.order_id = o.order_id
GROUP BY o.order_id, o.customer_id, o.order_ts, o.status, o.channel;
```

We created a **view** — a saved query that behaves like a table — called `order_revenue`, with one row per order and its revenue. Views are covered properly in Chapter 6.

```
SELECT order_id, customer_id, revenue,
       ROW_NUMBER() OVER (ORDER BY revenue DESC) AS row_num,
       RANK()       OVER (ORDER BY revenue DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY revenue DESC) AS dense_rnk
```

```
FROM order_revenue
ORDER BY revenue DESC, order_id;
```

Result (executed on DuckDB):

order_id	customer_id	revenue	row_num	rnk	dense_rnk
1004	3	15999.00	1	1	1
1008	6	15999.00	2	1	1
1006	5	13898.00	3	3	2
1002	2	12999.00	4	4	3
1011	4	12499.00	5	5	4
1003	1	5398.00	6	6	5
1005	4	5296.00	7	7	6
1012	3	4596.00	8	8	7
1010	7	4499.00	9	9	8
1007	2	2996.00	10	10	9
1001	1	2547.00	11	11	10
1009	1	1899.00	12	12	11

12 rows.

Orders 1004 and 1008 both have revenue 15,999 (a tie). `ROW_NUMBER` breaks the tie arbitrarily (1, 2); `RANK` gives both 1 and then skips to 3; `DENSE_RANK` gives both 1 and continues with 2.

> **NON-DETERMINISTIC ROW_NUMBER**
>
> With ties in the `ORDER BY`, `ROW_NUMBER()` may assign numbers differently between runs. Add a unique tie-breaker — `ORDER BY revenue DESC, order_id` — whenever the choice matters.

Running totals and comparisons with previous rows

```
SELECT customer_id,
       order_id,
       order_ts::DATE AS order_date,
       revenue,
       SUM(revenue) OVER (PARTITION BY customer_id ORDER BY order_ts)  AS running_total,
       LAG(order_ts::DATE) OVER (PARTITION BY customer_id ORDER BY order_ts) AS previous_orde
r_date,
       order_ts::DATE - LAG(order_ts::DATE) OVER (PARTITION BY customer_id ORDER BY order_ts)
                                                                          AS days_since_previou
s
FROM order_revenue
WHERE customer_id IN (1, 2)
ORDER BY customer_id, order_ts;
```

Result (executed on DuckDB):

customer_id	order_id	order_date	revenue	running_total	previous_order_date	days_since_previous
1	1001	2026-01-05	2547.00	2547.00	NULL	NULL
1	1003	2026-01-20	5398.00	7945.00	2026-01-05	15
1	1009	2026-03-01	1899.00	9844.00	2026-01-20	40
2	1002	2026-01-07	12999.00	12999.00	NULL	NULL
2	1007	2026-02-14	2996.00	15995.00	2026-01-07	38

5 rows.

- `SUM(…) OVER (PARTITION BY customer_id ORDER BY order_ts)` restarts for each customer and accumulates in time order.
- `LAG(x)` returns `x` from the previous row in the window; `LEAD(x)` from the next. The first order of each customer has no previous row, so `LAG` returns NULL.

Moving averages with a frame

```
SELECT order_id,
       order_ts::DATE AS order_date,
       revenue,
       ROUND(AVG(revenue) OVER (ORDER BY order_ts
                                ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moving_avg_3
_orders
FROM order_revenue
ORDER BY order_ts;
```

Result (executed on DuckDB):

order_id	order_date	revenue	moving_avg_3_orders
1001	2026-01-05	2547.00	2547
1002	2026-01-07	12999.00	7773
1003	2026-01-20	5398.00	6981.33
1004	2026-01-28	15999.00	11465.33
1005	2026-02-03	5296.00	8897.67
1006	2026-02-11	13898.00	11731
1007	2026-02-14	2996.00	7396.67
1008	2026-02-20	15999.00	10964.33
1009	2026-03-01	1899.00	6964.67
1010	2026-03-03	4499.00	7465.67
1011	2026-03-08	12499.00	6299
1012	2026-03-09	4596.00	7198

12 rows.

Percent of total

```
SELECT channel,
       SUM(revenue)                                                AS revenue,
       ROUND(100.0 * SUM(revenue) / SUM(SUM(revenue)) OVER (), 1)  AS pct_of_total
FROM order_revenue
WHERE status <> 'CANCELLED'
GROUP BY channel
ORDER BY revenue DESC;
```

Result (executed on DuckDB):

channel	revenue	pct_of_total
app	54390.00	65.8
web	28236.00	34.2

2 rows.

`SUM(SUM(revenue)) OVER ()` looks odd but is logical: the inner `SUM` is the GROUP BY aggregate per channel; the outer `SUM … OVER ()` adds those per-channel values across the whole result (an empty `OVER ()` means "all rows").

Filtering on a window result: QUALIFY

You cannot use a window function in `WHERE`, because windows are computed *after* `WHERE` (Section 3.16). Standard SQL needs a subquery; Snowflake (and DuckDB) offer `QUALIFY`, which filters on window results directly. The most recent order per customer:

```
SELECT customer_id, order_id, order_ts
FROM orders
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts DESC) = 1
ORDER BY customer_id;
```

Result (executed on DuckDB):

customer_id	order_id	order_ts
1	1009	2026-03-01 13:45:00
2	1007	2026-02-14 09:20:00
3	1012	2026-03-09 11:00:00
4	1011	2026-03-08 18:25:00
5	1006	2026-02-11 16:30:00
6	1008	2026-02-20 23:55:00
7	1010	2026-03-03 07:10:00

7 rows.

3.12 Set operators

Set operators combine the *rows* of two queries with the same number of compatible columns.

Operator	Result
`UNION`	Rows in either query, duplicates removed
`UNION ALL`	Rows in either query, duplicates kept (faster — no de-duplication)
`INTERSECT`	Rows in both queries
`EXCEPT` (Snowflake also accepts `MINUS`)	Rows in the first query but not the second

```
SELECT customer_id FROM orders WHERE channel = 'web'
INTERSECT
SELECT customer_id FROM orders WHERE channel = 'app'
ORDER BY customer_id;
```

Result (executed on DuckDB):

customer_id
1
4

2 rows.

Customers who have used *both* channels.

```
SELECT customer_id FROM customers
EXCEPT
SELECT customer_id FROM orders
ORDER BY customer_id;
```

Result (executed on DuckDB):

customer_id
8

1 row.

```
SELECT 'customer' AS entity, COUNT(*) AS n FROM customers
UNION ALL
SELECT 'product', COUNT(*) FROM products
UNION ALL
SELECT 'order', COUNT(*) FROM orders;
```

Result (executed on DuckDB):

entity	n
customer	8
product	7
order	12

3 rows.

PREFER UNION ALL

Use `UNION ALL` unless you specifically need duplicates removed. `UNION` must sort or hash every row to find duplicates, which costs time on large data — and silently removing legitimate duplicate rows can hide data problems.

3.13 DDL: creating and changing structure

You have already used `CREATE TABLE` and `CREATE VIEW`. The main DDL commands:

Command	Purpose
`CREATE TABLE … (columns)`	New empty table
`CREATE TABLE … AS SELECT …` (CTAS)	New table filled from a query
`CREATE VIEW … AS SELECT …`	Saved query
`ALTER TABLE … ADD COLUMN / RENAME COLUMN / DROP COLUMN`	Change structure
`DROP TABLE [IF EXISTS] …`	Remove a table
`TRUNCATE TABLE …`	Remove all rows, keep structure

```
CREATE TABLE customer_summary AS
SELECT c.customer_id, c.name, c.country,
       COUNT(o.order_id) AS orders_placed
FROM customers c
```

```
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name, c.country;

ALTER TABLE customer_summary ADD COLUMN segment VARCHAR(20);

SELECT * FROM customer_summary ORDER BY customer_id LIMIT 4;
```

Result (executed on DuckDB):

customer_id	name	country	orders_placed	segment
1	Aarav Sharma	IN	3	NULL
2	Priya Nair	IN	2	NULL
3	Oliver Smith	UK	2	NULL
4	Amelia Jones	UK	2	NULL

4 rows.

SNOWFLAKE DDL EXTRAS

Snowflake adds several DDL conveniences you will use constantly: `CREATE OR REPLACE` (atomically replace an object), `CREATE … IF NOT EXISTS`, `CREATE OR ALTER` (declaratively converge an object to a definition), `CREATE TABLE … CLONE` (zero-copy clone, Chapter 21), `CREATE TABLE … LIKE` (copy structure) and `UNDROP TABLE` (restore a dropped table within the Time Travel period, Chapter 8). Snowflake DDL statements also commit any open transaction implicitly — see Section 3.15.

3.14 DML: INSERT, UPDATE, DELETE and MERGE

UPDATE

```
UPDATE customer_summary
SET segment = CASE WHEN orders_placed >= 2 THEN 'repeat'
                   WHEN orders_placed  = 1 THEN 'one-time'
                   ELSE 'prospect' END;

SELECT customer_id, name, orders_placed, segment
FROM customer_summary ORDER BY customer_id;
```

Result (executed on DuckDB):

customer_id	name	orders_placed	segment
1	Aarav Sharma	3	repeat
2	Priya Nair	2	repeat
3	Oliver Smith	2	repeat
4	Amelia Jones	2	repeat
5	Ethan Brown	1	one-time
6	Diya Patel	1	one-time
7	Noah Wilson	1	one-time
8	Kavya Reddy	0	prospect

8 rows.

> **UPDATE AND DELETE WITHOUT WHERE**
>
> `UPDATE t SET …` and `DELETE FROM t` without a `WHERE` clause affect **every row**. Before running an UPDATE or DELETE on important data, run the equivalent `SELECT … WHERE …` to see exactly which rows will change. In Snowflake, Time Travel (Chapter 8) can undo many mistakes — but only within the retention period and only if you notice in time.

DELETE

```
DELETE FROM customer_summary WHERE segment = 'prospect';

SELECT COUNT(*) AS remaining_rows FROM customer_summary;
```

Result (executed on DuckDB):

remaining_rows
7

1 row.

MERGE: insert or update in one statement

`MERGE` compares a *source* with a *target* by key: matching rows are updated (or deleted), non-matching source rows are inserted. It is the workhorse of incremental loading (Chapters 9 and 14) because it is naturally **idempotent** — running it twice with the same source gives the same result.

```
CREATE TABLE product_price_updates (product_id INTEGER, sku VARCHAR(30), product_name
VARCHAR(100),
                                    category VARCHAR(40), unit_price DECIMAL(10,2));
INSERT INTO product_price_updates VALUES
 (104, 'MUG-STEEL',  'Insulated Steel Mug', 'Home',        699.00),   -- price change
 (108, 'SSD-1TB',    'Portable SSD 1TB',    'Electronics', 7499.00);  -- new product
CREATE TABLE products_copy AS SELECT * FROM products;
```

```
MERGE INTO products_copy AS t
USING product_price_updates AS s
   ON t.product_id = s.product_id
WHEN MATCHED AND t.unit_price <> s.unit_price THEN
    UPDATE SET unit_price = s.unit_price
WHEN NOT MATCHED THEN
    INSERT (product_id, sku, product_name, category, unit_price)
    VALUES (s.product_id, s.sku, s.product_name, s.category, s.unit_price);

SELECT product_id, sku, unit_price FROM products_copy
WHERE product_id IN (104, 108) ORDER BY product_id;
```

Result (executed on DuckDB):

product_id	sku	unit_price
104	MUG-STEEL	699.00
108	SSD-1TB	7499.00

2 rows.

Line by line: `MERGE INTO products_copy AS t` names the target; `USING … AS s` the source; `ON` the matching key. `WHEN MATCHED AND … THEN UPDATE` changes existing rows only if the price actually differs (avoiding pointless writes). `WHEN NOT MATCHED THEN INSERT` adds new products. Run the MERGE a second time and nothing changes: the update condition is false and the insert finds a match.

> **SNOWFLAKE MERGE AND DUPLICATE SOURCE KEYS**
> If the source contains two rows for the same key, a MERGE becomes ambiguous: which one should update the target? Snowflake raises an error for nondeterministic updates by default (governed by the `ERROR_ON_NONDETERMINISTIC_MERGE` parameter). Always de-duplicate the source first — Section 3.17 shows how.

3.15 Transactions

Concept

A **transaction** is a group of statements that succeed or fail **as a unit**. Transactions provide the **ACID** properties:

Property	Meaning
Atomicity	All statements commit, or none do
Consistency	Constraints hold before and after
Isolation	Concurrent transactions do not see each other's uncommitted changes
Durability	Once committed, changes survive failures

```sql
BEGIN TRANSACTION;
DELETE FROM products_copy WHERE category = 'Electronics';
SELECT COUNT(*) AS rows_visible_inside_transaction FROM products_copy;
```

Result (executed on DuckDB):

rows_visible_inside_transaction
3

1 row.

```sql
ROLLBACK;
SELECT COUNT(*) AS rows_after_rollback FROM products_copy;
```

Result (executed on DuckDB):

rows_after_rollback
8

1 row.

The delete was visible *inside* the transaction, but `ROLLBACK` undid it. `COMMIT` would have made it permanent.

SNOWFLAKE TRANSACTION BEHAVIOUR

By default Snowflake runs in **autocommit** mode: each DML statement is its own transaction unless you start one with `BEGIN`. Snowflake uses **READ COMMITTED** isolation for tables. A **DDL statement commits any open transaction** before it runs (and is itself committed) — so do not mix `CREATE` / `ALTER` / `DROP` into a transaction you might roll back. Chapter 8 covers Snowflake transactions in detail, including transactions inside stored procedures.

3.16 How a query is evaluated: logical order

SQL is written in one order but **logically evaluated** in another. Knowing the logical order explains most "why can't I…?" errors.

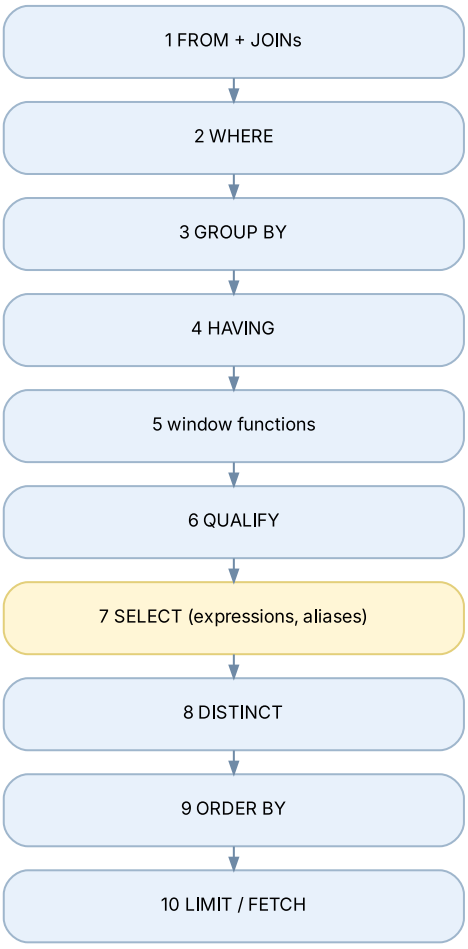


Figure 3.2 — Logical evaluation order of a SELECT statement. The database may physically execute work in a different order for speed, but results must be as if this order were followed.

Consequences:

- **You cannot use a window function in `WHERE`**, because windows are computed after `WHERE` — hence `QUALIFY`.
- **`HAVING` exists** because `WHERE` runs before aggregation.
- **`ORDER BY` can use SELECT aliases**, because it runs after SELECT.

- Standard SQL forbids SELECT aliases in `WHERE` and `GROUP BY` because those run first. Snowflake and DuckDB are more permissive and allow column aliases in `WHERE`, `GROUP BY`, `HAVING` and `QUALIFY` — convenient, but the code will not port to stricter databases.

```sql
SELECT order_id, revenue * 0.1 AS est_margin
FROM order_revenue
WHERE est_margin > 1000
ORDER BY est_margin DESC;
```

Result (executed on DuckDB):

order_id	est_margin
1004	1599.900
1008	1599.900
1006	1389.800
1002	1299.900
1011	1249.900

5 rows.

Logical order versus physical execution

The *logical* order defines the meaning. The *physical* plan is chosen by the **optimizer**, which may filter early, reorder joins, or skip data entirely if it can prove the result is unchanged. In Snowflake, the optimizer can skip whole blocks of storage (*micro-partitions*) using metadata — the most important performance mechanism you will learn (Chapter 5). You can see the physical plan with `EXPLAIN` and in Snowflake's **Query Profile** (Chapter 17).

3.17 Intermediate patterns

These patterns appear in almost every analytics codebase. Learn them as building blocks.

Pattern 1 — Top-N per group

The best-selling product (by revenue) in each category:

```sql
WITH product_revenue AS (
    SELECT p.category, p.product_name, SUM(l.quantity * l.unit_price) AS revenue
    FROM order_lines l
    JOIN products p ON p.product_id = l.product_id
    GROUP BY p.category, p.product_name
)
SELECT category, product_name, revenue
FROM product_revenue
QUALIFY ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC, product_name) = 1
ORDER BY category;
```

Result (executed on DuckDB):

category	product_name	revenue
Electronics	Noise-Cancelling Headphones	38497.00
Furniture	Ergonomic Chair	31998.00

category	product_name	revenue
Home	LED Desk Lamp	7596.00

3 rows.

Pattern 2 — De-duplication

Pipelines often receive the same record twice (retries, overlapping extracts). Keep the latest version of each key:

```
CREATE TABLE raw_customer_updates (customer_id INTEGER, email VARCHAR(200), loaded_at
TIMESTAMP);
INSERT INTO raw_customer_updates VALUES
 (2, 'priya@example.com',     TIMESTAMP '2026-03-01 01:00:00'),
 (2, 'priya.nair@example.com', TIMESTAMP '2026-03-05 01:00:00'),
 (2, 'priya.nair@example.com', TIMESTAMP '2026-03-05 01:00:00'),
 (5, 'ethan@example.com',     TIMESTAMP '2026-03-02 01:00:00');
```

```
SELECT customer_id, email, loaded_at
FROM raw_customer_updates
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY loaded_at DESC) = 1
ORDER BY customer_id;
```

Result (executed on DuckDB):

customer_id	email	loaded_at
2	priya.nair@example.com	2026-03-05 01:00:00
5	ethan@example.com	2026-03-02 01:00:00

2 rows.

This de-duplicated result is exactly what you would feed into the `USING` clause of a MERGE.

Pattern 3 — Finding duplicates

```
SELECT customer_id, email, loaded_at, COUNT(*) AS copies
FROM raw_customer_updates
GROUP BY customer_id, email, loaded_at
HAVING COUNT(*) > 1;
```

Result (executed on DuckDB):

customer_id	email	loaded_at	copies
2	priya.nair@example.com	2026-03-05 01:00:00	2

1 row.

Pattern 4 — First event per entity (cohorts)

Customers grouped by the month of their first order:

```
WITH first_orders AS (
    SELECT customer_id, MIN(order_ts) AS first_order_ts
    FROM orders
    WHERE status <> 'CANCELLED'
    GROUP BY customer_id
)
```

```
SELECT DATE_TRUNC('month', first_order_ts)::DATE AS cohort_month,
       COUNT(*)                                  AS new_customers
FROM first_orders
GROUP BY 1
ORDER BY 1;
```

Result (executed on DuckDB):

cohort_month	new_customers
2026-01-01	2
2026-02-01	3
2026-03-01	2

3 rows.

Pattern 5 — Filling gaps with a calendar

Monthly revenue including months with no sales requires a list of all months to join against. Here we generate one with a set of values; in Snowflake you would typically use a calendar table or `GENERATOR` (Chapter 8).

```
WITH months AS (
    SELECT m::DATE AS month
    FROM (VALUES (DATE '2025-12-01'), (DATE '2026-01-01'),
                 (DATE '2026-02-01'), (DATE '2026-03-01')) AS v(m)
),
monthly AS (
    SELECT DATE_TRUNC('month', order_ts)::DATE AS month, SUM(revenue) AS revenue
    FROM order_revenue
    WHERE status <> 'CANCELLED'
    GROUP BY 1
)
SELECT m.month, COALESCE(r.revenue, 0) AS revenue
FROM months m
LEFT JOIN monthly r ON r.month = m.month
ORDER BY m.month;
```

Result (executed on DuckDB):

month	revenue
2025-12-01	0.00
2026-01-01	20944.00
2026-02-01	38189.00
2026-03-01	23493.00

4 rows.

December 2025 appears with zero revenue instead of disappearing — essential for correct charts and averages.

3.18 Common mistakes

SQL MISTAKES TO RECOGNISE INSTANTLY

1. **Comparing with NULL using `=`.** Use `IS NULL` / `IS NOT NULL`.
2. **`NOT IN` with a subquery that can return NULL.** Use `NOT EXISTS`.
3. **Filtering the right table of a LEFT JOIN in WHERE.** Put the condition in `ON`.
4. **Summing after a one-to-many join (fan-out).** Aggregate to a common grain first.
5. **`BETWEEN` on timestamps for whole days.** Use `>= start AND < next_start`.
6. **`ROW_NUMBER()` without a unique tie-breaker.** Results change between runs.
7. **`SELECT *` in production queries.** Name columns.
8. **`UNION` when `UNION ALL` was meant.** Silent de-duplication.
9. **UPDATE/DELETE without previewing with SELECT.** Check first.
10. **Relying on unenforced keys in Snowflake.** Validate uniqueness in the pipeline.
11. **Integer division surprises.** In some databases `7/2 = 3`; Snowflake and DuckDB return 3.5, but write `7 / 2.0` or cast when porting code.

3.19 Troubleshooting exercise

EXERCISE — THE DASHBOARD COUNTS TOO MANY ORDERS

An analyst writes the query below to count orders per country and reports 8 orders for India, but Indian customers placed only 6 orders. Explain the bug and fix it. (Solution in Section 3.24.)

```sql
SELECT c.country, COUNT(*) AS orders
FROM customers c
JOIN orders o       ON o.customer_id = c.customer_id
JOIN order_lines l  ON l.order_id = o.order_id
GROUP BY c.country;
```

3.20 Practice exercises

Write a query for each. Solutions, executed against the sample data, are in Section 3.24.

1. List all products in the Electronics category priced under ₹5,000, cheapest first.
2. Count customers per country.
3. Show customers who signed up in 2025.
4. Total quantity sold per product name, including products never sold (show 0).
5. Revenue per category, excluding cancelled orders, highest first.
6. Customers whose email domain is `example.com` (exactly, not `example.co.uk`).
7. The average number of lines per order, rounded to 2 decimals.

8. Orders placed between 18:00 and 23:59 (any day).
9. For each customer, their first and most recent order date (customers with orders only).
10. Customers with no loyalty tier, showing `'NONE'` instead of NULL.
11. The second most expensive product.
12. Each order with its revenue and the customer's average order revenue alongside.
13. Products that were sold at a price different from the current catalogue price.
14. Monthly revenue with month-over-month change (excluding cancelled orders).
15. Customers who ordered in January 2026 *and* in March 2026.
16. Each customer's share of total non-cancelled revenue, as a percentage with 1 decimal.
17. Customers who referred at least one other customer, with the number referred.
18. The order with the most distinct products per channel.
19. Classify customers as 'Big spender' (> ₹15,000 non-cancelled revenue), 'Regular' (> 0) or 'Inactive'.
20. Write a MERGE that upserts this record into `customer_summary`: customer 9, 'Ishaan Gupta', 'IN', 0 orders, segment 'prospect'. Then show the row.

3.21 Chapter summary

SQL describes *what* result you want; the database decides *how* to compute it. A SELECT reads from tables, filters rows with WHERE, groups and aggregates, filters groups with HAVING, computes window functions, filters on them with QUALIFY, projects columns, sorts and limits — in that logical order. Joins combine tables; the join type determines what happens to unmatched rows, and joining one-to-many relationships repeats rows, which must be handled before aggregating. NULL represents unknown values and follows three-valued logic, which causes the most subtle bugs. CTEs organise complex queries into readable steps; window functions compute rankings, running totals and comparisons without collapsing rows. DDL defines structure, DML changes data, MERGE performs idempotent upserts, and transactions group statements atomically. In Snowflake, most constraints are informational, so pipelines must enforce uniqueness themselves.

3.22 Key takeaways

- Think in **grain**: know what one row represents at every step.
- Respect **NULL**: `IS NULL`, `COALESCE`, `NOT EXISTS`.
- Choose joins deliberately; check row counts before and after.
- Use CTEs to make logic readable and testable.
- Window functions + `QUALIFY` solve ranking, de-duplication and top-N elegantly.
- Prefer idempotent `MERGE` for incremental changes.

3.23 Further reading

- Snowflake SQL command reference: https://docs.snowflake.com/en/sql-reference-commands

- Snowflake — Query syntax (SELECT, QUALIFY, joins): https://docs.snowflake.com/en/sql-reference/constructs
- Snowflake — Window functions: https://docs.snowflake.com/en/sql-reference/functions-window
- Snowflake — Transactions: https://docs.snowflake.com/en/sql-reference/transactions
- Snowflake — Constraints overview: https://docs.snowflake.com/en/sql-reference/constraints-overview
- DuckDB SQL documentation: https://duckdb.org/docs/current/sql/introduction.html

3.24 Solutions

Troubleshooting exercise

Joining `order_lines` multiplies each order by its number of lines (the fan-out trap), so `COUNT(*)` counts *lines*, not orders. The lines table is not needed at all. Either drop the join or count distinct orders:

```
SELECT c.country, COUNT(DISTINCT o.order_id) AS orders
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.country
ORDER BY c.country;
```

Result (executed on DuckDB):

country	orders
IN	6
UK	4
US	2

3 rows.

Practice exercise solutions

1.

```
SELECT product_name, unit_price
FROM products
WHERE category = 'Electronics' AND unit_price < 5000
ORDER BY unit_price;
```

Result (executed on DuckDB):

product_name	unit_price
HDMI Cable 2m	399.00
USB-C Charger 65W	899.00
Mechanical Keyboard	4499.00

3 rows.

2.

```
SELECT country, COUNT(*) AS customers FROM customers GROUP BY country ORDER BY country;
```

Result (executed on DuckDB):

country	customers
IN	4
UK	2
US	2

3 rows.

3.

```
SELECT name, signup_date FROM customers
WHERE signup_date >= DATE '2025-01-01' AND signup_date < DATE '2026-01-01'
ORDER BY signup_date;
```

Result (executed on DuckDB):

name	signup_date
Amelia Jones	2025-01-11
Ethan Brown	2025-02-14
Diya Patel	2025-06-30
Noah Wilson	2025-09-05

4 rows.

4. A LEFT JOIN keeps the never-sold HDMI cable; `COALESCE` converts its NULL sum to 0.

```
SELECT p.product_name, COALESCE(SUM(l.quantity), 0) AS qty_sold
FROM products p
LEFT JOIN order_lines l ON l.product_id = p.product_id
GROUP BY p.product_name
ORDER BY qty_sold DESC, p.product_name;
```

Result (executed on DuckDB):

product_name	qty_sold
Insulated Steel Mug	7
USB-C Charger 65W	7
LED Desk Lamp	4
Noise-Cancelling Headphones	3
Ergonomic Chair	2
Mechanical Keyboard	2
HDMI Cable 2m	0

7 rows.

5.

```
SELECT p.category, SUM(l.quantity * l.unit_price) AS revenue
FROM order_lines l
JOIN orders o   ON o.order_id = l.order_id
JOIN products p ON p.product_id = l.product_id
WHERE o.status <> 'CANCELLED'
GROUP BY p.category
ORDER BY revenue DESC;
```

Result (executed on DuckDB):

category	revenue
Electronics	53788.00
Furniture	15999.00
Home	12839.00

3 rows.

6. `LIKE '%@example.com'` anchors the pattern to the end of the string, so `.co.uk` addresses are excluded.

```
SELECT name, email FROM customers WHERE email LIKE '%@example.com' ORDER BY name;
```

Result (executed on DuckDB):

name	email
Aarav Sharma	aarav@example.com
Diya Patel	diya@example.com
Ethan Brown	ethan@example.com
Kavya Reddy	kavya@example.com
Noah Wilson	noah@example.com
Priya Nair	priya@example.com

6 rows.

7.

```
SELECT ROUND(AVG(line_count), 2) AS avg_lines_per_order
FROM (SELECT order_id, COUNT(*) AS line_count FROM order_lines GROUP BY order_id) t;
```

Result (executed on DuckDB):

avg_lines_per_order
1.42

1 row.

8.

```
SELECT order_id, order_ts FROM orders
WHERE EXTRACT(HOUR FROM order_ts) >= 18
ORDER BY order_ts;
```

Result (executed on DuckDB):

order_id	order_ts
1002	2026-01-07 19:42:00
1004	2026-01-28 21:10:00
1008	2026-02-20 23:55:00
1011	2026-03-08 18:25:00

4 rows.

9.

```
SELECT c.name,
       MIN(o.order_ts)::DATE AS first_order,
       MAX(o.order_ts)::DATE AS latest_order
FROM customers c
```

```
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.name
ORDER BY first_order;
```

Result (executed on DuckDB):

name	first_order	latest_order
Aarav Sharma	2026-01-05	2026-03-01
Priya Nair	2026-01-07	2026-02-14
Oliver Smith	2026-01-28	2026-03-09
Amelia Jones	2026-02-03	2026-03-08
Ethan Brown	2026-02-11	2026-02-11
Diya Patel	2026-02-20	2026-02-20
Noah Wilson	2026-03-03	2026-03-03

7 rows.

10.

```
SELECT name, COALESCE(loyalty_tier, 'NONE') AS tier
FROM customers WHERE loyalty_tier IS NULL ORDER BY name;
```

Result (executed on DuckDB):

name	tier
Noah Wilson	NONE
Oliver Smith	NONE

2 rows.

11. `DENSE_RANK` handles ties correctly (if two products shared the top price, the "second most expensive" would be the next distinct price).

```
SELECT product_name, unit_price
FROM products
QUALIFY DENSE_RANK() OVER (ORDER BY unit_price DESC) = 2;
```

Result (executed on DuckDB):

product_name	unit_price
Noise-Cancelling Headphones	12999.00

1 row.

12.

```
SELECT order_id, customer_id, revenue,
       ROUND(AVG(revenue) OVER (PARTITION BY customer_id), 2) AS customer_avg_revenue
FROM order_revenue
ORDER BY customer_id, order_id;
```

Result (executed on DuckDB):

order_id	customer_id	revenue	customer_avg_revenue
1001	1	2547.00	3281.33
1003	1	5398.00	3281.33

order_id	customer_id	revenue	customer_avg_revenue
1009	1	1899.00	3281.33
1002	2	12999.00	7997.5
1007	2	2996.00	7997.5
1004	3	15999.00	10297.5
1012	3	4596.00	10297.5
1005	4	5296.00	8897.5
1011	4	12499.00	8897.5
1006	5	13898.00	13898
1008	6	15999.00	15999
1010	7	4499.00	4499

12 rows.

13.

```
SELECT l.order_id, p.product_name, l.unit_price AS sold_at, p.unit_price AS catalogue_price
FROM order_lines l
JOIN products p ON p.product_id = l.product_id
WHERE l.unit_price <> p.unit_price;
```

Result (executed on DuckDB):

order_id	product_name	sold_at	catalogue_price
1011	Noise-Cancelling Headphones	12499.00	12999.00

1 row.

14. `LAG` over the monthly aggregate gives the previous month's revenue.

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_ts)::DATE AS month, SUM(revenue) AS revenue
    FROM order_revenue
    WHERE status <> 'CANCELLED'
    GROUP BY 1
)
SELECT month, revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS change_vs_prev_month
FROM monthly
ORDER BY month;
```

Result (executed on DuckDB):

month	revenue	change_vs_prev_month
2026-01-01	20944.00	NULL
2026-02-01	38189.00	17245.00
2026-03-01	23493.00	-14696.00

3 rows.

15.

```
SELECT customer_id FROM orders
WHERE order_ts >= TIMESTAMP '2026-01-01' AND order_ts < TIMESTAMP '2026-02-01'
INTERSECT
SELECT customer_id FROM orders
WHERE order_ts >= TIMESTAMP '2026-03-01' AND order_ts < TIMESTAMP '2026-04-01';
```

Result (executed on DuckDB):

customer_id
1
3

2 rows.

16.

```sql
SELECT customer_id,
       SUM(revenue) AS revenue,
       ROUND(100.0 * SUM(revenue) / SUM(SUM(revenue)) OVER (), 1) AS pct_of_total
FROM order_revenue
WHERE status <> 'CANCELLED'
GROUP BY customer_id
ORDER BY revenue DESC;
```

Result (executed on DuckDB):

customer_id	revenue	pct_of_total
4	17795.00	21.5
6	15999.00	19.4
2	15995.00	19.4
5	13898.00	16.8
1	9844.00	11.9
3	4596.00	5.6
7	4499.00	5.4

7 rows.

17.

```sql
SELECT r.name AS referrer, COUNT(*) AS customers_referred
FROM customers c
JOIN customers r ON r.customer_id = c.referred_by
GROUP BY r.name
ORDER BY customers_referred DESC, r.name;
```

Result (executed on DuckDB):

referrer	customers_referred
Aarav Sharma	2
Oliver Smith	1
Priya Nair	1

3 rows.

18.

```sql
SELECT o.channel, o.order_id, COUNT(DISTINCT l.product_id) AS distinct_products
FROM orders o
JOIN order_lines l ON l.order_id = o.order_id
GROUP BY o.channel, o.order_id
QUALIFY ROW_NUMBER() OVER (PARTITION BY o.channel
                           ORDER BY COUNT(DISTINCT l.product_id) DESC, o.order_id) = 1
ORDER BY o.channel;
```

Result (executed on DuckDB):

channel	order_id	distinct_products
app	1003	2
web	1001	2

2 rows.

19.

```sql
WITH spend AS (
    SELECT c.customer_id, c.name,
           COALESCE(SUM(CASE WHEN r.status <> 'CANCELLED' THEN r.revenue END), 0) AS revenue
    FROM customers c
    LEFT JOIN order_revenue r ON r.customer_id = c.customer_id
    GROUP BY c.customer_id, c.name
)
SELECT name, revenue,
       CASE WHEN revenue > 15000 THEN 'Big spender'
            WHEN revenue > 0     THEN 'Regular'
            ELSE 'Inactive' END AS segment
FROM spend
ORDER BY revenue DESC, name;
```

Result (executed on DuckDB):

name	revenue	segment
Amelia Jones	17795.00	Big spender
Diya Patel	15999.00	Big spender
Priya Nair	15995.00	Big spender
Ethan Brown	13898.00	Regular
Aarav Sharma	9844.00	Regular
Oliver Smith	4596.00	Regular
Noah Wilson	4499.00	Regular
Kavya Reddy	0.00	Inactive

8 rows.

Oliver Smith is "Regular" because his cancelled order is excluded but his delivered order 1012 counts.

20.

```sql
MERGE INTO customer_summary t
USING (SELECT 9 AS customer_id, 'Ishaan Gupta' AS name, 'IN' AS country,
              0 AS orders_placed, 'prospect' AS segment) s
   ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET name = s.name, country = s.country,
                             orders_placed = s.orders_placed, segment = s.segment
WHEN NOT MATCHED THEN INSERT (customer_id, name, country, orders_placed, segment)
                      VALUES (s.customer_id, s.name, s.country, s.orders_placed, s.segment);

SELECT * FROM customer_summary WHERE customer_id = 9;
```

Result (executed on DuckDB):

customer_id	name	country	orders_placed	segment
9	Ishaan Gupta	IN	0	prospect

1 row.

PART II

Snowflake Core

Level 2 of 7. What Snowflake is, how its three-layer architecture executes a query, which objects you build with, and how data types and table types behave.

CHAPTER 4

Introduction to Snowflake

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- explain what Snowflake is — and what it is not;
- summarise how Snowflake evolved from a cloud data warehouse into a broader data and AI platform;
- list Snowflake's major workloads and core use cases;
- explain how Snowflake runs on AWS, Azure and Google Cloud, and what an account, region and account identifier are;
- compare the Standard, Enterprise, Business Critical and Virtual Private Snowflake editions;
- weigh Snowflake's strengths, limitations and architectural trade-offs;
- log in to Snowsight, create a warehouse and a lab role, and run your first queries.

Prerequisites: Chapters 1–3.

Where this leads: Chapter 5 opens up the architecture; Chapter 6 introduces every object type you create; Chapter 7 covers data types and table types.

4.1 What Snowflake is

Concept

Snowflake is a fully managed, cloud-native data platform that stores data and runs SQL (and Python, Java, Scala and AI workloads) against it, delivered as a service on Amazon Web Services, Microsoft Azure and Google Cloud. Snowflake describes itself today as the *AI Data Cloud*.

Unpacking that sentence:

- **Fully managed:** there is no software to install, no servers or operating systems to configure, no storage to provision, no indexes to rebuild and no upgrades to schedule (Chapter 2's SaaS model).
- **Cloud-native:** it was designed from the beginning for cloud infrastructure, built around cheap object storage and on-demand compute, rather than adapted from an on-premises database.
- **Data platform:** it began as a data warehouse, but now also handles semi-structured and unstructured data, data engineering pipelines, open-format lakehouse tables, data sharing, applications, transactional tables and AI.

What Snowflake is not

Understanding the boundaries prevents design mistakes:

Snowflake is not…	Because…	Use instead or alongside
A replacement for your application's main transactional database (in most cases)	Standard tables are optimised for large analytical scans, not thousands of single-row updates per second	PostgreSQL, MySQL, cloud OLTP databases — or, for some workloads, Snowflake hybrid tables or Snowflake Postgres (Chapter 7)
A file system or general object store	Data in standard tables lives in Snowflake-managed storage accessed through SQL	Cloud object storage; Snowflake stages reference it
A message queue	It ingests streams but does not act as a broker	Apache Kafka, cloud messaging services
A BI tool	Snowsight can chart query results, but enterprise reporting usually uses dedicated tools	Power BI, Tableau, Looker, Sigma, Streamlit
Self-securing	It provides strong controls, but you configure them (Chapter 2 shared responsibility)	Your RBAC, network, masking and monitoring design
Free when idle	Storage is always billed; compute only when running — but you must configure suspension	Auto-suspend, budgets, resource monitors

4.2 A short history and evolution

Knowing how Snowflake evolved explains why its features are shaped the way they are.

Period	Development	Why it matters to you
2012	Founded by Benoît Dageville and Thierry Cruanes (formerly at Oracle) and Marcin Żukowski (co-founder of the Vectorwise columnar engine) to build a data warehouse designed for the cloud	Explains the core design: columnar engine, separation of storage and compute
2014–2015	Public launch and general availability on AWS	The original three-layer architecture (Chapter 5) dates from here
Late 2010s	Expansion to Microsoft Azure and Google Cloud; Secure Data Sharing; data marketplace	Multi-cloud accounts; sharing data without copying (Chapter 21)
2020	Initial public offering	Broad enterprise adoption; ecosystem of partners
Early 2020s	Snowpark (Python/Java/Scala on Snowflake compute), Streamlit acquisition, Snowpark Container Services, dynamic tables, hybrid tables, Iceberg tables, Native Apps	Snowflake becomes a platform for engineering and applications, not just SQL analytics
2023–2025	Snowflake Cortex: LLM functions, Cortex Search, Cortex Analyst, Cortex Agents, Snowflake Intelligence, Cortex Code; Openflow; Gen2 warehouses; managed MCP server	AI workloads move inside the governed data platform (Part VII)
2026	Snowflake Intelligence renamed CoWork; Cortex Code renamed CoCo; Adaptive Compute GA; Snowflake Optima; Iceberg v3; Snowflake Postgres GA	See "What's New in Snowflake" at the front of this volume

The through-line is constant: **keep data in one governed place and bring different kinds of compute — SQL, Python, containers, AI models — to the data**, rather than copying data to many specialised systems.

4.3 Core use cases and workloads

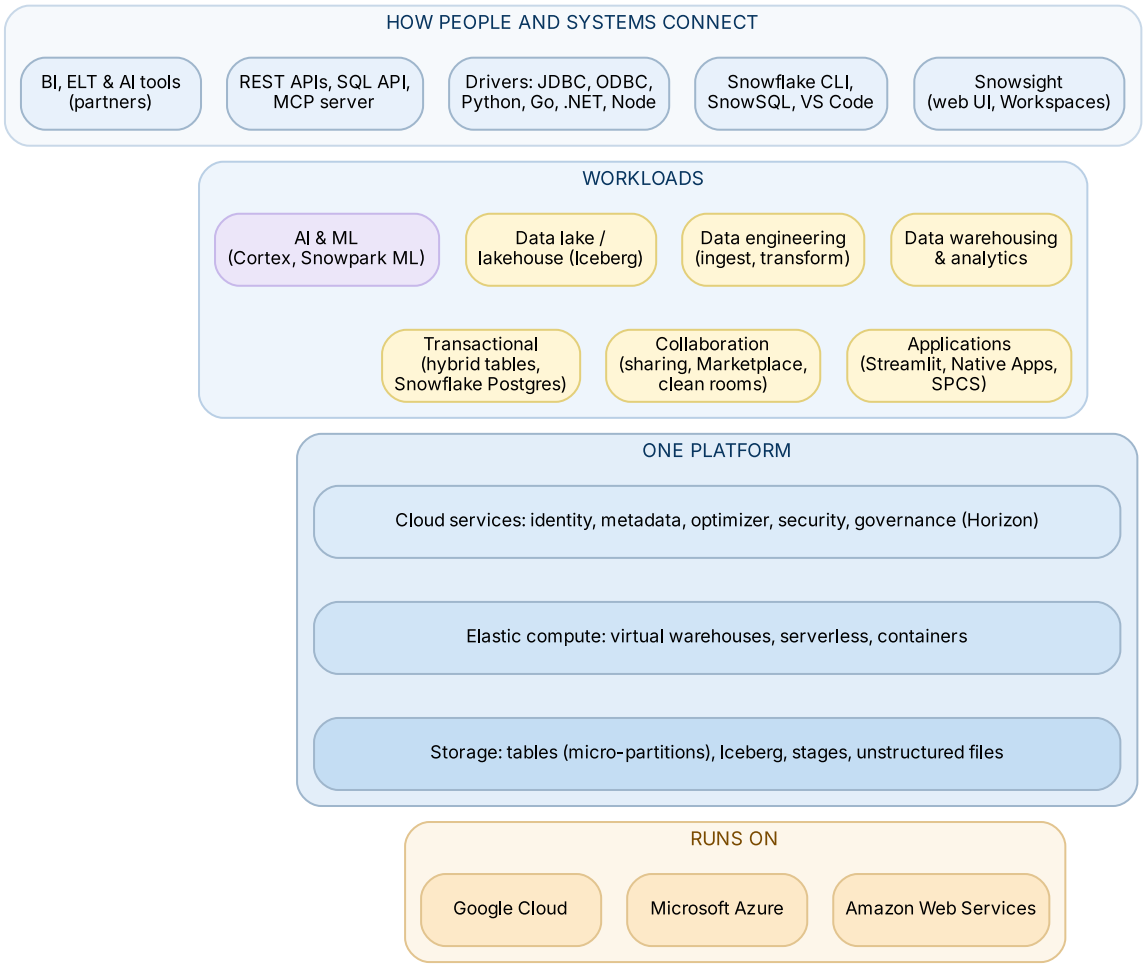


Figure 4.1 — Snowflake as one platform: several ways to connect, many workloads, three architectural layers, three clouds.

Workload	What it means	ByteMart example	Chapters
Data warehousing & analytics	Integrated, historical, modelled data for reporting and analysis	Sales dashboards, finance reconciliation	8, 9, 17
Data engineering	Ingestion and transformation pipelines, batch and streaming	Loading orders, clickstream, CDC from PostgreSQL	12–16
Data lake / lakehouse	Querying and managing open-format files and Iceberg tables	Sharing clickstream Iceberg tables with a Spark-based team	7, 37
AI and ML	LLM functions, search, agents, feature engineering, model training and serving	Ticket summarisation, product Q&A assistant, churn model	23–31
Applications	Data apps and services running next to the data	Streamlit returns-approval app	32–34
Collaboration	Sharing live data across accounts and organisations; marketplace; clean rooms	Sharing sales-through data with a supplier	21, 38

Workload	What it means	ByteMart example	Chapters
Transactional / operational	Low-latency point reads and writes	Inventory reservation lookups	7
Cybersecurity / observability analytics	Large-scale log and event analysis	Security log retention and threat queries	17, 19

4.4 How Snowflake runs on the clouds

Accounts, regions and clouds

When an organisation signs up, it receives an **organization** (a container for one or more accounts) and at least one **account**. Each account lives in exactly **one cloud provider and one region** — for example, AWS Asia Pacific (Mumbai). All of that account's storage and compute run there.

Key consequences:

- To be in two regions (for residency or disaster recovery), you create **two accounts** in the same organization and use replication or sharing between them (Chapters 21, 38, 39).
- Feature availability — especially particular AI models — can differ by region (Chapter 24).
- Compute and storage resources are managed by Snowflake on the cloud provider; you do not see them in your own cloud console.

Account identifiers and URLs

Each account has an **account identifier**. The preferred form combines the organization name and account name:

```
<orgname>-<account_name>                        e.g.  bytemart-analytics_prod
https://<orgname>-<account_name>.snowflakecomputing.com
```

An older form, the **account locator** (for example `xy12345` plus a region suffix), still works and appears in some tools and documentation. Prefer the organization-based identifier in new configurations: it is stable even when accounts move or replicate.

> **UNDERSCORES VERSUS HYPHENS IN HOSTNAMES**
>
> Account names may contain underscores, but some tools have trouble with underscores in hostnames. Snowflake documents a hyphenated form of the URL for such clients — the Snowflake-managed MCP server documentation, for example, explicitly recommends hyphens. If a connection fails mysteriously, try the hyphenated hostname.

Interfaces

Interface	Use it for
Snowsight (`app.snowflake.com`)	The web interface: worksheets, Workspaces (files, notebooks, dbt projects), dashboards, data browsing, administration, cost views, CoCo assistant
Snowflake CLI (`snow`)	Scripting, deploying Snowpark/Streamlit/Native Apps, running SQL, CI/CD

Interface	Use it for
SnowSQL	The older command-line SQL client; still available
Drivers and connectors	Applications in Python, Java (JDBC), ODBC, Go, .NET, Node.js, PHP; Spark and Kafka connectors
REST APIs	SQL API, Snowflake REST APIs for objects, Cortex REST APIs
Partner tools	BI, ELT, catalog, observability and AI tools

4.5 Snowflake editions

An **edition** is a feature and security tier chosen per account. Each edition includes everything in the editions below it. The edition also affects the price of a credit.

> **VERIFIED AGAINST THE EDITION MATRIX AT THE RESEARCH CUTOFF**
>
> The table below summarises the official matrix. It is not exhaustive; consult the documentation for the full list. An account's edition can be changed with `ALTER ACCOUNT` (GA 6 May 2026) or through Snowflake.

Capability	Stand-ard	Enter-prise	Business Critical	VPS
Core SQL, all data types, Snowpark, Streamlit, Cortex AI functions, dynamic tables, hybrid tables, Iceberg, streams, tasks, Snowpipe, Snowpipe Streaming	✔	✔	✔	✔
Time Travel	up to 1 day	up to 90 days	up to 90 days	up to 90 days
Fail-safe (7 days after Time Travel)	✔	✔	✔	✔
Network policies, MFA, SSO, OAuth, automatic encryption	✔	✔	✔	✔
Multi-cluster warehouses		✔	✔	✔
Materialized views		✔	✔	✔
Search Optimization Service; Query Acceleration Service		✔	✔	✔
Column-level security (masking policies); row access policies		✔	✔	✔
Aggregation and projection policies; differential privacy		✔	✔	✔
Sensitive data classification; `ACCESS_HISTORY`		✔	✔	✔
Data metric functions (data quality)		✔	✔	✔
Periodic rekeying of encrypted data		✔	✔	✔
Adaptive Warehouses (select regions)		✔	✔	✔
Tri-Secret Secure (customer-managed keys)			✔	✔
Private connectivity (AWS PrivateLink, Azure Private Link, Google Private Service Connect)			✔	✔
HIPAA/HITRUST (with a BAA), PCI DSS, FedRAMP/ITAR regions, IRAP-Protected			✔	✔
Account failover/failback and client redirect			✔	✔

Capability	Stand-ard	Enter-prise	Business Critical	VPS
Completely separate Snowflake environment; dedicated metadata store and compute pool				✔

Choosing an edition

- **Standard** suits small teams and non-sensitive analytics.
- **Enterprise** is the common choice for companies with multiple teams: multi-cluster concurrency, longer Time Travel, and the governance policies (masking, row access) that most organisations eventually need. **This book's labs assume Enterprise.**
- **Business Critical** is for regulated or highly sensitive data, private networking and cross-region business continuity.
- **VPS** is for organisations requiring complete isolation from other Snowflake customers.

ByteMart stores customer personal data in India and the UK and wants failover for its revenue dashboards. It starts on **Enterprise** for development and analytics, and plans **Business Critical** for the production account once it needs private connectivity and failover (Chapter 39).

4.6 Strengths, limitations and trade-offs

Strengths

1. **Separation of storage and compute.** Many independent warehouses can read the same data simultaneously without contention; each can be sized and billed separately (Chapter 5).
2. **Elasticity.** Warehouses start in seconds, resize instantly, and suspend automatically; Adaptive Warehouses remove sizing altogether for supported accounts.
3. **Near-zero administration.** No indexes, vacuuming, statistics jobs or storage tuning in the traditional sense; automatic micro-partitioning and metadata.
4. **Semi-structured data as a first-class citizen.** Load JSON, Avro, Parquet, ORC and XML into `VARIANT` and query with SQL (Chapters 7–8).
5. **Data protection features built in.** Time Travel, Fail-safe and zero-copy cloning (Chapters 8, 21, 39).
6. **Secure Data Sharing.** Share live data with other accounts without copying (Chapter 21).
7. **Governance built into the platform.** RBAC, masking, row access policies, tags, classification, lineage and access history (Chapters 20–21).
8. **AI next to the data.** Cortex functions, search, agents and the MCP server operate inside the governance boundary (Part VII).
9. **Multi-cloud and cross-region.** The same SQL and features across three clouds, with replication.

Limitations and trade-offs

Limitation or trade-off	Explanation	Mitigation
Not a general OLTP database	Standard tables favour bulk operations; frequent single-row updates are inefficient	Hybrid tables or Snowflake Postgres for suitable workloads; keep OLTP in operational databases
Most constraints not enforced on standard tables	PK/UNIQUE/FK are informational; duplicates are possible	Enforce uniqueness in pipelines (MERGE, de-duplication, tests)
No traditional indexes	Performance relies on pruning, clustering, caching	Clustering keys, search optimization, materialized views, Optima (Chapter 17)
Small-query overhead	Every query goes through cloud-services compilation; very-low-latency serving needs care	Result cache; interactive warehouses; application caches
Consumption billing requires discipline	Runaway compute or AI calls cost real money quickly	Auto-suspend, resource monitors, budgets, per-user AI quotas (Chapter 18)
Storage format is proprietary for standard tables	Other engines cannot read native tables directly	Iceberg tables for open interoperability; Horizon Catalog for external engine access
Feature availability varies by edition and region	Especially private connectivity and AI models	Plan edition/region early; cross-region inference for AI
Egress costs	Moving data across regions/clouds costs money	Co-locate with sources; share instead of copying

COMMON MISCONCEPTION: 'SNOWFLAKE IS SERVERLESS, SO I DON'T NEED TO THINK ABOUT COMPUTE'

Snowflake removes infrastructure management, but warehouse configuration, query design and scheduling still determine performance and cost. A poorly written query on a 4X-Large warehouse costs 128 times more per hour than on an X-Small. Chapters 17 and 18 teach the discipline.

4.7 Guided lab: your first Snowflake session

LAB 4.1 — SIGN IN, CREATE A LAB ENVIRONMENT AND RUN YOUR FIRST QUERIES

Environment: Snowsight, in a trial or development account where you hold the `ACCOUNTADMIN` role. **Time:** 30–45 minutes. **Cost:** a few minutes of X-Small compute.

Step 1 — Sign in and orient yourself. Open the account URL or `app.snowflake.com`. Explore the left navigation: projects/workspaces (where you write SQL and notebooks), data (browse databases), compute (warehouses), monitoring (query history) and administration (users, roles, cost). Note the **role** and **warehouse** selectors: every query runs as one role, using one warehouse.

Step 2 — Open a worksheet or SQL file in Workspaces and run:

```
SELECT CURRENT_ACCOUNT(), CURRENT_REGION(), CURRENT_ROLE(), CURRENT_USER(),
CURRENT_VERSION();
```

These context functions answer "where am I and who am I?". `CURRENT_REGION()` shows the cloud and region of the account. This query needs no warehouse — Snowflake answers it from cloud services metadata, a first hint of the architecture in Chapter 5.

Step 3 — Create a lab role, warehouse and database. We create a dedicated role so that everyday work does not use `ACCOUNTADMIN`. Chapter 6 explains every line; for now, run it as written, replacing `<your_user_name>`.

```
USE ROLE ACCOUNTADMIN;

-- A small warehouse that suspends itself after 60 seconds idle
CREATE WAREHOUSE IF NOT EXISTS lab_wh
  WAREHOUSE_SIZE      = 'XSMALL'
  AUTO_SUSPEND        = 60
  AUTO_RESUME         = TRUE
  INITIALLY_SUSPENDED = TRUE
  COMMENT = 'Learning labs - Snowflake Learning Bible';

-- A role for lab work, granted to you and placed under SYSADMIN
CREATE ROLE IF NOT EXISTS lab_role COMMENT = 'Learning labs';
GRANT ROLE lab_role TO ROLE SYSADMIN;
GRANT ROLE lab_role TO USER <your_user_name>;

-- Let the lab role use the warehouse and create a database
GRANT USAGE, OPERATE ON WAREHOUSE lab_wh TO ROLE lab_role;
GRANT CREATE DATABASE ON ACCOUNT TO ROLE lab_role;

-- Switch to the lab role from now on
USE ROLE lab_role;
USE WAREHOUSE lab_wh;
CREATE DATABASE IF NOT EXISTS bytemart_lab COMMENT = 'Learning labs';
CREATE SCHEMA IF NOT EXISTS bytemart_lab.sales;
USE SCHEMA bytemart_lab.sales;
```

Line by line (short version):

- `USE ROLE ACCOUNTADMIN` — the most powerful role; used here only to set up.
- `CREATE WAREHOUSE … WAREHOUSE_SIZE='XSMALL'` — smallest compute size (1 credit/hour for Gen1 standard); `AUTO_SUSPEND = 60` seconds; `AUTO_RESUME = TRUE` so queries start it automatically; `INITIALLY_SUSPENDED = TRUE` so creation itself costs nothing.
- `CREATE ROLE lab_role` / `GRANT ROLE lab_role TO ROLE SYSADMIN` — creates a role and attaches it under `SYSADMIN` so administrators can manage objects it creates (a Snowflake best practice explained in Chapter 6).
- `GRANT USAGE, OPERATE ON WAREHOUSE` — allows using and starting/stopping the warehouse.
- `GRANT CREATE DATABASE ON ACCOUNT` — allows creating databases; the lab role will **own** what it creates.

Step 4 — Query sample data. Many accounts include the read-only `SNOWFLAKE_SAMPLE_DATA` database, shared from Snowflake's `SFC_SAMPLES` account. Check:

```
SHOW DATABASES LIKE 'SNOWFLAKE_SAMPLE_DATA';
```

If it is missing (older accounts), an `ACCOUNTADMIN` can create it:

```
USE ROLE ACCOUNTADMIN;
CREATE DATABASE SNOWFLAKE_SAMPLE_DATA FROM SHARE SFC_SAMPLES.SAMPLE_DATA;
GRANT IMPORTED PRIVILEGES ON DATABASE SNOWFLAKE_SAMPLE_DATA TO ROLE lab_role;
USE ROLE lab_role;
```

Then run a query against the TPC-H benchmark data — the SQL you learned in Chapter 3 works unchanged:

```
SELECT l_returnflag,
       l_linestatus,
       COUNT(*)                       AS line_count,
       SUM(l_quantity)                AS total_qty,
       ROUND(AVG(l_extendedprice), 2)   AS avg_price
FROM snowflake_sample_data.tpch_sf1.lineitem
WHERE l_shipdate <= DATE '1998-09-02'
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
```

Expected output: four rows (combinations of return flag and line status) summarising roughly six million line items. The `lineitem` table in `TPCH_SF1` has 6,001,215 rows according to Snowflake's sample-data documentation. Note the elapsed time in the results pane.

Step 5 — Run it again. Re-run the identical query. It should return almost instantly. You just used the **result cache**, explained in Chapter 5. The warehouse may even have been suspended — reused results do not need compute.

Step 6 — Inspect what happened. Open **Query History** (Monitoring), click your two queries, and compare. The first scanned data with the warehouse; the second was served from the result cache. Open the **Query Profile** of the first and look at "Partitions scanned" and "Partitions total" — the subject of Chapter 5.

Step 7 — Clean up compute.

```
ALTER WAREHOUSE lab_wh SUSPEND;
```

(It would suspend itself after 60 idle seconds anyway.)

4.8 Production use case: ByteMart's platform decision

ByteMart's data team evaluated three options for its analytics platform: (1) a self-managed PostgreSQL read replica, (2) a Spark-based lake on object storage, and (3) Snowflake.

Criterion	PostgreSQL replica	Self-managed lake + Spark	Snowflake
Large analytical scans	Weak (row store)	Strong	Strong
Concurrency for 40 analysts	Limited	Requires cluster tuning	Separate warehouses or multi-cluster

Criterion	PostgreSQL replica	Self-managed lake + Spark	Snowflake
Semi-structured clickstream	JSONB, slower at scale	Strong	Strong (`VARIANT`)
Operations effort	Medium	High	Low
Governance (masking, row policies)	Manual	Requires additional tools	Built in (Enterprise)
AI over documents and data	External tools	External tools	Cortex functions, search, agents
Open format interoperability	n/a	Native	Iceberg tables
Cost model	Fixed servers	Clusters + engineers	Consumption; needs FinOps discipline

The team chose Snowflake for low operations effort, governance and AI features, with two conditions recorded in its architecture decision log: **keep transactional workloads in PostgreSQL**, and **establish cost controls before onboarding analysts** (Chapter 18). It also decided to land clickstream as Iceberg tables so that a future Spark-based data-science team could read them without copying.

4.9 Common mistakes

FIRST-MONTH MISTAKES

1. Working as `ACCOUNTADMIN` every day.
2. Creating warehouses without auto-suspend, or with long auto-suspend times.
3. Choosing a region without checking data residency and AI-feature availability.
4. Starting on Standard edition and discovering masking or multi-cluster needs later (edition can be changed, but plan it).
5. Treating Snowflake as an OLTP database for the main application.
6. Assuming primary keys prevent duplicates.
7. Sharing one warehouse among all workloads, so ETL slows dashboards (Chapter 5 explains isolation).

4.10 Troubleshooting exercise

EXERCISE — 'NO ACTIVE WAREHOUSE SELECTED'

A colleague runs `SELECT COUNT(*) FROM bytemart_lab.sales.orders;` and receives an error saying no active warehouse is selected in the session. Yet `SELECT CURRENT_USER();` worked. Explain why, and list three ways to fix it. (Solution at the end of the chapter.)

4.11 Knowledge check

1. Give a one-sentence definition of Snowflake and explain what "fully managed" removes from your responsibilities.

2. Name three things Snowflake is not designed to be, with an alternative for each.
3. Can one Snowflake account span two regions? How do you achieve multi-region presence?
4. Which edition is the minimum for (a) masking policies, (b) private connectivity, (c) multi-cluster warehouses, (d) 30-day Time Travel?
5. Why did the second run of the lab query return almost instantly?
6. What is the preferred format of an account identifier, and why is it preferred over the account locator?
7. List three limitations of Snowflake and a mitigation for each.

4.12 Hands-on assignment

1. Complete Lab 4.1 and capture: the output of the context query; the two execution times of the TPC-H query; partitions scanned versus total for the first run.
2. Write and run three more queries against `SNOWFLAKE_SAMPLE_DATA.TPCH_SF1` using Chapter 3 techniques: a join between `orders` and `customer`; a window-function ranking; a `QUALIFY` top-N per group.
3. Write a half-page edition recommendation for a fictional company of your choice using the edition table.

4.13 Chapter summary

Snowflake is a fully managed, cloud-native data platform running on AWS, Azure and Google Cloud. It began as a cloud data warehouse built on the separation of storage and compute, and has expanded into data engineering, lakehouse, applications, collaboration, transactional and AI workloads while keeping data in one governed place. Each account lives in one cloud region; multi-region designs use multiple accounts with replication or sharing. Editions determine features — Enterprise adds multi-cluster warehouses, extended Time Travel and governance policies; Business Critical adds private connectivity, customer-managed keys, compliance and failover; VPS adds complete isolation. Snowflake's strengths are elasticity, low administration, semi-structured support, built-in data protection, sharing, governance and AI; its trade-offs include unenforced constraints, no traditional indexes, small-query overhead and the need for cost discipline.

4.14 Key takeaways

- One account = one cloud + one region.
- Pick the edition by governance, connectivity and continuity needs; this book assumes Enterprise.
- Do daily work in a least-privilege role, not `ACCOUNTADMIN`.
- Every query runs as a role and (usually) on a warehouse.
- Snowflake removes infrastructure work, not design responsibility.

4.15 Further reading

- Key concepts and architecture: https://docs.snowflake.com/en/user-guide/intro-key-concepts
- Snowflake editions: https://docs.snowflake.com/en/user-guide/intro-editions

- Overview of key features: https://docs.snowflake.com/en/user-guide/intro-supported-features
- Account identifiers: https://docs.snowflake.com/en/user-guide/admin-account-identifier
- Supported cloud regions: https://docs.snowflake.com/en/user-guide/intro-regions
- Using the sample database: https://docs.snowflake.com/en/user-guide/sample-data-using
- Snowsight quick tour: https://docs.snowflake.com/en/user-guide/ui-snowsight-quick-tour

4.16 Solutions

Troubleshooting — no active warehouse. Context functions such as `CURRENT_USER()` are answered by the cloud services layer from session metadata and need no compute. Reading table data requires a virtual warehouse, and the session has none: a session has no warehouse until one is set by the user's `DEFAULT_WAREHOUSE`, the client configuration, the connection parameters, or a `USE WAREHOUSE` statement. Fixes: (1) run `USE WAREHOUSE lab_wh;`; (2) set a default — `ALTER USER <name> SET DEFAULT_WAREHOUSE = lab_wh;`; (3) choose a warehouse in the Snowsight selector or in the driver connection settings. Also verify the current role has `USAGE` on that warehouse — otherwise `USE WAREHOUSE` fails with an authorisation error.

Knowledge check answers.

1. Snowflake is a fully managed cloud data platform that stores data and runs SQL, Python and AI workloads on AWS, Azure and Google Cloud. "Fully managed" removes hardware, operating-system, software installation, patching, storage provisioning and index/statistics maintenance.
2. Examples: the main OLTP database for an application (use PostgreSQL/MySQL or, selectively, hybrid tables/Snowflake Postgres); a message broker (use Kafka); a BI tool (use Power BI/Tableau/Streamlit); a general file system (use object storage).
3. No. Create accounts in each region within one organization, then use database/account replication, failover groups or data sharing.
4. (a) Enterprise; (b) Business Critical; (c) Enterprise; (d) Enterprise (Standard allows up to 1 day).
5. The identical query's result was reused from the result cache, which requires no warehouse compute provided the underlying data had not changed and other reuse conditions held (Chapter 5).
6. `<orgname>-<account_name>`; it is human-readable, unique within Snowflake, and stable across replication and account moves, whereas a locator is tied to a specific deployment and region.
7. For example: unenforced keys → pipeline de-duplication and tests; no indexes → clustering, search optimization, materialized views; consumption cost risk → auto-suspend, resource monitors, budgets, AI quotas; proprietary native format → Iceberg tables; regional feature gaps → plan region, cross-region inference.

CHAPTER 5

Snowflake Architecture in Depth

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- draw Snowflake's three-layer architecture and explain each layer's responsibilities;
- explain why separating storage and compute changes cost, concurrency and operations;
- describe micro-partitions, columnar storage and the metadata Snowflake keeps about them;
- explain how pruning lets queries skip most of a large table;
- trace a query from submission to result through every layer;
- distinguish the result cache, the warehouse (local disk) cache and metadata-based answers;
- explain warehouse sizing, scaling up versus out, multi-cluster warehouses and the newer warehouse types (Gen2, Snowpark-optimized, interactive, Adaptive);
- explain concurrency, isolation, queuing and locking at a practical level;
- describe fault tolerance and the role of Snowflake Optima;
- read the key numbers in a Query Profile.

Prerequisites: Chapters 2 (compute, storage, elasticity), 3 (SQL, logical query order) and 4 (accounts, warehouses).

Where this leads: Chapter 7 (storage implications of types and table kinds), Chapter 17 (performance optimisation applies everything here), Chapter 18 (cost), Chapter 39 (resilience).

5.1 The big picture: three layers

Concept

Snowflake's architecture has three independently scaling layers:

1. **Database storage** — where table data lives, in Snowflake-managed cloud object storage.
2. **Query processing (compute)** — **virtual warehouses**: clusters of compute resources that execute queries. There can be many, of different sizes, all reading the same data.
3. **Cloud services** — the "brain": authentication, access control, metadata, query parsing and optimisation, transaction management, and the result cache.

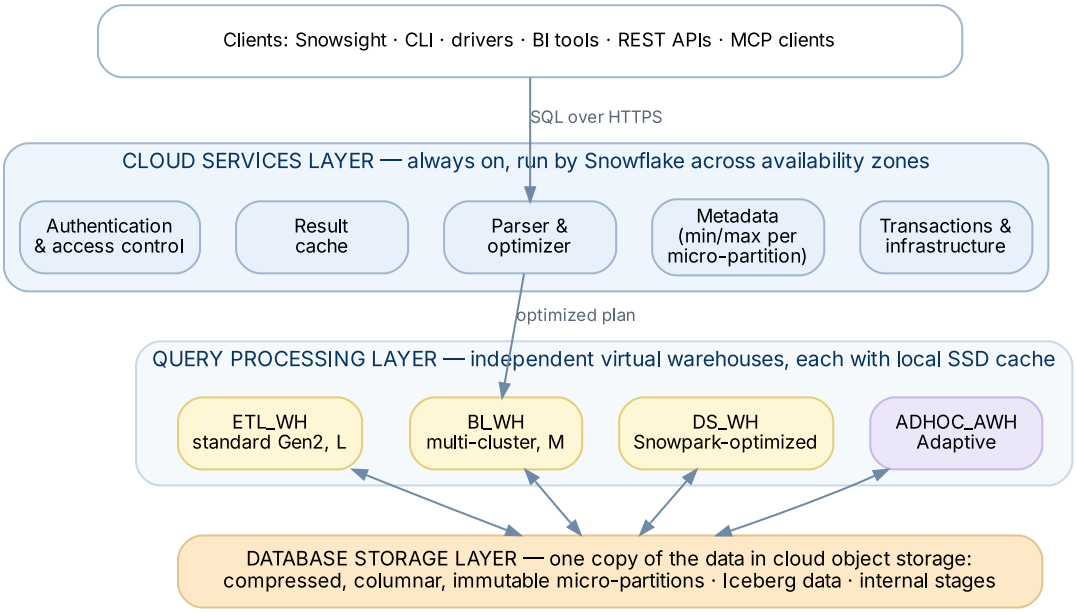


Figure 5.1 — Snowflake's three layers. Many independent warehouses share one copy of the data; cloud services coordinates everything using metadata.

Shared-disk meets shared-nothing

Database architects classify parallel databases in two families:

- **Shared-disk:** all compute nodes access one shared storage system. Simple data management, but the storage can become a bottleneck.
- **Shared-nothing:** each node owns a slice of the data on its own disks. Scales well, but adding nodes means **redistributing data**, and compute cannot be scaled independently of storage.

Snowflake combines them. Like shared-disk, there is **one central copy of the data** accessible to every warehouse. Like shared-nothing, each warehouse is an **MPP (massively parallel processing)** cluster whose nodes process portions of the data in parallel and **cache** what they read on local disks. The result: you can add or remove compute, or create entirely separate warehouses, **without moving or copying data**.

Why separation of storage and compute matters

Benefit	What it means at ByteMart
Independent scaling	50 TB of history does not require a large always-on cluster; compute is sized for the *work*, not the *data*
Workload isolation	The nightly ETL runs on `ETL_WH` and cannot slow analysts' dashboards on `BI_WH`
Pay for compute only when used	Warehouses suspend when idle; storage is billed separately at a much lower rate
Instant elasticity	Resize for a heavy backfill; resize back afterwards
Concurrency without copies	Data science, BI and engineering read the same tables simultaneously
Simplified data sharing	A consumer account's warehouse can query the provider's storage directly (Chapter 21)

5.2 The storage layer

Micro-partitions

When data is loaded into a standard Snowflake table, Snowflake automatically divides it into **micro-partitions**: contiguous units of storage, each containing **between 50 MB and 500 MB of uncompressed data** (smaller once compressed). You never define them; Snowflake creates them as data arrives, generally following the order in which data is inserted.

Inside each micro-partition, data is stored **by column**: all values of `order_date` together, all values of `amount` together, and so on. Each column is **compressed independently**, with Snowflake choosing a suitable compression scheme per column per micro-partition.

Columnar storage, again

Chapter 1 explained why analytical queries benefit from columnar layout. In Snowflake the effect is direct: a query that references 3 columns of a 60-column table reads roughly 3/60 of the bytes from each micro-partition it needs. This is why `SELECT *` is expensive in Snowflake and why wide tables are not a problem in themselves.

Metadata about every micro-partition

For every micro-partition, Snowflake records metadata in the cloud services layer, including for each column:

- the **range of values** (minimum and maximum);
- the **number of distinct values**;
- additional properties used for optimisation (for example, counts of NULLs).

This metadata is small, always available, and is the key to performance: it lets Snowflake decide **which micro-partitions it does not need to read at all**.

Immutability and DML

Snowflake does not edit a micro-partition in place. When you `UPDATE`, `DELETE` or `MERGE`, Snowflake writes **new** micro-partitions containing the changed rows and marks the old ones as no longer current. The old micro-partitions are retained for the **Time Travel** and **Fail-safe** periods (Chapter 8) — which is how Snowflake can show you a table "as it was an hour ago", and why frequent small updates on large tables increase storage.

Some operations are metadata-only: deleting *all* rows from a table, or dropping a column (the column's data is not rewritten; it stays in existing micro-partitions until they are rewritten for other reasons).

5.3 Pruning: how Snowflake skips data

Concept

Pruning is the process of using micro-partition metadata to eliminate micro-partitions that cannot contain rows satisfying a query's filter. It happens in two stages: first unneeded micro-partitions are skipped, then within the remaining micro-partitions only the referenced columns are read.

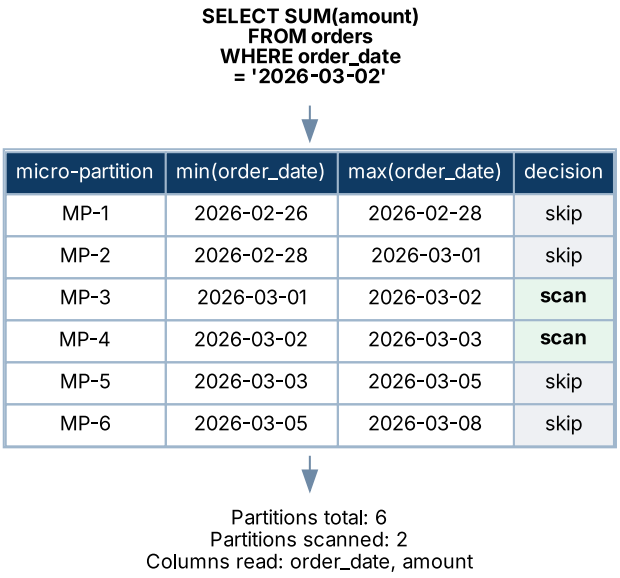


Figure 5.2 — Pruning: for order_date = '2026-03-02', only micro-partitions whose min–max range includes that date are scanned.

Why load order matters: natural clustering

Pruning works well when values of the filtered column are **clustered** — that is, rows with similar values sit in the same micro-partitions, so each micro-partition covers a narrow range. ByteMart loads orders every night in time order, so `order_date` is naturally well clustered: each micro-partition holds a few days of orders. A filter on `order_date` prunes beautifully.

A filter on `customer_id` is different: every night's load contains customers from the whole range, so every micro-partition's min–max range for `customer_id` spans nearly all customers, and almost nothing can be pruned.

Clustering depth

Snowflake measures clustering for a set of columns with **clustering depth**: the average depth of overlapping micro-partitions for those columns. A depth near 1 means little overlap (excellent pruning); larger values mean more overlap. An empty table has depth 0. You can inspect it with `SYSTEM$CLUSTERING_INFORMATION` (Chapter 17), and, for very large tables where queries filter on a column that is not naturally clustered, you can define a **clustering key** and let Snowflake's **automatic clustering** service reorganise micro-partitions in the background — at a cost in serverless credits.

PRUNING LIMITATIONS TO REMEMBER

- Predicates that contain a subquery are not used for pruning, even when the subquery returns a constant. Compute the value first or use a join that the optimizer can prune at run time.
- Wrapping the filtered column in a function (for example `TO_CHAR(order_date, 'YYYY-MM') = '2026-03'`) can prevent effective pruning. Filter on the raw column with a range instead: `order_date >= '2026-03-01' AND order_date < '2026-04-01'`. (Snowflake Optima Metadata can recover some expression-based cases on Gen2 warehouses — see Section 5.10 — but do not design for it.)
- Pruning only helps when the data is clustered on the filtered column.

5.4 The compute layer: virtual warehouses

Concept

A **virtual warehouse** is a named cluster of compute resources (CPU, memory, temporary local storage) that executes queries, DML and loading. Despite the name, a warehouse stores no data — think of it as a **rented engine**, not a building.

Important properties:

Property	Meaning	Typical setting
`WAREHOUSE_TYPE`	`STANDARD`, `'SNOWPARK-OPTIMIZED'` or `ADAPTIVE` (interactive warehouses are created with `CREATE INTERACTIVE WAREHOUSE`)	Standard for most SQL
`WAREHOUSE_SIZE`	XSMALL ... X6LARGE for standard; each step doubles compute and credit rate	Start small, measure, adjust
`GENERATION`	`'1'` or `'2'` for standard warehouses	Gen2 where available
`AUTO_SUSPEND`	Seconds of inactivity before suspending	60–300 for most; longer for latency-sensitive BI
`AUTO_RESUME`	Start automatically when a query arrives	`TRUE`
`MIN_CLUSTER_COUNT`, `MAX_CLUSTER_COUNT`, `SCALING_POLICY`	Multi-cluster settings (Enterprise)	See Section 5.7
`MAX_CONCURRENCY_LEVEL`	Target number of concurrent statements per cluster before queuing (default 8)	Usually leave default
`STATEMENT_TIMEOUT_IN_SECONDS`, `STATEMENT_QUEUED_TIMEOUT_IN_SECONDS`	Cancel runaway or long-queued statements	Set for safety
`RESOURCE_MONITOR`	Credit quota with notify/suspend actions	Always for production

Lifecycle and billing

A warehouse is **started** (resumed), **running**, or **suspended**. While running it consumes credits per second, with a **60-second minimum each time it starts**. When suspended it consumes nothing — and loses (or reduces) its local disk cache.

- **Resizing** a running warehouse takes effect for new queries; queries already running finish on the existing resources. Resizing up does not speed up queries already in progress.
- Suspending a warehouse waits for running statements to finish.

Warehouse types

Type	What it is	Use it for
Standard Gen1	The original warehouse generation	Existing workloads; sizes not supported by Gen2 (5X-Large, 6X-Large)
Standard Gen2	Newer hardware and software optimisations for analytics and data engineering; required for Snowflake Optima	Most new standard warehouses; becoming the default generation (bundle 2026_03)
Snowpark-optimized	More memory per node (configurable resource constraints)	Memory-heavy Python/Java workloads, ML training on a single node
Interactive	Low-latency, high-concurrency engine; queries limited to 5 seconds (longer ones re-run on a fallback warehouse); minimum auto-suspend 24 hours; works best with interactive tables	Real-time dashboards, data-powered APIs
Adaptive (GA, Enterprise+, select regions)	No size to choose; Snowflake allocates compute per query from an account-dedicated pool, bounded by `MAX_QUERY_PERFORMANCE_LEVEL` and `QUERY_THROUGHPUT_MULTIPLIER`; per-query billing	Mixed and variable workloads where you prefer Snowflake to handle sizing, scaling, QAS and suspend/resume

```
-- Standard Gen2 warehouse for transformations
CREATE WAREHOUSE IF NOT EXISTS etl_wh
  WAREHOUSE_SIZE = 'MEDIUM'
  GENERATION = '2'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE
  STATEMENT_TIMEOUT_IN_SECONDS = 3600;

-- Adaptive warehouse with default limits (Enterprise+, supported regions)
CREATE ADAPTIVE WAREHOUSE IF NOT EXISTS analytics_awh;

-- Adaptive warehouse with explicit bounds
CREATE ADAPTIVE WAREHOUSE IF NOT EXISTS finance_awh
  WITH MAX_QUERY_PERFORMANCE_LEVEL = LARGE
       QUERY_THROUGHPUT_MULTIPLIER = 4;
```

Line by line: `GENERATION = '2'` requests a Gen2 standard warehouse (omit to accept the account/region default); `STATEMENT_TIMEOUT_IN_SECONDS = 3600` cancels any statement running longer than an hour — a cheap insurance policy against runaway queries. For the Adaptive examples, defaults are `MAX_QUERY_PERFORMANCE_LEVEL = XLARGE` and `QUERY_THROUGHPUT_MULTIPLIER = 2`; size, cluster counts and scaling policy cannot be set on an Adaptive Warehouse.

> **FEATURE STATUS**
> Gen2 standard warehouses: GA in most regions (not for X5LARGE/X6LARGE). Gen2-as-default: behaviour change 2250 in bundle 2026_03 — check the bundle's status in your account. Adaptive Compute: GA for Enterprise Edition or higher in select AWS, Azure and GCP regions. Interactive warehouses: documented; the overview page did not state a release status at the research cutoff — check before production use.

5.5 The cloud services layer

The cloud services layer is a collection of always-on services, run by Snowflake across availability zones, that coordinate everything:

Service	Responsibilities
Authentication	Passwords, MFA, key-pair, SSO/SAML, OAuth, workload identity federation
Access control	Evaluates roles and privileges for every statement; policies (masking, row access) are applied during query planning
Metadata management	Micro-partition statistics, object definitions, Time Travel versions, clustering information
Query compilation and optimisation	Parsing, name resolution, cost-based optimisation, pruning decisions
Transaction management	Snapshot isolation of committed versions, locks for DML
Infrastructure management	Provisioning and health of warehouses and storage
Result cache	Persisted query results

Cloud services compute is billed in credits **only for the portion exceeding 10 % of the day's warehouse compute** (Chapter 2). Workloads that run enormous numbers of tiny queries or metadata operations can exceed this allowance.

5.6 The query lifecycle

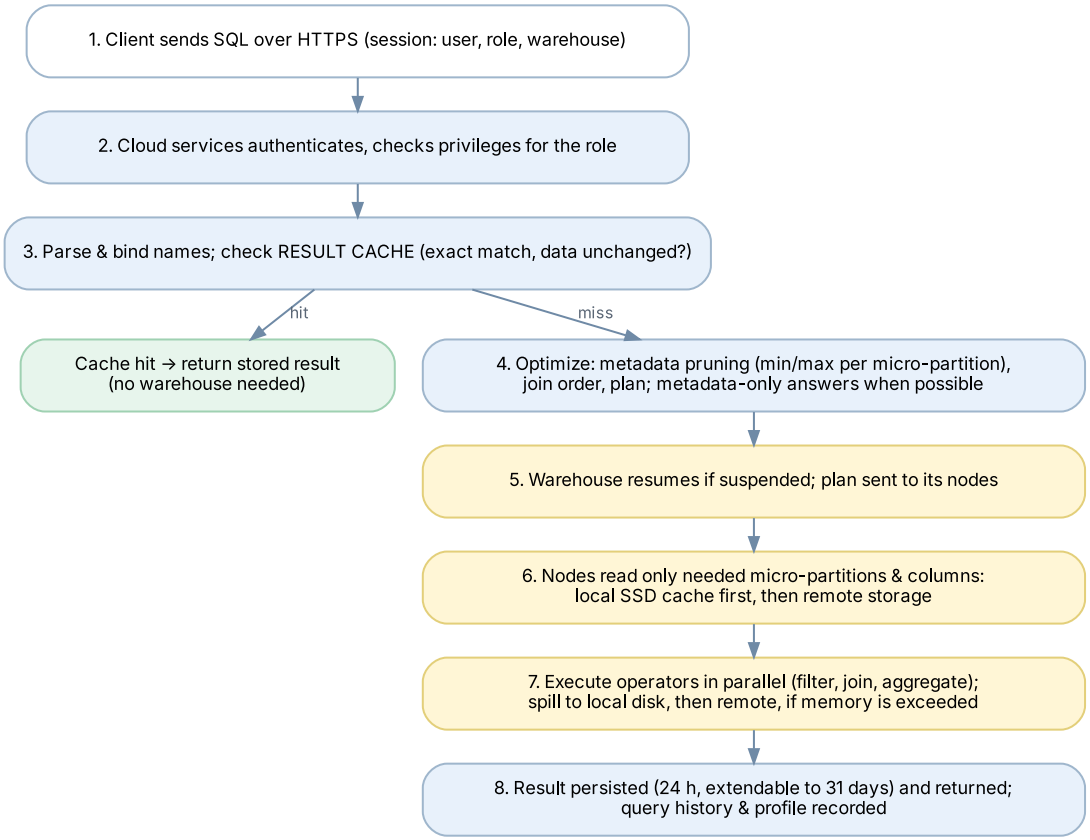


Figure 5.3 — The life of a query. Steps 1–4 and 8 happen in cloud services; steps 5–7 in the warehouse.

Let us trace a ByteMart dashboard query: *revenue by country for March 2026*.

1. **Submission.** Power BI sends the SQL over HTTPS using a session that has a user, a current role (`BI_READER`) and a current warehouse (`BI_WH`).
2. **Authentication and authorisation.** Cloud services verifies the session and checks that `BI_READER` has `SELECT` on the tables and `USAGE` on the database, schema and warehouse. If a masking policy applies to a column, it will be woven into the plan.
3. **Parsing and result-cache check.** The SQL is parsed and names are resolved. If an *identical* query ran recently and its underlying data has not changed (Section 5.8), the stored result is returned immediately — no warehouse required.
4. **Optimisation.** The cost-based optimizer chooses join order and algorithms, pushes filters down, and uses micro-partition metadata to **prune** — for March 2026, only micro-partitions whose `order_date` range overlaps March are kept. Some queries can be answered from metadata alone (`SELECT COUNT(*) FROM orders` needs no data scan).
5. **Dispatch.** If `BI_WH` is suspended and auto-resume is on, it starts (typically within seconds). The plan is sent to the warehouse's nodes.
6. **Scanning.** Each node reads its share of the remaining micro-partitions — only the needed columns — first checking its **local SSD cache**, then fetching the rest from remote storage.

7. **Execution.** Operators run in parallel across nodes: filter, join, aggregate, sort. If intermediate data does not fit in memory, it **spills** to local disk and then to remote storage — a major cause of slowness (Chapter 17).
8. **Result.** The final result is returned to the client and **persisted** in the result cache. Query history records timings, bytes scanned, partitions scanned versus total, spill, queuing and more — visible in the **Query Profile**.

5.7 Scaling: up, out, and concurrency

Scaling up (bigger warehouse)

Each size step roughly doubles the compute available to a query. For a large query that parallelises well, doubling the size can roughly halve its runtime at roughly the same credit cost (Chapter 2's worked example). Scaling up helps:

- large scans, joins and aggregations;
- queries that **spill** because they lack memory.

Scaling up does *not* help a query that is waiting in a **queue** behind other queries, nor a query dominated by compilation or by a single-threaded step.

Scaling out (multi-cluster warehouses)

A **multi-cluster warehouse** (Enterprise Edition) consists of up to several clusters of the same size. When many queries arrive at once, additional clusters start so that queries do not queue; when demand drops, clusters shut down.

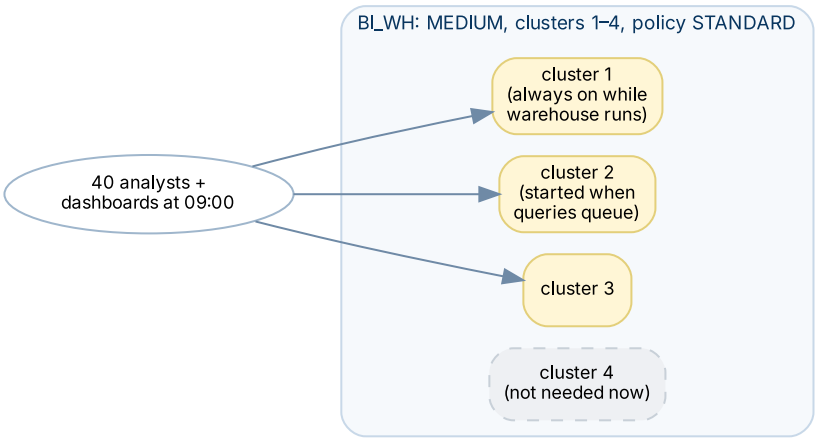


Figure 5.4 — A multi-cluster warehouse in auto-scale mode adds clusters as concurrency rises and removes them as it falls.

```
CREATE WAREHOUSE IF NOT EXISTS bi_wh
  WAREHOUSE_SIZE    = 'MEDIUM'
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 4
  SCALING_POLICY    = 'STANDARD'
```

```
  AUTO_SUSPEND      = 300
  AUTO_RESUME       = TRUE;
```

- **Auto-scale mode:** `MIN_CLUSTER_COUNT` < `MAX_CLUSTER_COUNT`; Snowflake starts and stops clusters automatically.
- **Maximized mode:** `MIN_CLUSTER_COUNT` = `MAX_CLUSTER_COUNT`; all clusters run whenever the warehouse runs — predictable capacity, higher cost.
- **Scaling policy:** `STANDARD` favours starting clusters quickly to avoid queuing; `ECONOMY` favours keeping running clusters fully loaded, starting a new cluster only when Snowflake estimates there is enough work to keep it busy for several minutes — saving credits at the cost of some queuing.

Choosing between up and out

Symptom	Likely cause	Remedy
Single large query is slow; profile shows spilling	Not enough memory per query	Scale **up**
Single large query is slow; profile shows scanning most partitions	Poor pruning	Fix filters/clustering before scaling (Chapter 17)
Many queries slow at 09:00; profile shows high **queued** time	Concurrency	Scale **out** (multi-cluster), or separate workloads
Queries fast individually but dashboards slow	Too many tiny queries; compilation overhead	Result cache, fewer queries, interactive warehouses
Workload highly variable and sizing keeps going wrong	Mixed shapes	Consider an Adaptive Warehouse

Concurrency, queuing and isolation

Each warehouse cluster runs several statements at once. When the resources a new statement needs are not available, it **queues**. `MAX_CONCURRENCY_LEVEL` (default 8) is a target, not a hard limit: Snowflake also considers each statement's estimated resource needs. Queued time appears in query history as `QUEUED_OVERLOAD_TIME`.

Isolation between transactions follows the **READ COMMITTED** level for tables: a statement sees data committed before the statement began. Because micro-partitions are immutable, readers never block writers and writers never block readers — a reader keeps using the micro-partition versions that were current when its statement started. **Writers** do take locks: `UPDATE`, `DELETE` and `MERGE` statements on the same table generally serialise, whereas `INSERT` and `COPY` can usually run concurrently with each other. A blocked statement waits up to `LOCK_TIMEOUT` before failing. Chapter 8 explores transactions further.

5.8 The three kinds of caching

Snowflake avoids repeating work at three levels. Knowing which one helped a query is essential when you benchmark — otherwise you may credit an optimisation that was really a cache hit.

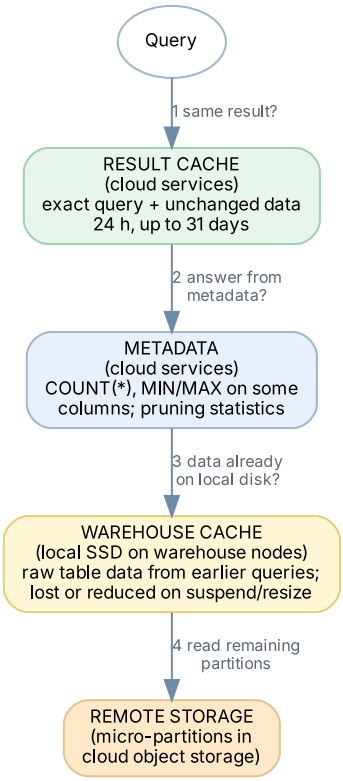


Figure 5.5 — Caching levels checked during a query. Each lower level is slower but more general.

1. Result cache (cloud services)

Snowflake persists the result of every query. A later query can reuse it if — among other conditions documented by Snowflake —

- the new query **exactly matches** the earlier one (differences in case, syntax or aliases prevent full reuse);
- it contains no non-reusable functions (such as `RANDOM`, `UUID_STRING`) and no external functions;
- it does not select from **hybrid tables**;
- the contributing table data has **not changed**, and its micro-partitions have not been reclustered or consolidated;
- the persisted result has not expired;
- the role has the required privileges (for `SHOW` commands, the role must be the same);
- options that affect how the result was produced have not changed.

A persisted result is kept for **24 hours**; each reuse resets the 24 hours, up to **31 days** from the first execution. Result reuse is controlled by the `USE_CACHED_RESULT` parameter (enabled by default, settable at account, user and session level). Meeting all conditions does not *guarantee* reuse.

You can also query a previous result as a table with `RESULT_SCAN` — handy for post-processing the output of `SHOW` or `CALL`:

```
SHOW WAREHOUSES;
SELECT "name", "size", "state", "auto_suspend"
FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
```

Note the double-quoted lowercase column names: `SHOW` output columns are lowercase identifiers.

2. Warehouse cache (local disk)

Each warehouse node keeps recently read table data on local SSD. A later query on the *same warehouse* that needs the same micro-partitions reads them locally instead of from remote storage. This cache:

- is per warehouse — another warehouse does not benefit;
- is lost or reduced when the warehouse **suspends** (and affected when it is resized);
- appears in the Query Profile as **"Percentage scanned from cache"**.

This creates a genuine trade-off for **auto-suspend**: a short auto-suspend saves credits but discards the cache; a longer one keeps the cache warm for frequent dashboard queries but costs idle time. For BI warehouses with steady traffic, a few minutes is often a good compromise; for ETL warehouses, suspend quickly.

3. Metadata-based answers

Because cloud services stores per-micro-partition statistics, some queries — `COUNT(*)` of a table, and `MIN`/`MAX` of certain columns — can be answered **without scanning data**, sometimes without a running warehouse. The Query Profile shows a *metadata-based result* in such cases.

> **BENCHMARK HONESTLY**
>
> When measuring an optimisation, run `ALTER SESSION SET USE_CACHED_RESULT = FALSE;`, use a consistent warehouse, and note whether the warehouse cache was warm. Compare **bytes scanned** and **partitions scanned**, not only elapsed time.

5.9 Fault tolerance and availability

- **Storage durability** comes from the cloud provider's object storage, which replicates data across availability zones.
- **Cloud services** are run by Snowflake redundantly across availability zones within a region.
- **Warehouse nodes** that fail during a query are replaced, and affected work is retried automatically; you may see a query take longer rather than fail.
- **Region-level** failures are not covered automatically. Protection requires **replication** of databases (and, in Business Critical, account failover and client redirect) to another region (Chapter 39).
- **Logical errors** — a bad `DELETE`, a broken pipeline — are covered by Time Travel and, as a last resort, Fail-safe (which only Snowflake can use, on request). Neither replaces proper backup and recovery design (Chapter 39).

5.10 Snowflake Optima: automatic optimisation

Snowflake Optima is a set of automatic optimisations included in all editions at no additional cost, **available only on Gen2 standard warehouses**. It continuously observes recurring workload patterns and applies optimisations without configuration, on a best-effort basis:

- **Optima Indexing** builds hidden indexes for repetitive point-lookup queries, using the Search Optimization Service internally.
- **Optima Metadata** generates extra pruning metadata for frequently used filter expressions such as `UPPER(col) = …` or `LIKE` patterns.
- **Optima Planning** captures execution information and reuses learned plans for recurring queries such as scheduled reports and ELT jobs.

Snowflake's engineering blog (August 2026) also describes an **Optima Clustering** component that reorganises data layout for a specific access pattern.

When a query benefits, the Query Profile and query insights indicate it (for example, that a learned plan was applied). Optima **does not replace** good design: Snowflake explicitly recommends applying the Search Optimization Service directly when you need guaranteed index freshness and consistent performance, and you should continue to follow the practices in Chapter 17.

5.11 Reading the Query Profile

The **Query Profile** (in Snowsight query history, or programmatically via `GET_QUERY_OPERATOR_STATS`) is the window into everything in this chapter. The numbers to look at first:

Metric	Where it comes from	What it tells you
Partitions scanned / Partitions total	Pruning (5.3)	Low ratio = good pruning; near 100 % on a filtered query = pruning problem
Bytes scanned	Storage reads	Work done; compare before/after changes
Percentage scanned from cache	Warehouse cache (5.8)	High = warm cache; beware when benchmarking
Bytes spilled to local / remote storage	Memory pressure (5.6)	Any remote spill is a strong signal to scale up or reduce data
Compilation time vs execution time	Cloud services vs warehouse	High compilation = very complex SQL or many small queries
Queued time	Concurrency (5.7)	Scale out or separate workloads
Most expensive operator nodes	Plan	Where the time actually goes (joins, sorts, aggregations)

5.12 Guided labs

LAB 5.1 — SEE PRUNING

Environment: Snowsight, `lab_role`, `lab_wh` (from Lab 4.1). Uses `SNOWFLAKE_SAMPLE_DATA.TPCH_SF1`.

```
USE ROLE lab_role;
USE WAREHOUSE lab_wh;
ALTER SESSION SET USE_CACHED_RESULT = FALSE;   -- make sure we measure real work

-- Query A: no filter
SELECT COUNT(*), SUM(l_extendedprice)
FROM snowflake_sample_data.tpch_sf1.lineitem;

-- Query B: filter on the ship date
SELECT COUNT(*), SUM(l_extendedprice)
FROM snowflake_sample_data.tpch_sf1.lineitem
WHERE l_shipdate >= DATE '1995-01-01' AND l_shipdate < DATE '1995-02-01';

-- Query C: same date range, hidden behind a function
SELECT COUNT(*), SUM(l_extendedprice)
FROM snowflake_sample_data.tpch_sf1.lineitem
WHERE TO_CHAR(l_shipdate, 'YYYY-MM') = '1995-01';
```

For each query, open the Query Profile and record *Partitions scanned* and *Partitions total*.

What to expect: Query A reads all partitions. Whether Query B prunes depends on how the sample table's micro-partitions happen to be organised on `l_shipdate`; compare it with Query C, where the filter is applied to an expression of the column. Record your actual numbers — the purpose of the lab is to learn to *read* the profile, not to reproduce a specific figure. Query results for B and C should be identical; their work may not be.

LAB 5.2 — OBSERVE THE THREE CACHES

```
USE ROLE lab_role;
USE WAREHOUSE lab_wh;

-- 1. Metadata-based answer
SELECT COUNT(*) FROM snowflake_sample_data.tpch_sf1.orders;

-- 2. Result cache: run the same statement twice with caching enabled
ALTER SESSION SET USE_CACHED_RESULT = TRUE;
SELECT o_orderpriority, COUNT(*) FROM snowflake_sample_data.tpch_sf1.orders GROUP BY 1 ORDER
BY 1;
SELECT o_orderpriority, COUNT(*) FROM snowflake_sample_data.tpch_sf1.orders GROUP BY 1 ORDER
BY 1;

-- 3. Warehouse cache: disable result reuse, run twice on a warm warehouse
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
SELECT o_orderpriority, COUNT(*) FROM snowflake_sample_data.tpch_sf1.orders GROUP BY 1 ORDER
BY 1;
SELECT o_orderpriority, COUNT(*) FROM snowflake_sample_data.tpch_sf1.orders GROUP BY 1 ORDER
BY 1;

-- 4. Cold cache: suspend, resume, run again
ALTER WAREHOUSE lab_wh SUSPEND;
```

```
ALTER WAREHOUSE lab_wh RESUME;
SELECT o_orderpriority, COUNT(*) FROM snowflake_sample_data.tpch_sf1.orders GROUP BY 1 ORDER
BY 1;
```

What to look for: (1) the profile shows a metadata-based result; (2) the second run is served from the result cache; (3) the second run shows a higher *Percentage scanned from cache*; (4) after suspend/resume the percentage drops. If `ALTER WAREHOUSE … SUSPEND` reports the warehouse is already suspended, it auto-suspended — simply continue.

LAB 5.3 — QUERY HISTORY IN SQL (OPTIONAL; NEEDS ACCOUNT_USAGE ACCESS)

Account Usage views live in the shared `SNOWFLAKE` database. Access is granted by an administrator (for example, `GRANT IMPORTED PRIVILEGES ON DATABASE SNOWFLAKE TO ROLE lab_role;` or a narrower `SNOWFLAKE` database role). Data in `ACCOUNT_USAGE.QUERY_HISTORY` can lag by up to about 45 minutes.

```
SELECT query_id,
       LEFT(query_text, 60)            AS query_start,
       warehouse_name,
       total_elapsed_time / 1000       AS elapsed_s,
       partitions_scanned,
       partitions_total,
       percentage_scanned_from_cache,
       bytes_spilled_to_local_storage,
       bytes_spilled_to_remote_storage,
       queued_overload_time
FROM snowflake.account_usage.query_history
WHERE user_name = CURRENT_USER()
  AND start_time > DATEADD(hour, -3, CURRENT_TIMESTAMP())
ORDER BY start_time DESC
LIMIT 20;
```

5.13 Production use case: separating ByteMart's workloads

ByteMart initially ran everything — nightly ELT, dashboards, data-science notebooks — on one Large warehouse that never suspended because something was always running. Symptoms: dashboards slowed at 02:00 when ELT ran; ELT failed when data scientists ran huge joins; the bill was dominated by idle hours.

Redesign, using this chapter's concepts:

Warehouse	Type / size	Settings	Rationale
`ELT_WH`	Standard Gen2, Medium	`AUTO_SUSPEND = 60`, statement timeout 2 h	Isolated, short-lived bursts; suspend immediately
`BI_WH`	Standard Gen2, Small, multi-cluster 1–3	`AUTO_SUSPEND = 300`, `SCALING_POLICY = STANDARD`	Concurrency at 09:00; warm cache for dashboards
`DS_WH`	Snowpark-optimized, Medium	`AUTO_SUSPEND = 120`	Memory-heavy Python feature engineering
`ADHOC_AWH`	Adaptive (pilot)	Default bounds, budget attached	Unpredictable analyst queries without sizing debates

Each warehouse has a resource monitor (Chapter 18). Result: ELT and BI no longer interfere; idle hours drop sharply; per-team cost attribution becomes visible per warehouse.

5.14 Common mistakes and misconceptions

ARCHITECTURE MISCONCEPTIONS

1. **"A bigger warehouse fixes everything."** It does not fix queuing (scale out), poor pruning (fix filters/clustering) or compilation overhead.
2. **"The warehouse holds my data."** It holds only cache; data lives in storage.
3. **"Suspending a warehouse loses data."** It loses only local cache.
4. **"The result cache makes all repeated dashboards free."** Only *identical* queries on *unchanged* data; any data load invalidates reuse for affected tables.
5. **"Snowflake has no partitions to think about."** You don't define micro-partitions, but load order and clustering decide whether pruning works.
6. **"Concurrency is unlimited."** Each cluster runs a limited number of statements; excess queues.
7. **"Optima will tune it for me."** Optima is best-effort and Gen2-only; design still matters.
8. **Benchmarking with caches on** — and drawing wrong conclusions.

5.15 Troubleshooting exercise

EXERCISE — THE 9 A.M. SLOWDOWN

Every weekday between 09:00 and 09:30, ByteMart's dashboards take 40–60 seconds instead of 5. At other times they take 5 seconds. The warehouse is a single-cluster Medium. Query profiles at 09:15 show execution times of about 4 seconds. What is happening, how would you confirm it, and what are two possible fixes with their trade-offs? (Solution at the end of the chapter.)

5.16 Knowledge check

1. Name the three layers of Snowflake's architecture and two responsibilities of each.
2. How does Snowflake combine shared-disk and shared-nothing ideas?
3. What is the size range of a micro-partition, and what metadata enables pruning?
4. Why does an `UPDATE` of one row in a large table not modify an existing micro-partition, and what is the storage consequence?
5. Give two reasons a filter might fail to prune.
6. List four conditions for result-cache reuse.
7. What happens to the warehouse cache when a warehouse suspends? How does this affect the choice of `AUTO_SUSPEND`?
8. When should you scale up, and when out?
9. What isolation level does Snowflake use for tables, and why do readers not block writers?

10. Which warehouse generation is required for Snowflake Optima?
11. Name three settings you cannot set on an Adaptive Warehouse, and the two properties you can.

5.17 Hands-on assignment

1. Complete Labs 5.1 and 5.2. Write a one-page report with your numbers for partitions scanned, bytes scanned and percentage scanned from cache, and explain each in terms of this chapter.
2. Design a warehouse layout for a company with: hourly ELT (heavy joins, 10 minutes), 200 dashboard users concentrated at month-end, three data scientists, and an external partner querying shared data. Specify type, size, multi-cluster settings, auto-suspend and timeouts for each, with justification.
3. Draw the query lifecycle for a query that is answered from the result cache, and for one that spills to remote storage.

5.18 Chapter summary

Snowflake separates storage, compute and cloud services. Data lives once, as compressed, columnar, immutable micro-partitions in cloud object storage, with metadata about each micro-partition held in cloud services. Virtual warehouses are independent MPP compute clusters that read the shared data, cache it locally, and can be resized, multiplied or suspended in seconds; newer types — Gen2, Snowpark-optimized, interactive and Adaptive — specialise for different workloads. Cloud services authenticates, authorises, optimises using metadata, manages transactions and caches results. Performance depends first on pruning, which depends on data being clustered on the filtered columns, then on caching and on matching compute to the workload: scale up for large queries, out for concurrency. Readers never block writers thanks to immutable micro-partitions. Snowflake Optima adds best-effort automatic optimisation on Gen2. The Query Profile exposes all of this, and learning to read it is the foundation of performance work.

5.19 Key takeaways

- Storage and compute scale independently; isolate workloads with separate warehouses.
- Pruning is the most important performance mechanism; write filters that can prune.
- Three caches: result, warehouse (local disk), metadata — know which one you are measuring.
- Scale up for big queries and spills; scale out for queuing.
- Auto-suspend balances idle cost against cache warmth.
- Read the Query Profile before changing anything.

5.20 Further reading

- Key concepts and architecture: https://docs.snowflake.com/en/user-guide/intro-key-concepts
- Micro-partitions and data clustering: https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions
- Overview of warehouses: https://docs.snowflake.com/en/user-guide/warehouses-overview

- Multi-cluster warehouses: https://docs.snowflake.com/en/user-guide/warehouses-multicluster
- Gen2 standard warehouses: https://docs.snowflake.com/en/user-guide/warehouses-gen2
- Adaptive Compute: https://docs.snowflake.com/en/user-guide/warehouses-adaptive
- Interactive tables and warehouses: https://docs.snowflake.com/en/user-guide/interactive
- Using persisted query results: https://docs.snowflake.com/en/user-guide/querying-persisted-results
- Snowflake Optima: https://docs.snowflake.com/en/user-guide/snowflake-optima
- Transactions: https://docs.snowflake.com/en/sql-reference/transactions

5.21 Solutions

Troubleshooting — the 9 a.m. slowdown. Execution time is still about 4 seconds, so the queries themselves are not slower; the extra 35–55 seconds is spent **waiting in the queue**. At 09:00 many users open dashboards at once and the single cluster cannot run them all concurrently. *Confirm* by checking `QUEUED_OVERLOAD_TIME` in query history (or "Queued" time in profiles) for that window, and warehouse load history. *Fixes:* (1) make `BI_WH` a **multi-cluster** warehouse (for example 1–3 clusters, `STANDARD` policy) — removes queuing; costs more only during the peak; requires Enterprise; (2) reduce the number of queries — dashboards often fire many similar queries; consolidating them, or relying on the result cache for identical tiles, reduces load; (3) consider an **Adaptive Warehouse** with a suitable `QUERY_THROUGHPUT_MULTIPLIER`; (4) scaling *up* would make each query faster and so drain the queue somewhat sooner, but is a less targeted fix for concurrency.

Knowledge check answers.

1. Storage — holds micro-partitions, compression, durability; compute (warehouses) — executes queries, caches data locally; cloud services — authentication/access control, metadata, optimisation, transactions, result cache.
2. One central copy of data accessible to all warehouses (shared-disk) with MPP warehouses whose nodes process data in parallel and cache locally (shared-nothing style), so compute scales without redistributing data.
3. 50–500 MB uncompressed. Per-column min/max ranges and distinct counts (and other properties) per micro-partition.
4. Micro-partitions are not modified in place; DML writes new micro-partitions. The old versions are retained for Time Travel and Fail-safe, increasing storage for heavily updated tables.
5. Data is not clustered on the filtered column; the filter applies a function to the column; the predicate contains a subquery.
6. Exact query match; no non-reusable or external functions; no hybrid tables; underlying data unchanged; result not expired; required privileges; unchanged configuration options.
7. It is lost or reduced. Short auto-suspend saves idle credits but loses cache; longer auto-suspend keeps cache warm for frequent queries at the cost of idle time.
8. Up for large/complex queries and spilling; out (multi-cluster) for many concurrent queries that queue.
9. READ COMMITTED. Readers use immutable micro-partition versions current when their statement started, so writers create new versions without blocking readers.
10. Gen2 standard warehouses.

11. Cannot set: `WAREHOUSE_SIZE`, `MIN_CLUSTER_COUNT`/`MAX_CLUSTER_COUNT`, `SCALING_POLICY` (also QAS and suspend/resume policies are managed for you). Can set: `MAX_QUERY_PERFORMANCE_LEVEL`, `QUERY_THROUGHPUT_MULTIPLIER`.

CHAPTER 6

Snowflake Account and Object Fundamentals

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- describe the organization → account → database → schema → object hierarchy;
- name the main account-level and schema-level object types and state what each is for;
- create databases, schemas, tables, views, stages, file formats, warehouses, sequences, streams, tasks, pipes, dynamic tables, materialized views, UDFs, procedures, tags and policies;
- explain users, roles, privileges, ownership and the system roles;
- follow identifier and naming rules and design a naming convention;
- reason about object dependencies and the object lifecycle (CREATE, ALTER, DROP, UNDROP, REPLACE, CREATE OR ALTER);
- build ByteMart's lab object skeleton with a least-privilege role.

Prerequisites: Chapters 3–5.

Where this leads: each object introduced here gets a dedicated deep dive later — data types and table kinds (Chapter 7), SQL (Chapter 8), procedures and UDFs (Chapter 10), stages and loading (Chapter 12), pipes (Chapter 13), streams (Chapter 14), tasks and dynamic tables (Chapter 15), roles (Chapter 20), tags and policies (Chapter 21).

6.1 The object hierarchy

Concept

Everything you create in Snowflake is an **object**, and objects live in a strict hierarchy. Knowing the hierarchy tells you how to name an object, who can see it, and what happens when you drop its container.

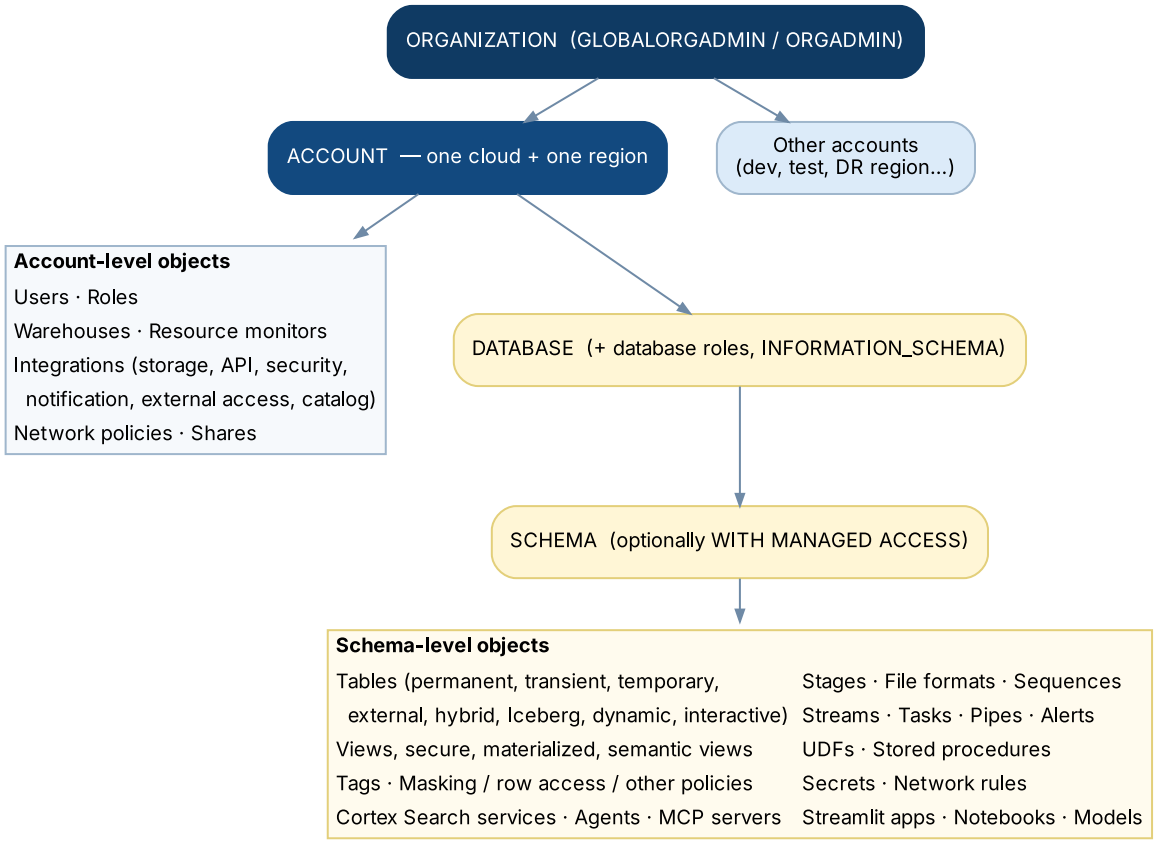


Figure 6.1 — Snowflake's object hierarchy. Account-level objects are named once per account; schema-level objects are fully qualified as database.schema.object.

Organization

An **organization** groups the accounts a company owns. Organization-level administration — creating accounts, viewing organization-wide usage, enabling replication between accounts — is performed by organization administrators. Snowflake now provides a dedicated **organization account** with the **GLOBALORGADMIN** role for this; the older **ORGADMIN** role, used from a regular account, is planned to be phased out in favour of GLOBALORGADMIN.

Account

An **account** is the unit you log in to; it lives in one cloud region (Chapter 4). **Account-level objects** are not inside any database:

Object	Purpose	Chapter
User	A person or service identity that can authenticate	20
Role (account role)	A named set of privileges, granted to users and other roles	6, 20
Warehouse	Compute	5, 17
Resource monitor	Credit quota with notify/suspend actions	18
Integration	Trusted connection to an external system: storage, API, security (SSO/OAuth), notification, external access, catalog	12, 20

Object	Purpose	Chapter
Network policy	Allowed/blocked network sources	20
Share	Live data shared to other accounts	21
Database	Container for schemas	here

Database and schema

A **database** is a logical container for **schemas**; a **schema** is a container for **schema-level objects**. Each new database automatically contains two schemas: `PUBLIC` (an empty default schema) and `INFORMATION_SCHEMA` (read-only views describing the database's objects).

Objects are referred to by their **fully qualified name**: `database.schema.object`. If you set a current database and schema with `USE`, you can use shorter names.

```
USE DATABASE bytemart_lab;
USE SCHEMA sales;                -- or: USE SCHEMA bytemart_lab.sales;
SELECT * FROM orders;            -- resolves to bytemart_lab.sales.orders
SELECT * FROM bytemart_lab.sales.orders;  -- fully qualified, works anywhere
```

FULLY QUALIFY NAMES IN PRODUCTION CODE

Pipelines, tasks, procedures and views should use fully qualified names (or be created in a well-defined context). Code that depends on the session's current schema works on your laptop and fails in the scheduler.

6.2 Schema-level objects at a glance

Category	Objects	What they do
Data	Tables — permanent, transient, temporary, external, hybrid, Iceberg, dynamic, interactive	Store or expose data (Chapter 7)
Logic over data	Views, secure views, materialized views, semantic views	Saved queries; precomputed results; business semantics
Files	Stages (internal, external), file formats	Where files are and how to parse them (Chapter 12)
Identifiers	Sequences	Generate unique numbers
Pipelines	Pipes, streams, tasks, alerts	Continuous loading, change tracking, scheduling, alerting (Chapters 13–15, 19)
Code	User-defined functions (UDFs, UDTFs), stored procedures	Custom logic in SQL, Python, Java, JavaScript, Scala (Chapter 10)
Governance	Tags, masking policies, row access policies, aggregation/projection policies, data metric functions	Classify and protect data (Chapter 21)
Security	Secrets, network rules	Credentials and network definitions for integrations (Chapter 20)
AI & apps	Cortex Search services, agents, MCP servers, Streamlit apps, notebooks, models	AI and application components (Parts VII–VIII)

6.3 Identifiers and naming conventions

Identifier rules

- **Unquoted identifiers** must start with a letter or underscore and contain only letters, digits, underscores and dollar signs. They are **stored and resolved in uppercase**: `orders`, `Orders` and `ORDERS` are the same object `ORDERS`.
- **Double-quoted identifiers** (`"Order Details"`) are **case-sensitive** and may contain spaces and special characters — but must then always be written exactly, with quotes.
- Maximum length: **255 characters**.
- The session parameter `QUOTED_IDENTIFIERS_IGNORE_CASE` (default `FALSE`) makes quoted identifiers case-insensitive; Snowflake recommends choosing a setting early and enforcing it account-wide, because changing it later can make mixed-case objects unreachable.

> **COMMON MISTAKE: MIXED-CASE QUOTED NAMES FROM TOOLS**
> Some tools (and pandas DataFrames written with certain connectors) create columns like `"customerId"`. Afterwards, `SELECT customerId FROM t` fails because Snowflake looks for `CUSTOMERID`. Prefer unquoted, upper/snake-case names; normalise column names on ingestion.

A naming convention for ByteMart

Conventions matter more than the specific choice. ByteMart adopts:

Element	Convention	Example
Databases	`<domain>_<layer>` or `<layer>` per environment account	`RAW`, `ANALYTICS`
Schemas	source system or subject area	`RAW.POSTGRES_SHOP`, `ANALYTICS.SALES`
Tables	snake_case nouns; `fct_`/`dim_` for models	`fct_order_lines`, `dim_customer`
Views	`v_` prefix only if needed to distinguish	`v_customer_current`
Warehouses	`<workload>_WH`	`ELT_WH`, `BI_WH`
Roles	functional `<team>_<function>`; access `<db>_<schema>_<RO/RW>`	`ANALYST`, `ANALYTICS_SALES_RO`
Tasks / streams	`<target>_task`, `<source>_stream`	`fct_orders_task`, `raw_orders_stream`
Policies & tags	purpose-based	`mask_email`, `rap_country`, `pii_type`

Environment separation (dev/test/prod) is usually by **separate accounts** or **separate databases**; Chapter 38 compares the options.

6.4 Users, roles, privileges and ownership

Concept

Snowflake combines two access-control models:

- **Role-based access control (RBAC):** privileges are granted to **roles**; roles are granted to **users** (and to other roles). A user acts through one **primary role** at a time, optionally with **secondary roles** active.
- **Discretionary access control (DAC):** every object has exactly **one owner role**, by default the role that created it. The owner has all privileges on the object and can grant them to others (except in managed access schemas).

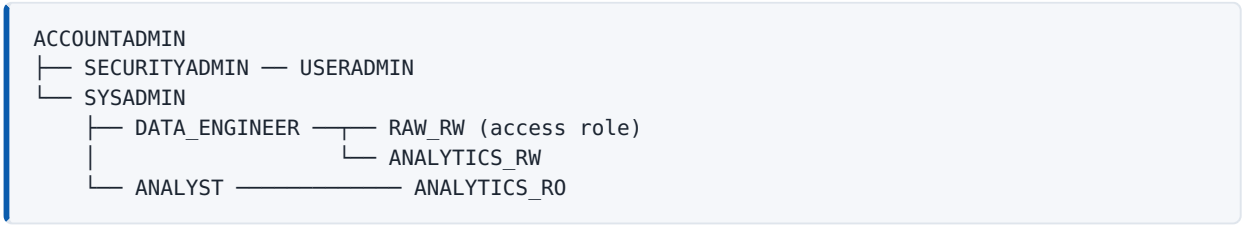
A **privilege** is a specific permission on a specific object: `SELECT` on a table, `USAGE` on a warehouse or schema, `CREATE TABLE` on a schema, `OPERATE` on a warehouse, `OWNERSHIP` on anything.

System roles

Role	Purpose
GLOBALORGADMIN	Organization-level administration from the organization account
ORGADMIN	Organization-level operations from a regular account (being phased out in favour of GLOBALORGADMIN)
ACCOUNTADMIN	Top-level account role; encapsulates SYSADMIN and SECURITYADMIN. Grant to very few people; never use for daily work
SECURITYADMIN	Holds `MANAGE GRANTS` (can modify any grant); manages users and roles; inherits USERADMIN
USERADMIN	Creates users and roles
SYSADMIN	Creates warehouses, databases and other objects; should sit at the top of your custom role hierarchy
PUBLIC	Granted automatically to everyone; anything it owns or is granted is available to all

The recommended hierarchy

Snowflake recommends building a hierarchy of **custom roles** and granting the top-most custom role(s) to **SYSADMIN**, so that system administrators can manage every object while user/role management stays with USERADMIN. A custom role *not* connected to SYSADMIN can only be managed by roles with `MANAGE GRANTS`.



This *functional role → access role* pattern (users get functional roles; functional roles inherit access roles that hold object privileges) is developed fully in Chapter 20.

Database roles

A **database role** is scoped to one database. It cannot be activated directly in a session; it is granted to account roles. Database roles are ideal for packaging privileges that travel with a database — for example, when sharing a database (Chapter 21). Snowflake itself uses database roles in the `SNOWFLAKE` database, such as `SNOWFLAKE.CORTEX_USER` and `SNOWFLAKE.AI_FUNCTIONS_USER`.

Managed access schemas and future grants

- In a **managed access schema** (`CREATE SCHEMA … WITH MANAGED ACCESS`), object owners can no longer grant privileges on their objects; only the schema owner or a role with `MANAGE GRANTS` can. This centralises grant decisions — a governance best practice.
- **Future grants** define privileges for objects created later: `GRANT SELECT ON FUTURE TABLES IN SCHEMA analytics.sales TO ROLE analytics_ro;`. Existing objects still need ordinary grants.

6.5 Core object types in practice

The examples below build ByteMart objects. Each is a short preview; later chapters give the full option lists. All statements assume `USE ROLE lab_role; USE WAREHOUSE lab_wh;` and the database from Lab 4.1.

Databases and schemas

```
CREATE DATABASE IF NOT EXISTS bytemart_lab
  DATA_RETENTION_TIME_IN_DAYS = 1
  COMMENT = 'Learning labs';

CREATE SCHEMA IF NOT EXISTS bytemart_lab.raw;
CREATE SCHEMA IF NOT EXISTS bytemart_lab.silver;
CREATE SCHEMA IF NOT EXISTS bytemart_lab.gold WITH MANAGED ACCESS;
```

`DATA_RETENTION_TIME_IN_DAYS` sets the default Time Travel period for objects created inside (Chapter 8). The `gold` schema uses managed access because it is consumed by many teams.

Tables

```
CREATE OR REPLACE TABLE bytemart_lab.silver.customers (
    customer_id   NUMBER(38,0)  NOT NULL,
    name          VARCHAR(100)  NOT NULL,
    email         VARCHAR(200),
    country       VARCHAR(2)    NOT NULL,
    signup_date   DATE,
    loyalty_tier  VARCHAR(10),
    CONSTRAINT pk_customers PRIMARY KEY (customer_id)   -- informational on standard tables
)
COMMENT = 'One row per customer, current state';
```

Chapter 7 covers table kinds (permanent, transient, temporary, external, hybrid, Iceberg, dynamic) and data types.

Views

A **view** is a named query. It stores no data; every query against it runs the underlying SQL.

```
CREATE OR REPLACE VIEW bytemart_lab.gold.v_customers_by_country AS
SELECT country, COUNT(*) AS customers
FROM bytemart_lab.silver.customers
GROUP BY country;
```

A **secure view** (`CREATE SECURE VIEW`) hides its definition from unauthorised users and prevents certain optimisations that could reveal underlying data; it is required for views exposed through data sharing (Chapter 21). A **materialized view** stores precomputed results and is maintained automatically (Enterprise Edition; a single base table, no joins — Chapter 17). A **semantic view** describes business entities, dimensions, metrics and relationships for BI and AI (Chapter 27).

File formats and stages

```
CREATE OR REPLACE FILE FORMAT bytemart_lab.raw.ff_csv
  TYPE = CSV
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  NULL_IF = ('', 'NULL');

CREATE OR REPLACE STAGE bytemart_lab.raw.landing
  FILE_FORMAT = bytemart_lab.raw.ff_csv
  DIRECTORY = (ENABLE = TRUE)
  COMMENT = 'Internal stage for uploaded files';
```

A **file format** describes how to parse files; a **stage** is a location for files — internal (Snowflake-managed) like this one, or external (your S3/Azure/GCS bucket via a storage integration). `DIRECTORY = (ENABLE = TRUE)` adds a *directory table* listing the stage's files, used for unstructured data (Chapters 12, 25).

Sequences

```
CREATE OR REPLACE SEQUENCE bytemart_lab.silver.seq_customer_sk START = 1 INCREMENT = 1;
SELECT bytemart_lab.silver.seq_customer_sk.NEXTVAL;
```

Sequences generate unique numbers, for example surrogate keys. Snowflake guarantees uniqueness, **not gap-free or strictly ordered** values across concurrent use. Columns can also use `AUTOINCREMENT` / `IDENTITY`, which is backed by a sequence.

Streams and tasks

A **stream** records changes (inserts, updates, deletes) to a table since the stream was last consumed — Snowflake's change data capture mechanism. A **task** runs SQL on a schedule or when triggered.

```
CREATE OR REPLACE TABLE bytemart_lab.raw.customers_landing LIKE
bytemart_lab.silver.customers;

CREATE OR REPLACE STREAM bytemart_lab.raw.customers_landing_stream
  ON TABLE bytemart_lab.raw.customers_landing;
```

```
CREATE OR REPLACE TASK bytemart_lab.raw.merge_customers_task
  WAREHOUSE = lab_wh
  SCHEDULE  = '60 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('bytemart_lab.raw.customers_landing_stream')
AS
  MERGE INTO bytemart_lab.silver.customers t
  USING bytemart_lab.raw.customers_landing_stream s
     ON t.customer_id = s.customer_id
  WHEN MATCHED AND s.METADATA$ACTION = 'INSERT' THEN UPDATE SET
       name = s.name, email = s.email, country = s.country,
       signup_date = s.signup_date, loyalty_tier = s.loyalty_tier
  WHEN NOT MATCHED AND s.METADATA$ACTION = 'INSERT' THEN INSERT
       (customer_id, name, email, country, signup_date, loyalty_tier)
       VALUES (s.customer_id, s.name, s.email, s.country, s.signup_date, s.loyalty_tier);
```

Line by line: `CREATE TABLE … LIKE` copies a table's column definitions. The stream exposes changed rows plus metadata columns such as `METADATA$ACTION`. The task runs hourly, but the `WHEN SYSTEM$STREAM_HAS_DATA(…)` condition skips runs when nothing changed (no warehouse is started for a skipped run). Consuming a stream in a DML statement advances its offset. This simplified MERGE ignores deletes and assumes one change per key per run — Chapter 14 handles both properly. Tasks are created **suspended**; `ALTER TASK … RESUME` starts the schedule (requires the `EXECUTE TASK` account privilege).

Pipes

A **pipe** wraps a `COPY INTO` statement for continuous loading by **Snowpipe**, typically triggered by cloud storage event notifications:

```
-- Requires an external stage on cloud storage with event notifications (Chapter 13)
CREATE OR REPLACE PIPE bytemart_lab.raw.orders_pipe
  AUTO_INGEST = TRUE
AS
  COPY INTO bytemart_lab.raw.orders_landing
  FROM @bytemart_lab.raw.s3_orders_stage
  FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_csv');
```

Shown for recognition only; it cannot run until Chapter 13 creates the external stage and target table. Snowpipe Streaming's high-performance architecture also uses PIPE objects as the entry point for row-level streaming.

Dynamic tables

A **dynamic table** is defined by a query and a **target lag**; Snowflake refreshes it automatically — incrementally where possible — to stay within that lag.

```
CREATE OR REPLACE DYNAMIC TABLE bytemart_lab.gold.customers_by_tier
  TARGET_LAG = '1 hour'
  WAREHOUSE  = lab_wh
AS
SELECT COALESCE(loyalty_tier, 'NONE') AS tier, COUNT(*) AS customers
FROM bytemart_lab.silver.customers
GROUP BY 1;
```

Dynamic tables are often a simpler alternative to streams + tasks for declarative transformations (Chapter 15 compares them).

Functions and procedures

```
-- SQL scalar UDF
CREATE OR REPLACE FUNCTION bytemart_lab.silver.price_incl_gst(price NUMBER(10,2))
  RETURNS NUMBER(12,2)
AS 'price * 1.18';

SELECT bytemart_lab.silver.price_incl_gst(899);

-- Snowflake Scripting stored procedure
CREATE OR REPLACE PROCEDURE bytemart_lab.silver.count_customers(p_country VARCHAR)
  RETURNS NUMBER
  LANGUAGE SQL
AS
$$
DECLARE
  n NUMBER;
BEGIN
  SELECT COUNT(*) INTO :n FROM bytemart_lab.silver.customers WHERE country = :p_country;
  RETURN n;
END;
$$;

CALL bytemart_lab.silver.count_customers('IN');
```

A **UDF** returns a value per row and is used inside queries; a **stored procedure** is called with `CALL` and can run multiple statements, control flow and DDL/DML. Chapter 10 explains when to use which, plus Python, Java and JavaScript handlers.

Tags, masking policies and row access policies

```
CREATE SCHEMA IF NOT EXISTS bytemart_lab.governance;

CREATE OR REPLACE TAG bytemart_lab.governance.pii_type
  ALLOWED_VALUES 'email', 'phone', 'address', 'name';

ALTER TABLE bytemart_lab.silver.customers
  MODIFY COLUMN email SET TAG bytemart_lab.governance.pii_type = 'email';

CREATE OR REPLACE MASKING POLICY bytemart_lab.governance.mask_email
  AS (val STRING) RETURNS STRING ->
  CASE
    WHEN IS_ROLE_IN_SESSION('PII_READER') THEN val
    ELSE REGEXP_REPLACE(val, '^[^@]+', '*****')
  END;

ALTER TABLE bytemart_lab.silver.customers
  MODIFY COLUMN email SET MASKING POLICY bytemart_lab.governance.mask_email;
```

Line by line: a **tag** is a key with optional allowed values attached to objects or columns for classification and cost attribution. A **masking policy** is a function evaluated at query time: here, roles that have `PII_READER` active see the real email; everyone else sees `*****@example.com`. A **row access policy** similarly returns TRUE/FALSE per row to filter what each role sees. Masking and row access policies require **Enterprise Edition**. Chapter 21 covers tag-based masking, policy privileges and testing.

IF APPLYING THE POLICY FAILS WITH AN AUTHORISATION ERROR

Applying a masking policy to a column needs appropriate privileges on both the policy and the table. In a lab, an administrator can grant `APPLY MASKING POLICY ON ACCOUNT` to `lab_role`; in production, use the centralised or hybrid policy-management patterns in Chapter 21.

6.6 How objects work together in a real application

Objects are building blocks; their value appears when combined. Figure 6.2 shows ByteMart's customer pipeline using objects from this chapter.

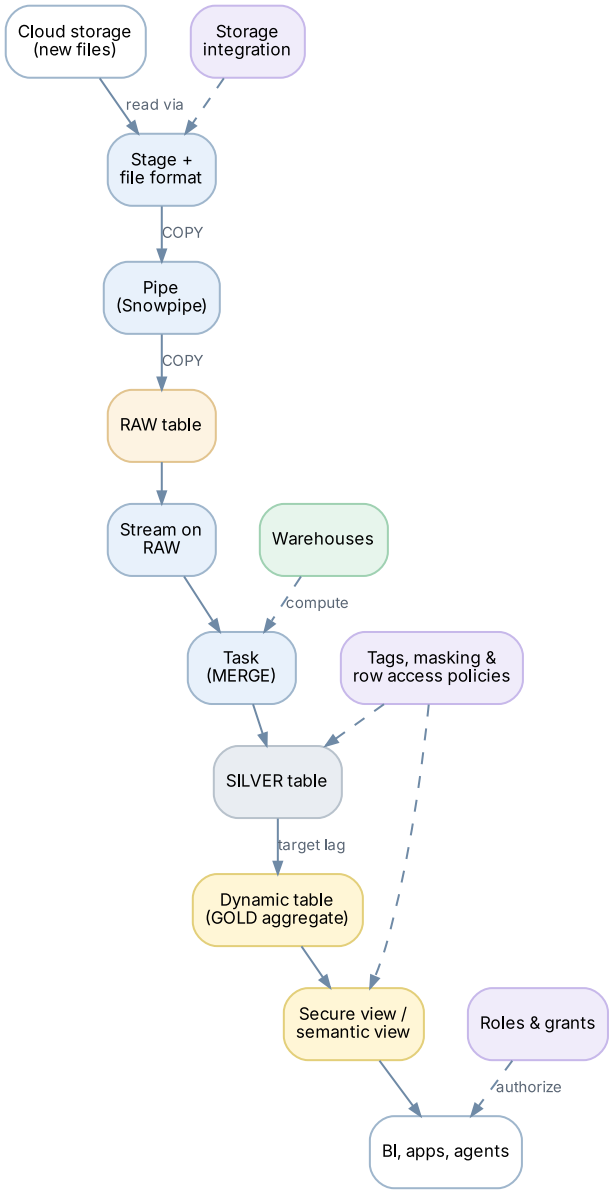


Figure 6.2 — A small ByteMart pipeline expressed as Snowflake objects. Dashed lines show configuration relationships: integrations, policies, compute and authorisation.

Reading it left to right: new files arrive in cloud storage; a **storage integration** lets a **stage** read them; a **pipe** copies them into a **raw table**; a **stream** captures changes; a **task** merges them into a **silver table** protected by **tags** and **policies**; a **dynamic table** maintains a gold aggregate within a target lag; a **secure or semantic view** exposes it; **roles** decide who may query; **warehouses** supply compute at each compute-consuming step.

6.7 Object dependencies

Objects frequently reference other objects: a view references tables; a task references a warehouse, stream and table; a masking policy is attached to columns; a dynamic table references its base tables.

Important behaviours:

- **Views are not automatically updated or invalidated** when underlying tables change. Dropping or renaming a referenced table or column causes the view to fail at query time.
- `CREATE OR REPLACE TABLE` replaces the table with a new object. Streams on the old table become stale and grants may be lost unless copied (`COPY GRANTS` is available for many objects).
- **Dropping a database or schema** drops everything inside it (recoverable with `UNDROP` within Time Travel).
- **Policies** cannot be dropped while attached to columns or tables.

You can query dependencies in the `SNOWFLAKE.ACCOUNT_USAGE.OBJECT_DEPENDENCIES` view (subject to Account Usage latency and access):

```
SELECT referencing_object_name, referencing_object_domain,
       referenced_object_name,  referenced_object_domain
FROM snowflake.account_usage.object_dependencies
WHERE referenced_database = 'BYTEMART_LAB'
ORDER BY referenced_object_name;
```

6.8 Object lifecycle and ownership

Operation	Purpose	Notes
`CREATE`	Create an object	`IF NOT EXISTS` makes scripts re-runnable
`CREATE OR REPLACE`	Drop and recreate atomically	Loses data for tables; use with care; `COPY GRANTS` where supported
`CREATE OR ALTER`	Converge an existing object to the given definition, preserving data where possible	Declarative deployments; check supported object types
`ALTER`	Change properties, rename, add/drop columns, suspend/resume	
`DROP`	Remove the object	
`UNDROP`	Restore a dropped table, schema or database (and certain other objects) within the retention period	Name must not be taken by a new object
`SHOW` / `DESCRIBE`	List objects / show details	Output can be post-processed with `RESULT_SCAN`

Operation	Purpose	Notes
`GRANT OWNERSHIP`	Transfer ownership	`COPY CURRENT GRANTS` keeps existing grants

```
SHOW TABLES IN SCHEMA bytemart_lab.silver;
DESCRIBE TABLE bytemart_lab.silver.customers;
DROP TABLE bytemart_lab.silver.customers;
UNDROP TABLE bytemart_lab.silver.customers;     -- within DATA_RETENTION_TIME_IN_DAYS
```

OWNERSHIP BY PERSONAL OR TEMPORARY ROLES

If objects are owned by a role that belongs to one person and that role is later dropped, ownership problems follow. Production objects should be owned by stable functional or deployment roles, granted under SYSADMIN, and created by automated deployment (Chapters 11 and 38).

6.9 Guided lab: build ByteMart's object skeleton

LAB 6.1 — FROM EMPTY ACCOUNT TO A WORKING MINI-PIPELINE

Environment: Snowsight. **Prerequisite:** Lab 4.1 (`lab_role`, `lab_wh`, `bytemart_lab`). **Edition:** Enterprise for Step 7.

Step 1 — Grant the extra privileges and test roles the lab needs (as an administrator, replacing `<your_user_name>`):

```
USE ROLE ACCOUNTADMIN;
GRANT EXECUTE TASK ON ACCOUNT TO ROLE lab_role;           -- needed to run and resume tasks
GRANT APPLY MASKING POLICY ON ACCOUNT TO ROLE lab_role;   -- lab convenience only

-- Two test roles for the masking experiment in Step 7
CREATE ROLE IF NOT EXISTS analyst_lab;
CREATE ROLE IF NOT EXISTS pii_reader;
GRANT ROLE analyst_lab TO ROLE SYSADMIN;
GRANT ROLE pii_reader  TO ROLE SYSADMIN;
GRANT USAGE ON WAREHOUSE lab_wh TO ROLE analyst_lab;
GRANT USAGE ON WAREHOUSE lab_wh TO ROLE pii_reader;
GRANT ROLE analyst_lab TO USER <your_user_name>;
GRANT ROLE pii_reader  TO USER <your_user_name>;

USE ROLE lab_role;
USE WAREHOUSE lab_wh;
```

Step 2 — Create schemas (Section 6.5, "Databases and schemas").

Step 3 — Create `silver.customers`, the landing table, stream and task (Section 6.5). Then load landing data:

```
INSERT INTO bytemart_lab.raw.customers_landing
  (customer_id, name, email, country, signup_date, loyalty_tier) VALUES
  (1, 'Aarav Sharma', 'aarav@example.com', 'IN', '2024-01-15', 'GOLD'),
  (2, 'Priya Nair',   'priya@example.com', 'IN', '2024-03-02', 'SILVER'),
  (3, 'Oliver Smith', 'oliver@example.co.uk', 'UK', '2024-05-20', NULL);
```

```
SELECT customer_id, name, METADATA$ACTION, METADATA$ISUPDATE
FROM bytemart_lab.raw.customers_landing_stream;
```

Expected: three rows, each with `METADATA$ACTION = 'INSERT'` and `METADATA$ISUPDATE = FALSE`.

Step 4 — Run the task once manually rather than waiting an hour:

```
EXECUTE TASK bytemart_lab.raw.merge_customers_task;
```

Wait a few seconds, then:

```
SELECT * FROM bytemart_lab.silver.customers ORDER BY customer_id;
SELECT name, state, error_message
FROM TABLE(bytemart_lab.INFORMATION_SCHEMA.TASK_HISTORY(
       TASK_NAME => 'MERGE_CUSTOMERS_TASK'))
ORDER BY scheduled_time DESC LIMIT 5;
SELECT COUNT(*) AS pending_changes FROM bytemart_lab.raw.customers_landing_stream;
```

Expected: three customers in `silver.customers`; the latest task run `SUCCEEDED`; the stream now empty (0 pending changes) because the MERGE consumed it.

Step 5 — Create the dynamic table (Section 6.5) and query it after its first refresh:

```
SELECT * FROM bytemart_lab.gold.customers_by_tier ORDER BY tier;
```

Step 6 — Create the UDF and procedure and call them.

Step 7 — Apply the tag and masking policy, then test it from three roles. As the owner (`lab_role`), first give the two test roles read access:

```
USE ROLE lab_role;
GRANT USAGE ON DATABASE bytemart_lab TO ROLE analyst_lab;
GRANT USAGE ON SCHEMA bytemart_lab.silver TO ROLE analyst_lab;
GRANT SELECT ON TABLE bytemart_lab.silver.customers TO ROLE analyst_lab;
GRANT USAGE ON DATABASE bytemart_lab TO ROLE pii_reader;
GRANT USAGE ON SCHEMA bytemart_lab.silver TO ROLE pii_reader;
GRANT SELECT ON TABLE bytemart_lab.silver.customers TO ROLE pii_reader;

-- Make sure only the primary role counts for this experiment
USE SECONDARY ROLES NONE;

USE ROLE analyst_lab;
SELECT customer_id, email FROM bytemart_lab.silver.customers;   -- masked: *****@example.com

USE ROLE pii_reader;
SELECT customer_id, email FROM bytemart_lab.silver.customers;   -- unmasked

USE ROLE lab_role;
SELECT customer_id, email FROM bytemart_lab.silver.customers;   -- masked: owning a table
does not bypass the policy

USE SECONDARY ROLES ALL;
SELECT customer_id, email FROM bytemart_lab.silver.customers;   -- unmasked!
```

Why the last result is unmasked: `IS_ROLE_IN_SESSION('PII_READER')` considers every role active in the session, including **secondary roles**. With `USE SECONDARY ROLES ALL`, all roles granted to your user — including `pii_reader` — become active, so the policy reveals the data. This is a frequent surprise in production and exactly why Chapter 21 teaches systematic policy testing, and why Snowflake recommends disabling secondary roles for sessions such as MCP connections (Chapter 30).

Step 8 — Inspect and clean up:

```
SHOW OBJECTS IN SCHEMA bytemart_lab.silver;
ALTER DYNAMIC TABLE bytemart_lab.gold.customers_by_tier SUSPEND;  -- stop refreshes
ALTER WAREHOUSE lab_wh SUSPEND;
```

Suspending the dynamic table prevents it from consuming credits between labs.

6.10 Production use case: ByteMart's first platform layout

Database	Schemas	Owner (deployment role)	Consumers
`RAW`	`POSTGRES_SHOP`, `ERP`, `CLICKSTREAM`, `DOCS`	`DEPLOY_RAW`	Data engineers only
`ANALYTICS`	`SILVER_SALES`, `SILVER_CUSTOMER`, `GOLD_SALES`, `GOLD_MARKETING` (managed access)	`DEPLOY_ANALYTICS`	Engineers (RW), analysts (RO on gold)
`GOVERNANCE`	`TAGS`, `POLICIES`	`GOVERNANCE_ADMIN`	Referenced by all
`AI`	`SEARCH`, `AGENTS`	`DEPLOY_AI`	Application roles

Warehouses follow Chapter 5's isolation design; roles follow the functional/access pattern; every object is created by CI/CD through deployment roles (Chapter 11); future grants on gold schemas give analysts read access to new models automatically.

6.11 Common mistakes

OBJECT-MANAGEMENT MISTAKES

1. Creating objects as `ACCOUNTADMIN`, so they are owned by it and invisible to others.
2. Custom roles not attached to SYSADMIN.
3. Relying on the current schema in scheduled code.
4. `CREATE OR REPLACE TABLE` in a deployment script for a table holding data.
5. Quoted mixed-case identifiers.
6. Forgetting that tasks start suspended — or forgetting the `EXECUTE TASK` privilege.
7. Leaving dynamic tables and tasks running in dev accounts.
8. Using `PUBLIC` to grant access "just for now".

6.12 Troubleshooting exercise

EXERCISE — 'OBJECT DOES NOT EXIST OR NOT AUTHORIZED'

An analyst using role `ANALYST` runs `SELECT * FROM analytics.gold_sales.fct_orders;` and receives *"Object 'ANALYTICS.GOLD_SALES.FCT_ORDERS' does not exist or not authorized."* The table definitely exists. List every check you would make, in order. (Solution at the end of the chapter.)

6.13 Knowledge check

1. Which objects are account-level and which are schema-level: warehouse, stage, role, sequence, integration, masking policy, share, task?
2. Why does `SELECT * FROM Orders` find a table created as `orders`, but not one created as `"Orders"`?
3. What is the difference between RBAC and DAC in Snowflake?
4. Why should custom role hierarchies roll up to SYSADMIN?
5. What changes in a managed access schema?
6. What do future grants do, and what do they not do?
7. Name two differences between a UDF and a stored procedure.
8. What does `WHEN SYSTEM$STREAM_HAS_DATA(...)` add to a task?
9. What happens to a stream when its source table is replaced with `CREATE OR REPLACE TABLE`?
10. Why should production objects be owned by deployment roles rather than personal roles?

6.14 Hands-on assignment

1. Complete Lab 6.1, including the masking test with a separate role that does not inherit `pii_reader`.
2. Write a naming-convention document for your own organisation covering databases, schemas, tables, roles, warehouses, tasks and policies.
3. Draw the object diagram (like Figure 6.2) for a pipeline that loads product CSVs nightly and publishes a product dimension.

6.15 Chapter summary

Snowflake objects form a hierarchy: organization, account, database, schema, schema-level objects. Account-level objects include users, roles, warehouses, resource monitors, integrations, network policies, shares and databases; schema-level objects include tables of many kinds, views, stages, file formats, sequences, streams, tasks, pipes, dynamic tables, functions, procedures, tags, policies and AI and application objects. Unquoted identifiers are case-insensitive and stored in uppercase; quoted identifiers are case-sensitive. Access combines role-based grants with single-role ownership; system roles separate organisation, account, security, user and

object administration, and custom roles should roll up to SYSADMIN. Objects depend on each other, and life-cycle operations — replace, drop, undrop, ownership transfer — have consequences that production designs must anticipate.

6.16 Key takeaways

- Fully qualify names in production code.
- Own objects with stable roles under SYSADMIN; never work as ACCOUNTADMIN.
- Use managed access and future grants for consumer-facing schemas.
- Prefer unquoted, consistent identifiers.
- Understand what `CREATE OR REPLACE` destroys.

6.17 Further reading

- Overview of access control: https://docs.snowflake.com/en/user-guide/security-access-control-overview
- Identifier requirements: https://docs.snowflake.com/en/sql-reference/identifiers-syntax
- Database, schema and share DDL: https://docs.snowflake.com/en/sql-reference/ddl-database
- Introduction to streams: https://docs.snowflake.com/en/user-guide/streams-intro
- Introduction to tasks: https://docs.snowflake.com/en/user-guide/tasks-intro
- Dynamic tables: https://docs.snowflake.com/en/user-guide/dynamic-tables/overview
- Object tagging: https://docs.snowflake.com/en/user-guide/object-tagging/introduction
- Column-level security (masking): https://docs.snowflake.com/en/user-guide/security-column-intro
- OBJECT_DEPENDENCIES view: https://docs.snowflake.com/en/sql-reference/account-usage/object_dependencies

6.18 Solutions

Troubleshooting — "does not exist or not authorized". Snowflake deliberately gives the same message whether an object is missing or simply not visible, so check: (1) **name and case** — is it really `FCT_ORDERS`, or was it created with quotes (`"fct_orders"`)? Run `SHOW TABLES LIKE '%ORDERS%' IN SCHEMA analytics.gold_sales` as a role that can see it. (2) **Current role** — is the session's primary role actually `ANALYST`, and are secondary roles as expected? `SELECT CURRENT_ROLE(), CURRENT_SECONDARY_ROLES();` (3) **USAGE on the database** `ANALYTICS` for the role. (4) **USAGE on the schema** `GOLD_SALES`. (5) **SELECT on the table** — directly or via an inherited access role; `SHOW GRANTS TO ROLE analyst;` and `SHOW GRANTS ON TABLE …;`. (6) If the table was recently recreated with `CREATE OR REPLACE`, grants may have been lost — check whether `COPY GRANTS` was used and whether future grants exist on the schema. (7) In a managed access schema, confirm the grant was made by the schema owner or a `MANAGE GRANTS` role. (8) A row access policy might return no rows — but that yields an *empty result*, not this error.

Knowledge check answers.

1. Account-level: warehouse, role, integration, share. Schema-level: stage, sequence, masking policy, task.

2. Unquoted `Orders` resolves to `ORDERS`, which matches the unquoted `orders` (stored as `ORDERS`). `"Orders"` was stored exactly as `Orders`, which does not equal `ORDERS`.
3. RBAC: privileges are granted to roles, and roles to users/roles. DAC: each object has one owner role with full control that can grant privileges (outside managed access schemas).
4. So that SYSADMIN (and thus account administrators) can manage all objects created by custom roles, keeping object administration separate from user/role administration.
5. Object owners can no longer grant privileges on their objects; only the schema owner or a role with `MANAGE GRANTS` can.
6. They grant privileges automatically on objects created in the future in a database or schema. They do not grant privileges on existing objects.
7. A UDF returns a value (or table) and is used inside SQL expressions; a procedure is invoked with `CALL`, can execute multiple statements, DDL and control flow, and returns a single value or table.
8. The task run is skipped unless the stream has unconsumed changes, avoiding pointless warehouse starts.
9. The stream on the old table can no longer be used (it becomes stale or invalid); recreate the stream after replacing the table.
10. Personal roles change or are dropped when people move; stable deployment roles keep ownership predictable, grants intact and changes auditable.

CHAPTER 7

Snowflake Data Types and Storage

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- choose correct numeric, string, binary, boolean, date and time types;
- explain TIMESTAMP_NTZ, TIMESTAMP_LTZ and TIMESTAMP_TZ and choose between them;
- store and query semi-structured data with VARIANT, OBJECT and ARRAY, and know when to use structured types instead;
- recognise DECFLOAT, UUID, VECTOR, FILE, GEOGRAPHY/GEOMETRY and user-defined types;
- apply explicit and implicit conversion safely, including `TRY_` functions;
- reason about precision, scale, NULL and collation;
- explain the storage implications of type choices;
- state which constraints Snowflake enforces on which table types;
- choose between permanent, transient, temporary, external, Iceberg, hybrid, dynamic and interactive tables.

Prerequisites: Chapters 3 (SQL), 5 (micro-partitions) and 6 (objects).

Where this leads: Chapter 8 (querying semi-structured data, Time Travel), Chapter 9 (modelling), Chapter 12 (loading into typed and VARIANT columns), Chapter 21 (retention and protection), Chapter 25 (FILE and VECTOR with Cortex AI).

7.1 Why types matter

A **data type** constrains which values a column may hold and determines how they are stored, compared, sorted and computed. Good type choices:

- **prevent bad data** — a `DATE` column rejects `'31/02/2026'`;
- **make computation correct** — money stored as `FLOAT` accumulates rounding errors; `NUMBER(12,2)` does not;
- **enable pruning and performance** — Snowflake's min/max metadata works best on properly typed columns; a date stored as text compares alphabetically;
- **save storage** — integers and dates compress better than equivalent strings.

7.2 Numeric types

NUMBER, DECIMAL and integers

`NUMBER(p, s)` stores exact fixed-point numbers with **precision** `p` (total digits, up to 38) and **scale** `s` (digits after the decimal point, up to 37). The default is `NUMBER(38,0)`.

- `DECIMAL`, `DEC` and `NUMERIC` are synonyms of `NUMBER`.
- `INT`, `INTEGER`, `BIGINT`, `SMALLINT`, `TINYINT` and `BYTEINT` are all synonyms of `NUMBER(38,0)` — unlike many databases, they do **not** differ in size or range.
- Digits to the left of the decimal point must fit in `p - s`: `NUMBER(20,2)` allows 18.

Storage: precision does **not** affect storage; scale **does**. `NUMBER(10,5)` uses more space than `NUMBER(5,0)` for the same value. Snowflake also sizes storage per micro-partition based on each column's minimum and maximum values, then compresses. Practical rule: choose the scale the business needs, and don't worry about declaring `NUMBER(38, s)`.

FLOAT

`FLOAT`, `FLOAT4`, `FLOAT8`, `DOUBLE`, `DOUBLE PRECISION` and `REAL` are all **64-bit IEEE 754 double precision** in Snowflake (about 15 significant digits). Special values `'NaN'`, `'inf'` and `'-inf'` exist. Snowflake differs from IEEE 754 in that `'NaN' = 'NaN'` is TRUE and NaN sorts above every other value, including infinity.

> **NEVER STORE MONEY IN FLOAT**
> Binary floating point cannot represent most decimal fractions exactly: 0.1 + 0.2 is not exactly 0.3. Errors grow in aggregates over millions of rows. Use `NUMBER(p, s)` for currency and quantities that must reconcile.

DECFLOAT

`DECFLOAT` stores numbers **exactly** with up to **38 significant digits** and a **variable base-10 exponent** (from −16383 to 16384). Use it when the scale is unknown or highly variable (ingesting data from systems with arbitrary decimals), for exact ledger and tax computations that overflow fixed scale, or when migrating from IEEE 754 decimal or 128-bit decimal systems.

Limitations at the research cutoff: not supported in Iceberg or hybrid tables, in non-SQL UDFs and procedures, in Snowpark, as clustering keys, with search optimization, differential privacy or sensitive data classification; older drivers return it as text; `NUMBER` and `FLOAT` may perform better. Use string or DECFLOAT literals, not numeric literals, when specifying exact values. The documentation does not label DECFLOAT as Preview.

7.3 String and binary types

`VARCHAR(n)` stores Unicode UTF-8 text. `STRING`, `TEXT`, `CHAR` and `CHARACTER` are synonyms (`CHAR` defaults to length 1). The **default** length is **16,777,216 bytes**; the **maximum** is **134,217,728 bytes** (128 MB).

- The declared length `n` is a **constraint**, not a storage size: a 10-character value occupies the same space in `VARCHAR(10)` and `VARCHAR`. Declare lengths when they express a real rule (country codes = 2) or when downstream tools need them.
- `CHAR(n)` in Snowflake does not pad values with spaces as some databases do.
- String comparison is **case-sensitive** by default.

`BINARY` (`VARBINARY`) stores bytes; values are commonly displayed as hexadecimal.

Collation

Collation specifies how strings compare and sort — case sensitivity, accent sensitivity, language rules. Snowflake supports collation specifications on columns and expressions:

```
CREATE OR REPLACE TABLE bytemart_lab.silver.city_names (
    city VARCHAR COLLATE 'en-ci'   -- English, case-insensitive
);
SELECT 'Bengaluru' = 'BENGALURU' COLLATE 'en-ci' AS equal_ci;   -- TRUE
```

Collation has trade-offs: collated comparisons are slower, and some functions and features do not support collated values. Use it deliberately for user-facing matching, not by default. For case-insensitive matching in a single query, `ILIKE`, `UPPER()` and `LOWER()` are often simpler.

7.4 BOOLEAN

`BOOLEAN` holds `TRUE`, `FALSE` or `NULL`. When converting text to boolean, Snowflake accepts strings such as `'true'`, `'false'`, `'yes'`, `'no'`, `'on'`, `'off'`, `'t'`, `'f'`, `'y'`, `'n'`, `'1'`, `'0'` (case-insensitive). Numbers convert as 0 → FALSE, non-zero → TRUE.

Prefer `BOOLEAN` over `'Y'/'N'` flags: comparisons are unambiguous and filters read naturally (`WHERE is_active`).

7.5 Date and time types

Type	Stores	Example
`DATE`	Calendar date	`2026-03-02`
`TIME(p)`	Time of day, 0–9 fractional digits, no time zone	`14:03:55.123`
`TIMESTAMP_NTZ(p)`	Wall-clock date and time with **no** time zone	`2026-03-02 14:03:55`
`TIMESTAMP_LTZ(p)`	An absolute instant stored in UTC, **displayed and computed in the session time zone**	`2026-03-02 14:03:55 +05:30` in an IST session

Type	Stores	Example
`TIMESTAMP_TZ(p)`	An instant plus the **offset** it was recorded in	`2026-03-02 14:03:55 +05:30`

- Precision `p` ranges from 0 (seconds) to 9 (nanoseconds); default 9.
- `TIMESTAMP` alone maps to one of the three according to the `TIMESTAMP_TYPE_MAPPING` parameter, **default `TIMESTAMP_NTZ`**. `DATETIME` is a synonym for `TIMESTAMP_NTZ`.
- `TIMESTAMP_TZ` stores the **offset**, not the time-zone *name*: `+01:00` does not remember whether it meant Paris in winter or London in summer, so daylight-saving rules are not re-applied later.
- The session's time zone is the `TIMEZONE` parameter (account default `America/Los_Angeles` unless your administrator changed it).
- Snowflake recommends years between 1582 and 9999.

Choosing a timestamp type

Situation	Recommended type	Why
Event times from systems in many zones (orders, clicks)	`TIMESTAMP_TZ`, or `TIMESTAMP_NTZ` **normalised to UTC** by convention	Unambiguous instants; NTZ-in-UTC is common and simple if *everyone* follows the convention
Business-local times that are not instants (store opening time; "local delivery slot")	`TIMESTAMP_NTZ` or `TIME`	They describe local wall-clock time, not a global instant
Values shown to users in their own zone	`TIMESTAMP_LTZ` (or convert at query time)	Displays in the session's zone automatically

THE SILENT TIME-ZONE BUG

If an extract writes IST wall-clock times into `TIMESTAMP_NTZ` and another writes UTC wall-clock times into the same column, the table contains a mix that no query can untangle. Decide a convention per column, document it in the column comment, and convert at ingestion: `CONVERT_TIMEZONE('Asia/Kolkata', 'UTC', ts_ntz)`.

```
ALTER SESSION SET TIMEZONE = 'Asia/Kolkata';
SELECT '2026-03-02 14:03:55 +00:00'::TIMESTAMP_TZ                AS tz_value,
       '2026-03-02 14:03:55 +00:00'::TIMESTAMP_LTZ               AS ltz_shown_in_ist,
       '2026-03-02 14:03:55'::TIMESTAMP_NTZ                      AS ntz_wall_clock,
       CONVERT_TIMEZONE('UTC', 'Asia/Kolkata',
                        '2026-03-02 14:03:55'::TIMESTAMP_NTZ)    AS ntz_utc_to_ist;
```

Expected (with the session in Asia/Kolkata): the TZ value keeps `+0000`; the LTZ value displays as `2026-03-02 19:33:55.000 +0530`; the NTZ value is unchanged; the converted value is `2026-03-02 19:33:55.000`.

7.6 Semi-structured types: VARIANT, OBJECT and ARRAY

Concept

- `VARIANT` can hold any value — a number, string, boolean, object, array, or SQL NULL — and **stores the value's type alongside it**.
- `OBJECT` is a set of key–value pairs (like a JSON object); keys are strings, values are VARIANTs.
- `ARRAY` is an ordered list of VARIANT values.

A VARIANT (and an OBJECT) can be up to **128 MB of uncompressed data**; practical limits are somewhat lower because of internal overhead.

How Snowflake stores VARIANT

When you load JSON into a VARIANT column, Snowflake does not keep a blob of text. For mostly regular JSON using native types (strings, numbers, booleans), Snowflake extracts frequently occurring paths into an internal columnar form, so storage and query performance can be similar to relational columns. Values that are not native JSON types — **dates and timestamps** — are stored as strings inside VARIANT, which is slower and larger. Practical rule: query raw JSON in VARIANT during exploration and in the raw layer, but **cast important fields to typed columns** in the silver layer.

Querying VARIANT

```
CREATE OR REPLACE TABLE bytemart_lab.raw.click_events (event VARIANT);

INSERT INTO bytemart_lab.raw.click_events
SELECT PARSE_JSON(column1) FROM VALUES
 ('{"event":"add_to_cart","ts":"2026-03-02T14:01:55Z","user":{"id":4211,"logged_in":true},
    "product":{"sku":"CHG-USBC-65W","qty":2},"experiment":["new_checkout_v2"]}'),
 ('{"event":"page_view","ts":"2026-03-02T14:00:10Z","user":{"id":4211,"logged_in":true},
    "page":"/chargers","referrer":null}');

SELECT event:event::STRING             AS event_type,
       event:ts::TIMESTAMP_TZ          AS event_ts,
       event:user.id::NUMBER           AS user_id,
       event:product.sku::STRING       AS sku,
       event:product.qty::NUMBER       AS qty,
       event:experiment[0]::STRING     AS first_experiment,
       event:referrer                  AS referrer_raw,
       IS_NULL_VALUE(event:referrer)   AS referrer_is_json_null
FROM bytemart_lab.raw.click_events;
```

Line by line:

- `PARSE_JSON(text)` converts a JSON string to VARIANT. (`INSERT … VALUES` cannot call functions on literal VARIANTs directly in every form, so `INSERT … SELECT … FROM VALUES` is the standard pattern.)
- `event:event` uses the **colon** to access a top-level key; `event:user.id` uses **dot notation** for nested keys; `[0]` indexes arrays (zero-based).
- `::STRING`, `::NUMBER`, `::TIMESTAMP_TZ` **cast** VARIANT values to SQL types. Without a cast, strings display with double quotes (`"add_to_cart"`).

- A missing path (`event:product.sku` for the page view) returns **SQL NULL**.

SQL NULL versus JSON null

These are different:

Situation	Meaning	Test
Key absent (`page` in the cart event)	SQL NULL	`IS NULL`
Key present with JSON `null` (`"referrer": null`)	A VARIANT holding a JSON null value	`IS_NULL_VALUE(x)`

A JSON null *is a value*: it equals itself in comparisons and is not SQL NULL. When loading JSON, the file format option `STRIP_NULL_VALUES = TRUE` removes keys whose value is JSON null, so they become absent instead. Chapter 8 continues with `FLATTEN`, `LATERAL`, object construction and schema detection.

Structured types

Structured `ARRAY`, `OBJECT` and `MAP` declare their element types:

```
CREATE OR REPLACE TABLE bytemart_lab.silver.customer_profile (
    customer_id NUMBER,
    address     OBJECT(street VARCHAR, city VARCHAR, postcode VARCHAR),
    phones      ARRAY(VARCHAR),
    preferences MAP(VARCHAR, VARCHAR)
);
```

Structured types give type safety and are the natural mapping for Iceberg `struct`, `list` and `map`. Limitations at the research cutoff include: not supported in dynamic, hybrid or external tables; at most 1,000 sub-columns per structured column; MAP keys must be VARCHAR or integer and not NULL; `TRY_CAST` is not supported with structured types; structured values cannot be nested inside semi-structured constructors; not supported in JavaScript UDFs and procedures. The documentation page does not state a release status.

When to use which: VARIANT for raw, evolving or unknown structure; structured types for stable nested structures you want type-checked; flat typed columns for anything frequently filtered, joined or aggregated.

7.7 Other types you will meet

Type	Purpose	Notes	Chapter
`GEOGRAPHY`	Points, lines, polygons on the Earth's surface (WGS 84)	Spatial functions (distance, containment)	8
`GEOMETRY`	Planar (projected) coordinates		8
`VECTOR(INT|FLOAT, n)`	Fixed-length numeric vectors, up to 4,096 dimensions; elements are 32-bit	Embeddings for similarity search; not in VARIANT, clustering keys, Iceberg tables, Snowpipe or Snowflake Scripting; load as ARRAY then cast	23, 25, 29
`FILE`	A reference to a staged file (internal or external) with metadata	Created with `TO_FILE`; used by Cortex AI functions on documents, images and audio	25

Type	Purpose	Notes	Chapter
`UUID`	128-bit identifier, displayed as 36-character hyphenated text	`UUID_STRING()` generates values; does **not** guarantee uniqueness by itself; not supported in hybrid tables, Snowpark or non-SQL UDFs	9
`PERIOD`	A range between two date/time points	GA (release 10.32, Sept 2026) — useful for migrations from systems that have period types	35
User-defined types	Named aliases of existing types: `CREATE TYPE age AS NUMBER(3,0)`	Coerce to the base type in expressions; must be dropped and recreated to change	9
`UNKNOWN`	Iceberg v3 type for columns whose type is not yet known; always reads as NULL	Iceberg tables only	37

7.8 Type conversion

Explicit conversion

Form	Behaviour on bad input
`CAST(x AS type)`, `x::type`	Error
`TO_NUMBER`, `TO_DATE`, `TO_TIMESTAMP_*`, `TO_BOOLEAN`, `TO_VARCHAR`/`TO_CHAR`	Error; accept format arguments
`TRY_CAST(x AS type)`, `TRY_TO_NUMBER`, `TRY_TO_DATE`, `TRY_TO_TIMESTAMP`…	**NULL** instead of an error

```
SELECT TRY_TO_DATE('2026-02-31')              AS bad_date,       -- NULL
       TRY_TO_NUMBER('12,499')                AS needs_format,   -- NULL
       TRY_TO_NUMBER('12,499', '99,999')      AS with_format,    -- 12499
       TO_DATE('02/03/2026', 'DD/MM/YYYY')    AS uk_style,       -- 2026-03-02
       TRY_CAST('abc' AS NUMBER)              AS not_a_number;   -- NULL
```

VALIDATE, DON'T SILENTLY NULL

`TRY_` functions are ideal in the raw → silver step, but a NULL produced by a failed conversion is *data loss* if nobody notices. Pair them with a quality check: count rows where the source value is not NULL but the converted value is NULL, and quarantine or alert (Chapter 19).

Implicit conversion (coercion)

Snowflake converts types automatically when necessary and possible — adding an integer to a float, comparing a string literal to a date column (`order_date = '2026-03-02'`). Implicit conversion is convenient but can surprise:

- Comparing a `VARCHAR` column to a number converts the strings to numbers — any non-numeric string causes an error at run time.
- Concatenating numbers with `||` converts them to text using default formatting.

When in doubt, cast explicitly.

Precision and scale in arithmetic

Multiplication adds scales (`NUMBER(10,2) × NUMBER(10,2)` produces scale 4); division results can carry more decimal places than the inputs. Snowflake caps results at 38 digits of precision; an intermediate result that exceeds it raises an out-of-range error (one of the problems DECFLOAT addresses). **Round explicitly** where the business defines rounding (`ROUND(amount * fx_rate, 2)`), and do it at a consistent step so totals reconcile.

7.9 NULL semantics in Snowflake

Everything in Chapter 3's NULL section applies. Snowflake-specific points:

- `NOT NULL` is **enforced** on all table types.
- `NVL`, `NVL2`, `ZEROIFNULL`, `NULLIFZERO`, `IFNULL`, `EQUAL_NULL(a, b)` (NULL-safe equality) and `IS DISTINCT FROM` are available.
- Default NULL ordering is controlled by `DEFAULT_NULL_ORDERING` (by default NULLs sort as the largest values).
- In VARIANT, distinguish SQL NULL from JSON null (Section 7.6).
- In joins, `NULL = NULL` is not TRUE; to join on nullable keys, use `EQUAL_NULL(a.k, b.k)` or `a.k IS NOT DISTINCT FROM b.k` — but question why join keys are NULL first.

7.10 Constraints and enforcement

Snowflake supports five ANSI constraint types. Enforcement depends on the table type:

Constraint	Standard tables	Hybrid tables
`PRIMARY KEY`	Optional, **not enforced**	**Required**, enforced
`UNIQUE`	Optional, not enforced	Optional, enforced
`FOREIGN KEY`	Optional, not enforced	Optional, enforced
`NOT NULL`	Optional, **enforced**	Optional, enforced
`CHECK`	Optional, **enforced** (INSERT, UPDATE, MERGE, CTAS); a `COPY INTO` into a table with CHECK constraints fails	Optional, enforced; can be defined only at table creation

Why are keys not enforced on standard tables? Enforcing uniqueness on every load would require checking each new row against billions of existing rows, defeating the bulk-loading and columnar design. Instead:

- unenforced keys **document** the model for people, BI tools and AI (Cortex Analyst and semantic layers use them);
- with the `RELY` property, the optimizer may use declared constraints for optimisations such as join elimination — **only declare `RELY` when your pipeline truly guarantees the constraint**, or results can be wrong;
- **your pipeline enforces** uniqueness (de-duplicate, MERGE by key, test).

```
CREATE OR REPLACE TABLE bytemart_lab.silver.order_lines (
    order_id    NUMBER  NOT NULL,
    line_no     NUMBER  NOT NULL,
    product_id  NUMBER  NOT NULL,
    quantity    NUMBER  NOT NULL,
    unit_price  NUMBER(10,2) NOT NULL,
    CONSTRAINT pk_order_lines PRIMARY KEY (order_id, line_no),
    CONSTRAINT chk_qty_positive CHECK (quantity > 0)
);

INSERT INTO bytemart_lab.silver.order_lines VALUES (1001, 1, 101, 2, 899.00);
INSERT INTO bytemart_lab.silver.order_lines VALUES (1001, 1, 101, 2, 899.00);  -- succeeds:
duplicate PK!
INSERT INTO bytemart_lab.silver.order_lines VALUES (1002, 1, 102, 0, 12999.00); -- fails:
CHECK
INSERT INTO bytemart_lab.silver.order_lines VALUES (1003, 1, NULL, 1, 4499.00);
-- fails: NOT NULL
```

Expected: the second insert succeeds (now two identical rows); the third fails with a CHECK-constraint violation; the fourth fails with a NOT NULL violation.

7.11 Storage implications

Choice	Storage / performance effect
Scale of NUMBER	Higher scale uses more storage; precision does not
VARCHAR length declaration	No storage effect
Dates as strings	Larger, slower, poor pruning — use DATE/TIMESTAMP
JSON kept in VARIANT	Efficient for regular JSON with native types; dates/timestamps inside are strings — cast to typed columns in silver
Wide tables	Not a problem in columnar storage; queries pay only for referenced columns
Heavily updated tables	Each change rewrites micro-partitions; old versions retained for Time Travel (and Fail-safe for permanent tables) → more storage
Time Travel retention	Longer retention = more historical storage
Transient vs permanent	Transient has no Fail-safe storage
Low-cardinality columns	Compress extremely well

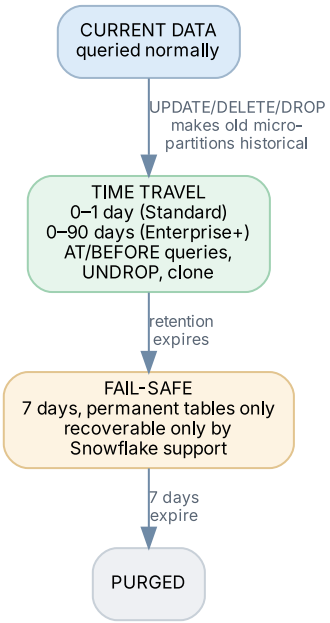


Figure 7.1 — Data lifecycle in storage: current data, Time Travel, Fail-safe, purge. Historical micro-partitions are billed as storage while retained.

7.12 Table types

Permanent, transient and temporary

Property	Permanent (default)	Transient	Temporary
Lifetime	Until dropped	Until dropped	Until the session ends
Visible to	Anyone with privileges	Anyone with privileges	Only the creating session
Time Travel	0–1 day (Standard), 0–90 days (Enterprise+)	0 or 1 day	0 or 1 day
Fail-safe	7 days	None	None
Typical use	Business data that must be re-coverable	Staging and intermediate data that can be rebuilt	Scratch work within a session or procedure

```
CREATE TRANSIENT TABLE bytemart_lab.raw.stg_orders_batch (payload VARIANT, loaded_at
TIMESTAMP_LTZ);
CREATE TEMPORARY TABLE tmp_my_analysis AS SELECT 1 AS x;
CREATE TRANSIENT SCHEMA bytemart_lab.scratch;   -- tables created inside default to transient
```

A TEMPORARY TABLE CAN SHADOW A PERMANENT TABLE

If you create a temporary table with the same name as an existing table in the same schema, your session sees the temporary one. This can make queries mysteriously return the wrong data. Use distinct names (`tmp_` prefix).

External tables

An **external table** is a read-only table over files in an external stage. Snowflake stores only metadata; each row exposes the file content in a VARIANT column called `VALUE`, plus pseudo-columns such as `METADATA$FILENAME`. You can define typed virtual columns and partition columns derived from file paths, and keep metadata current with `AUTO_REFRESH`. External tables suit occasional queries over data that must remain in the lake; for frequent querying, load the data or use Iceberg tables.

Apache Iceberg tables

An **Iceberg table** stores data in Apache Iceberg format (Parquet data files plus Iceberg metadata) so that other engines — Spark, Trino, Flink, Databricks — can read (and in many configurations write) the same tables. Variants:

- **Snowflake as the catalog** (`CATALOG = 'SNOWFLAKE'`): Snowflake manages metadata and writes; data is stored either in **your cloud storage** through an **external volume**, or — GA since 1 June 2026 — in **Snowflake-managed storage**. External engines can read (and, since 2026, write) through the **Horizon Catalog** Iceberg REST interface.
- **External catalog** (AWS Glue, Snowflake Open Catalog/Polaris, Unity Catalog, S3 Tables, BigLake Metastore and other Iceberg REST catalogs) via a **catalog integration**: another system manages the table; Snowflake queries it and, for supported catalogs, writes to it.

```
-- Snowflake-catalog Iceberg table in your storage (external volume created by an admin)
CREATE ICEBERG TABLE bytemart_lab.silver.click_events_iceberg (
    event_ts   TIMESTAMP_NTZ(6),
    user_id    NUMBER(38,0),
    event_type VARCHAR,
    sku        VARCHAR
)
  CATALOG = 'SNOWFLAKE'
  EXTERNAL_VOLUME = 'bytemart_lake_vol'
  BASE_LOCATION = 'clickstream/click_events';
```

Shown for recognition; it requires an external volume (Chapter 37). Iceberg v3 is supported (GA May 2026). Some Snowflake types are not available in Iceberg tables (for example DECFLOAT, VARIANT inside structured types, VECTOR), and Fail-safe does not apply to data in your own storage — your cloud storage's own protection applies.

Hybrid tables

A **hybrid table** is designed for **low-latency, high-concurrency, index-based random reads and writes** — operational lookups and updates — while remaining queryable with Snowflake SQL alongside standard tables.

- The **row store** is the primary data store; data is copied asynchronously to object storage, and some data may be cached in columnar form on the warehouse for analytics. Hybrid tables typically use **more storage** than standard tables.
- A **PRIMARY KEY is required and enforced**; UNIQUE, FOREIGN KEY, NOT NULL and CHECK constraints are enforced; **secondary indexes** can be declared and are maintained synchronously.
- **Row-level locking** (standard tables lock at partition or table level).

- **Availability:** GA in **AWS and Microsoft Azure commercial regions**.

```
CREATE OR REPLACE HYBRID TABLE bytemart_lab.silver.inventory_reservations (
    reservation_id NUMBER       PRIMARY KEY,
    sku            VARCHAR(30)  NOT NULL,
    customer_id    NUMBER       NOT NULL,
    qty            NUMBER       NOT NULL,
    reserved_at    TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP(),
    INDEX idx_sku (sku)
);
```

Use hybrid tables for application state that needs fast point operations near analytical data (reservations, feature lookups, workflow state). Do not use them for large analytical scans. Hybrid tables are also excluded from the result cache. For a full PostgreSQL-compatible operational database, **Snowflake Postgres** (GA February 2026) is a managed PostgreSQL service within Snowflake; Volume 5 compares the options.

Dynamic tables

A **dynamic table** (Chapter 6, 15) is a table whose contents are defined by a query and kept fresh automatically within a **target lag**. You do not insert into it; Snowflake refreshes it, incrementally where possible.

Interactive tables

An **interactive table** is a specialised table type for consistent low-latency queries on **interactive warehouses**. It requires `CLUSTER BY`, supports only `INSERT OVERWRITE` as DML (no `UPDATE`/`DELETE`), has Time Travel but no Fail-safe, and does not support streams. Snowflake recommends standard tables as the default; use interactive tables only when you need the most consistent latency and can accept these restrictions.

Choosing a table type

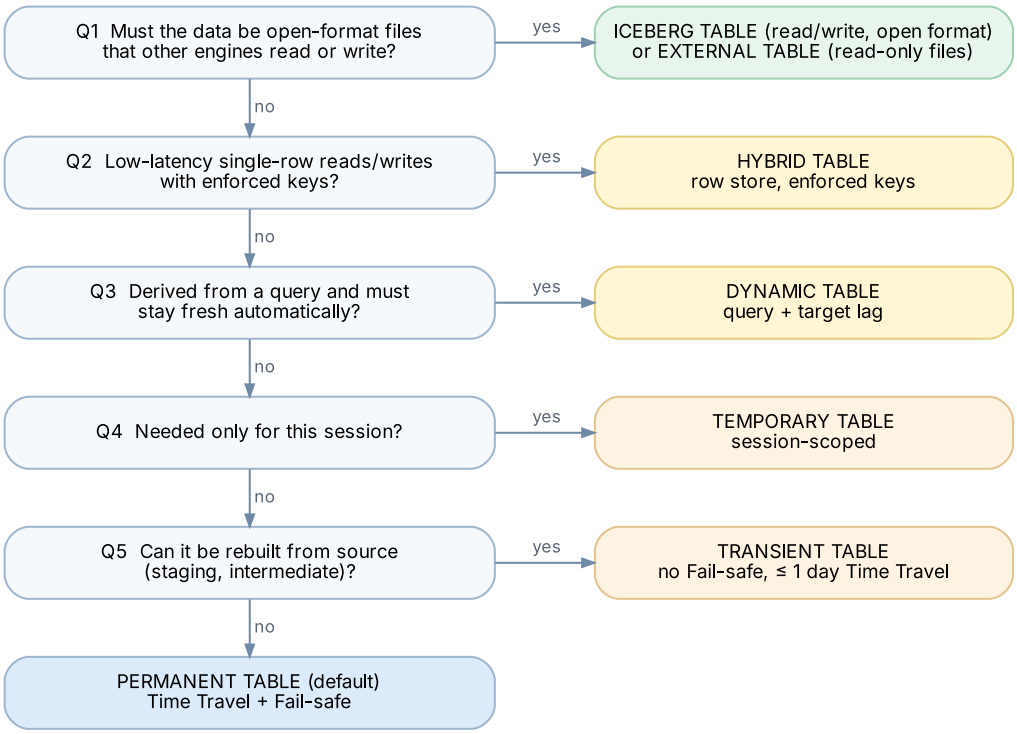


Figure 7.2 — Choosing a table type. Start from where the data must live and how it will be accessed.

7.13 Guided lab: types in practice

LAB 7.1 — TYPES, TIME ZONES, VARIANT AND CONSTRAINTS

Environment: Snowsight, `lab_role`, `lab_wh`, `bytemart_lab`. Enterprise edition not required.

Step 1 — Numeric precision. Run:

```
SELECT 0.1::FLOAT + 0.2::FLOAT = 0.3::FLOAT                    AS float_equal,
       0.1::NUMBER(10,2) + 0.2::NUMBER(10,2) = 0.3::NUMBER(10,2) AS number_equal,
       SYSTEM$TYPEOF(899.00::NUMBER(10,2) * 2::NUMBER(10,2))     AS product_type;
```

Expect `float_equal = FALSE`, `number_equal = TRUE`, and a result type with scale 4.

Step 2 — Time zones. Run the time-zone example in Section 7.5 twice — once with `ALTER SESSION SET TIMEZONE = 'Asia/Kolkata'` and once with `'Europe/London'` — and explain which columns change.

Step 3 — VARIANT. Create and query `raw.click_events` (Section 7.6). Then add an event with an extra nested field and confirm the query still works without any schema change.

Step 4 — Constraints. Run the order-lines constraint example (Section 7.10). Then write a query that finds duplicate primary keys:

```
SELECT order_id, line_no, COUNT(*) AS copies
FROM bytemart_lab.silver.order_lines
GROUP BY order_id, line_no
HAVING COUNT(*) > 1;
```

Step 5 — Retention. Create one permanent and one transient table and compare their `retention_time` and kind:

```
CREATE OR REPLACE TABLE bytemart_lab.silver.t_permanent (x NUMBER);
CREATE OR REPLACE TRANSIENT TABLE bytemart_lab.silver.t_transient (x NUMBER);
SHOW TABLES LIKE 'T_%' IN SCHEMA bytemart_lab.silver;
SELECT "name", "kind", "retention_time" FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
```

Step 6 — TRY functions. Build a small table of messy strings (`'12,499'`, `'abc'`, `'2026-02-31'`, `'2026-03-02'`) and convert them with `TRY_TO_NUMBER` / `TRY_TO_DATE`; count how many failed conversions a quality check would flag.

7.14 Production use case: typing ByteMart's silver layer

ByteMart's raw layer receives orders from PostgreSQL (typed), products from CSV (all text) and clickstream from JSON (VARIANT). The silver-layer typing rules adopted:

Data	Rule
Money	`NUMBER(12,2)` in the transaction currency, plus a currency code column; converted amounts rounded to 2 decimals at the conversion step

Data	Rule
Quantities	`NUMBER(38,0)`
Identifiers from sources	Keep source type (number or text); never convert account numbers or SKUs with leading zeros to numbers
Event times	`TIMESTAMP_TZ` at ingestion; also a `*_utc` `TIMESTAMP_NTZ` derived column for partition-friendly filtering and joins
Business dates	`DATE`
Flags	`BOOLEAN`
Clickstream	Raw VARIANT retained in bronze; top 15 fields cast to typed columns in silver; remainder kept in a VARIANT `attributes` column
Embeddings (later)	`VECTOR(FLOAT, n)` matching the embedding model's dimension
Staging tables	Transient
Inventory reservations for the website	Hybrid table (AWS region)
Clickstream for the data-science team using Spark	Iceberg table

Every conversion in raw → silver uses `TRY_` functions **plus** a failed-conversion count check that blocks publication above a threshold.

7.15 Common mistakes

TYPE AND TABLE MISTAKES

1. FLOAT for money.
2. Dates and timestamps stored as VARCHAR.
3. Mixing time zones in one NTZ column.
4. Converting identifiers with leading zeros to numbers.
5. Leaving everything in VARIANT forever.
6. Confusing JSON null with SQL NULL.
7. Trusting PRIMARY KEY to prevent duplicates on standard tables.
8. Declaring `RELY` on constraints the pipeline does not guarantee.
9. Using permanent tables for re-creatable staging data (paying for Fail-safe).
10. Using hybrid tables for analytics, or standard tables for high-rate single-row updates.

7.16 Troubleshooting exercise

EXERCISE — YESTERDAY'S ORDERS ARE MISSING... OR DOUBLED

ByteMart's daily report filters `WHERE order_ts::DATE = CURRENT_DATE() - 1`. Indian managers say some late-evening orders are missing; UK managers say some appear on the wrong day. `order_ts` is `TIMESTAMP_NTZ` loaded from PostgreSQL, where it was `timestamptz`. Diagnose the likely problems and propose a fix. (Solution at the end of the chapter.)

7.17 Knowledge check

1. Do `INT` and `BIGINT` differ in Snowflake? What does `NUMBER(20,2)` allow left of the decimal point?
2. Does declaring `VARCHAR(50)` instead of `VARCHAR` save storage?
3. Explain NTZ, LTZ and TZ in one sentence each. What does TZ *not* remember?
4. How do you access `user.id` in a VARIANT column called `event`, and why should you cast it?
5. How do you distinguish a JSON null from a missing key?
6. When would you choose DECFLOAT over NUMBER?
7. Which constraints are enforced on standard tables? On hybrid tables?
8. What is the difference between transient and temporary tables?
9. Give one reason to choose an Iceberg table and one limitation.
10. When are hybrid tables appropriate, and on which clouds are they GA?

7.18 Hands-on assignment

1. Complete Lab 7.1 and record all results.
2. Design typed silver tables for ByteMart's products (from CSV) and clickstream (from JSON), with column comments stating time-zone conventions and units.
3. Write a one-page "type standards" document for your team, including at least eight rules with rationale.

7.19 Chapter summary

Snowflake supports exact numerics (`NUMBER`, `DECFLOAT`), double-precision `FLOAT`, Unicode strings with optional collation, `BOOLEAN`, dates, times and three timestamp variants, semi-structured `VARIANT` / `OBJECT` / `ARRAY`, structured types, and specialised types for geospatial data, vectors, files, UUIDs and periods. Type choices affect correctness, pruning and storage: use exact numerics for money, proper date types for dates, explicit time-zone conventions for timestamps, and cast important JSON fields to typed columns. Conversion is explicit or implicit; `TRY_` functions avoid errors but must be paired with checks. On standard tables only `NOT NULL` and `CHECK` are enforced, so pipelines must enforce uniqueness; hybrid tables enforce keys. Table kinds trade durability,

scope and capability: permanent, transient and temporary tables differ in Time Travel and Fail-safe; external and Iceberg tables integrate with the lake; hybrid tables serve operational access; dynamic tables maintain query results; interactive tables serve low-latency workloads.

7.20 Key takeaways

- Money → `NUMBER(p,s)`; never FLOAT.
- One time-zone convention per column, documented and enforced at ingestion.
- VARIANT for raw and evolving data; typed columns for what you filter and join.
- Only `NOT NULL` and `CHECK` are enforced on standard tables.
- Choose the table type from where data lives, how it is accessed and how recoverable it must be.

7.21 Further reading

- Summary of data types: https://docs.snowflake.com/en/sql-reference/intro-summary-data-types
- Numeric data types: https://docs.snowflake.com/en/sql-reference/data-types-numeric
- Date & time data types: https://docs.snowflake.com/en/sql-reference/data-types-datetime
- Semi-structured data types: https://docs.snowflake.com/en/sql-reference/data-types-semistructured
- Structured data types: https://docs.snowflake.com/en/sql-reference/data-types-structured
- Vector data types: https://docs.snowflake.com/en/sql-reference/data-types-vector
- UUID data type: https://docs.snowflake.com/en/sql-reference/data-types-uuid
- Data type conversion: https://docs.snowflake.com/en/sql-reference/data-type-conversion
- Constraints overview: https://docs.snowflake.com/en/sql-reference/constraints-overview
- Working with temporary and transient tables: https://docs.snowflake.com/en/user-guide/tables-temp-transient
- Hybrid tables: https://docs.snowflake.com/en/user-guide/tables-hybrid
- Apache Iceberg tables: https://docs.snowflake.com/en/user-guide/tables-iceberg
- Interactive tables and warehouses: https://docs.snowflake.com/en/user-guide/interactive

7.22 Solutions

Troubleshooting — missing or doubled orders. Likely causes: (1) PostgreSQL `timestamptz` values were extracted and loaded into `TIMESTAMP_NTZ` **without a defined zone convention** — perhaps in UTC wall-clock time. An order at 23:30 IST on 1 March is 18:00 UTC on 1 March (fine), but one at 02:00 IST on 2 March is 20:30 UTC on 1 March — counted on the wrong Indian day. (2) `CURRENT_DATE()` is evaluated in the **session time zone** (by default America/Los_Angeles unless changed), so "yesterday" itself shifts depending on who runs the report and where the scheduler runs. (3) Different extracts may have used different zones, creating a mix. **Fix:** load event times as `TIMESTAMP_TZ` (or document NTZ as UTC and convert explicitly at ingestion with `CONVERT_TIMEZONE`); define each report's business zone explicitly — for India,

`CONVERT_TIMEZONE('UTC','Asia/Kolkata', order_ts_utc)::DATE = DATEADD(day, -1, CONVERT_TIMEZONE('Asia/Kolkata', CURRENT_TIMESTAMP())::DATE)` — and set the scheduler's session `TIMEZONE` explicitly. Back-fill historical rows once the original zone convention is confirmed.

Knowledge check answers.

1. No — both are `NUMBER(38,0)`. Eighteen digits (20 – 2).
2. No; the length is a constraint, not a storage size.
3. NTZ: wall-clock time with no zone. LTZ: an instant in UTC, shown in the session zone. TZ: an instant with the offset it was recorded in. TZ does not remember the time-zone *name* (and therefore not DST rules).
4. `event:user.id`; casting (`::NUMBER`) gives a proper SQL type for comparisons, joins and aggregation, and removes JSON quoting for strings.
5. A missing key returns SQL NULL (`IS NULL`); a JSON null returns a VARIANT null (`IS_NULL_VALUE(...)` is TRUE).
6. When the scale is unknown or highly variable, when exact results must avoid fixed-scale overflow (ledgers, taxes), or when migrating from decimal floating-point systems.
7. Standard: NOT NULL and CHECK. Hybrid: PRIMARY KEY (required), UNIQUE, FOREIGN KEY, NOT NULL, CHECK.
8. Transient tables persist until dropped and are visible to others, with ≤1 day Time Travel and no Fail-safe; temporary tables exist only for the creating session.
9. Reason: open format readable/writable by other engines without copying. Limitation (one of): some Snowflake types unsupported; requires catalog/volume setup; Fail-safe does not cover data in your storage.
10. For low-latency, high-concurrency point reads and writes with enforced keys (operational state near analytics); GA in AWS and Azure commercial regions.